DESIGN AND EVALUATION OF AN ARABIC-AWARE PROMPT COMPRESSION SELECTOR FOR LARGE LANGUAGE MODELS

By

Badr Abouabdou

A DISSERTATION

Submitted to

The University of Liverpool

in partial fulfilment of the requirements
for the degree of

MASTER OF SCIENCE

02/11/2026

DECLARATION

I hereby certify that this dissertation constitutes my own product, that where the language of others is set forth, quotation marks so indicate, and that appropriate credit is given where I have used the language, ideas, expressions, or writings of another.

I declare that the dissertation describes original work that has not previously been presented for the award of any other degree of any institution.

Signed,



Student, Supervisors and Classes:

Student name:	Badr Abouabdou
Student ID number:	30026
DL name:	Andrea Corradini
DA name:	Laud Charles Ochei

DESIGN AND EVALUATION OF AN ARABIC-AWARE PROMPT COMPRESSION SELECTOR FOR LARGE LANGUAGE MODELS

A DISSERTATION

Submitted to

The University of Liverpool

in partial fulfilment of the requirements
for the degree of

MASTER OF SCIENCE

02/11/2026

ABSTRACT

DESIGN AND EVALUATION OF AN ARABIC-AWARE PROMPT COMPRESSION SELECTOR FOR LARGE LANGUAGE MODELS

LLM APIs charge per token, and their tokenisers split Arabic words into far more tokens than English ones, so Arabic users pay more for the same content. Prompt compression can cut this cost, but existing methods were built and tested on English. This dissertation asks how well they work on Arabic, which prompt features drive their performance, and whether a recommender can choose the best strategy for each prompt.

Following design science, the project built AraPromptBench (1,000 Arabic prompts from six public corpora plus synthetic creative prompts) and a 400-prompt fresh exam set. It compared LLMLingua, LLMLingua-2 and random deletion at three rates on gpt-5.6-luna, a commercial model whose output tokens cost six times its input tokens, over more than 17,000 API calls. Fidelity was measured on the LLM's answers with AraBERT BERTScore, against a threshold derived from the model's own repeat-call noise.

LLMLingua-2 was the best compressor, beating random deletion by 0.04–0.05 F1 at every rate. Prompt-level similarity ranked the methods backwards. Task type and length explained outcomes; morphology did not. The artefact, the Arabic-Aware Prompt Compression Selector (APCS), reached 31% accuracy as a rule-based selector. A decision-tree redesign reached 38.5% on the pre-registered exam, significantly above the best fixed strategy (Holm p = 0.028). Unexpectedly, compression raised the total bill by 5–24%, because it deleted answer-length instructions and answers grew. Protecting those instructions turned the penalty into a 5–8% saving. Arabic prompt compression must therefore be judged on outputs and total cost, not tokens saved.

ACKNOWLEDGEMENTS

I am grateful to my Dissertation Advisor (DA), Dr. Laud Charles Ochei, for the guidance, constructive feedback and encouragement given throughout this project, and for questions that sharpened its methodology, including the suggestion to test the effect of synthetic data. I also thank the Dissertation Lead (DL), Dr. Andrea Corradini, for overseeing the dissertation process and for the support provided throughout the module.

This project uses publicly available datasets released by their authors: CIDAR, the Aya dataset, XL-Sum, the Essex Arabic Summaries Corpus, TyDi QA and ARCD. Their licences are recorded per prompt in AraPromptBench.

Generative AI tools were used in this project in two ways. First, to generate the synthetic datasets: the 500-prompt pilot set (gpt-5.4-mini), the v2 creative prompts and the Experiment 09 probe pool (Claude Fable 5), and the exam creative briefs (Claude Opus 5.5). These are identified as synthetic throughout, and their effect was measured (Experiment 09). Second, as a coding assistant for the experiment pipeline and the APCS package. The author directed this work throughout: the author defined the research questions, made all design and methodological decisions, oversaw every stage of the pipeline, and reviewed and revised every piece of code before it was used. All analyses and conclusions are the author's own, and every reported number was independently verified against the raw data.

TABLE OF CONTENTS

Page

LIST OF TABLES viii

LIST OF FIGURES ix

Chapter 1. Introduction 1

1.1 Chapter Introduction 1

1.2 Problem Statement 1

1.3 Project Aims and Objectives 3

1.4 Approach 4

1.5 Outcome 5

1.6 Chapter Summary 5

Chapter 2. Background and Review of Literature 6

2.1 Chapter Introduction 6

2.2 Research Background 6

2.3 Literature Review 7

2.3.1 Prompt compression methods 7
2.3.2 Adaptive and cost-aware LLM use 10
2.3.3 Multilingual tokenisation and morphology 11
2.3.4 Arabic NLP and LLM evaluation 12
2.3.5 Evaluating compressed prompts 13
2.3.6 The research gap 15

2.4 Theory 15

2.5 Terminology 16

2.6 Chapter Summary 18

Chapter 3. Analysis and Design 19

3.1 Chapter Introduction 19

3.2 Project Specification 19

3.3 Research Methods 20

3.4 Design Considerations 22

3.4.1 Benchmark design 22
3.4.2 Compression candidates 24
3.4.3 LLM under test and calling protocol 25
3.4.4 Fidelity measurement 25
3.4.5 Labels and metrics 26
3.4.6 APCS architecture 27
3.4.7 Evaluation plan and hypotheses 28

3.5 Chapter Summary 28

Chapter 4. Implementation (Realisation) 29

4.1 Chapter Introduction 29

4.2 Details of the Implementation 29

4.2.1 Environment and tooling 29
4.2.2 Pilot phase and the design changes it triggered 30
4.2.3 Building AraPromptBench v2 31
4.2.4 The benchmark pipeline 32
4.2.5 Calibrating APCS 1.0.0 33
4.2.6 The Experiment 11 redesign 33
4.2.7 Changes from the Specification and Design Report 35

4.3 The IT Artefact 36

4.4 Chapter Summary 38

Chapter 5. Results and Evaluation 39

5.1 Chapter Introduction 39

5.2 Evaluation of IT Artefact 39

5.2.1 Measurement validity: the noise ceiling 39
5.2.2 RQ1: effectiveness of compression methods on Arabic 40
5.2.3 Category fragility and task accuracy 43
5.2.4 RQ2: which prompt features matter 44
5.2.5 The cost finding: fewer input tokens, a bigger bill 46
5.2.6 RQ3 (stage 1): APCS 1.0.0 on the held-out test split 48
5.2.7 RQ3 (stage 2): the redesigned selectors on the fresh exam 49
5.2.8 The length-protected compressor 52
5.2.9 Synthetic data sensitivity 53
5.2.10 Robustness and verification 54
5.2.11 Answering the research questions 54

5.3 Chapter Summary 55

Chapter 6. Conclusions 56

6.1 Chapter Introduction 56

6.2 Lessons Learned 56

6.3 Strengths and Weakness of the Project 58

6.4 Academic Application and Limitations 60

6.5 Business Application and Limitations 61

6.6 Recommendations / Prospects for Future Research / Work 62

6.7 Chapter Summary 63

REFERENCES CITED 64

APPENDICES 70

LIST OF TABLES

Page

Table 1. cl100k_base fragmentation of Arabic clitic words compared with their English equivalents 7
Table 2. Prompt compression alternatives evaluated against the project's requirements 9
Table 3. Project components and the research questions they serve 19
Table 4. AraPromptBench v2 and exam set composition 22
Table 5. AraPromptBench v2 prompts by task category and length band (cl100k tokens) 23
Table 6. Implementation environment 29
Table 7. Pilot experiments and the design decisions they produced 31
Table 8. Experimental pipeline: stages, scripts and outputs 32
Table 9. Changes from the SDR and their justification 35
Table 10. Structure of the APCS package 36
Table 11. Repeat-call noise ceiling by task category (dev, AraBERT) 40
Table 12. Output-level fidelity of LLMLingua-2 vs random deletion (dev, n = 800, paired) 41
Table 13. OLS model of LLMLingua-2 @ 0.5 output fidelity (dev, $R^2 = 0.148$) 45
Table 14. Input-token saving vs total measured cost change (dev, output priced at 6× input) 46
Table 15. Selector and baseline performance on the held-out test split (n = 200) 48
Table 16. Pre-registered hypotheses on the fresh exam (n = 400; exact two-sided McNemar for H1 and H2, conservative for directional hypotheses) 50
Table 17. All policies on the fresh exam (n = 400) 50
Table 18. Plain vs length-protected LLMLingua-2 on exam prompts with a length instruction (n = 180) 52
Table 19. Answers to the research questions 55
Table 20. SDR project plan and outcomes 57
Table 21. APCS feature definitions 95
Table 22. Worked example of plain vs protected compression (exam-qa-043) 97
Table 23. Experiment log summary 98

LIST OF FIGURES

Page

Figure 1. Architecture of the APCS artefact and the evaluation pipeline that calibrates it 27
Figure 2. Repeat-call noise ceiling on the dev set: distribution of output F1 between two answers to the identical prompt, with the SDR threshold (0.85) and the adopted τ (0.65) 40
Figure 3. Compression trade-off by method on the dev set: mean TCR against mean output F1 at target rates 0.7, 0.5 and 0.3 41
Figure 4. The ranking flip: LLMLingua-2 minus random deletion at prompt level (negative) and output level (positive) 42
Figure 5. Spearman correlation between prompt features and output fidelity at target rate 0.5 (dev) 44
Figure 6. Input tokens saved vs total API bill saved per policy (dev; negative bill saving = costs more) 47
Figure 7. Confusion matrix of APCS 1.0.0 recommendations against best-balance labels (test) 49
Figure 8. Best-balance accuracy of each policy on the fresh exam, with 95% bootstrap confidence intervals 50
Figure 9. Total API cost relative to the uncompressed prompt, plain vs protected LLMLingua-2 (exam prompts with a length instruction) 52
Figure 10. Fidelity difference between synthetic and real prompts by category and rate, with 95% bootstrap intervals 53

Chapter 1. INTRODUCTION

1.1 Chapter Introduction

Arabic prompts cost more to send to a large language model (LLM) than English prompts with the same content, because commercial tokenisers trained mainly on English split Arabic words into many more billable sub-word tokens (Ahia et al., 2023; Petrov et al., 2023). Prompt compression, which shortens a prompt before it is sent, is the most practical remedy for developers who use commercial APIs (Li et al., 2025). Compression methods, however, have been designed and evaluated almost exclusively on English, so whether they work on Arabic, and how they should be evaluated there, is unknown.

This dissertation evaluates existing compression methods on Arabic and builds a lightweight recommender that chooses, for each Arabic prompt, the strategy that best balances token savings against fidelity. Its contributions are AraPromptBench, an Arabic benchmark for prompt compression; an empirical analysis of compression behaviour on Arabic; and the Arabic-Aware Prompt Compression Selector (APCS), a Python artefact that recommends a compression strategy.

1.2 Problem Statement

Practitioners who deploy LLMs for Arabic users face two unresolved problems, first identified in the project proposal and Specification and Design Report (SDR) (Appendices A and B).

The first is a **measurement problem**. Leading compression methods such as LLMLingua (Jiang et al., 2023) and LLMLingua-2 (Pan et al., 2024) report large reductions in prompt length with little loss of task performance, but their evidence comes from English benchmarks and English-trained scoring models. Arabic differs from English in ways that plausibly matter for compression: it is morphologically rich, attaches clitics such as conjunctions, prepositions and pronouns to words, and is written in a script that byte-level tokenisers

fragment heavily (Habash, 2010). Whether methods that decide which tokens are "unimportant" in English make sensible decisions in Arabic is an open empirical question.

The second is a **selection problem**. Even if compression helps on average, no single method or compression rate is likely to be best for every prompt. Short questions may tolerate no compression at all, while long articles to be summarised may tolerate heavy compression. Work on cost-efficient LLM use shows that adaptive, per-query strategies outperform fixed ones (Chen, Zaharia and Zou, 2024; Ding et al., 2024), but this principle has not been applied to choosing a compression strategy, and the prompt characteristics that should drive such a choice for Arabic are unknown.

Both problems are scoped to the setting most Arabic developers face: a closed commercial API that accepts only text. This rules out soft-prompt methods, which replace the prompt with learned vectors fed into the model's embedding layer and therefore need access to model weights (Mu, Li and Goodman, 2023; Ge et al., 2024). Of the text-to-text methods that remain, abstractive rewriting generates a new, shorter prompt with another LLM, adding a model call and a risk of altered content, whereas extractive token deletion keeps a subset of the original tokens and needs only a small local model. This project studies extractive deletion, the family implemented by leading tools such as LLMLingua.

The project addresses both problems with the three research questions fixed in the proposal:

- **RQ1.** How effective are existing prompt compression techniques on Arabic prompts?
- **RQ2.** How do prompt features influence compression performance?
- **RQ3.** Can a recommendation system accurately select the optimal compression strategy?

The proposed solution is an IT artefact, the APCS, that extracts features from an Arabic prompt and recommends a compression method and target rate. Its rules are derived from the benchmark built to answer RQ1 and RQ2.

1.3 Project Aims and Objectives

The aim of the project is to investigate the effectiveness of prompt compression techniques on Arabic prompts, and to design, implement and evaluate an Arabic-Aware Prompt Compression Selector that recommends the most suitable compression strategy for a given Arabic prompt while balancing token reduction against semantic fidelity.

The aim is pursued through five objectives, carried over from the proposal:

1. **Review** the literature on prompt compression, efficient LLM use, multilingual tokenisation, morphology and Arabic natural language processing (NLP) to identify the research gap (Chapter 2).
2. **Design** a benchmarking framework and recommendation architecture, including the structure of AraPromptBench, the prompt features, the evaluation criteria and the APCS decision logic (Chapter 3).
3. **Implement** the system: construct AraPromptBench, integrate the compression methods into a reproducible pipeline and build the APCS as a Python package (Chapter 4).
4. **Evaluate** the compression methods and the APCS against baseline strategies on held-out data, using token compression ratio, output-level semantic fidelity, recommendation accuracy, output-length change and measured total cost (Chapter 5).
5. **Recommend** how Arabic prompt compression should be used in practice, and where it should not (Chapter 6).

In scope are Modern Standard Arabic (MSA) prompts in four task categories (instruction following, summarisation, question answering and creative writing), extractive compression methods that work with a black-box API, and one commercial LLM used with deterministic settings. **Out of scope** are dialectal Arabic beyond a small probe, soft-prompt and abstractive methods, long-context retrieval-augmented generation, and training new compression models.

1.4 Approach

The project follows the design science research paradigm (Hevner et al., 2004): it builds a purposeful artefact, evaluates its utility rigorously, and reports what was learned both about the artefact and about the problem. The work proceeded through eleven documented experiments in four phases.

1. **Pilot (Experiments 01–07).** A 500-prompt, development-only pilot set was used to select the fidelity scorer, test which compressors run correctly on Arabic, measure LLM output variability and build a first APCS.
2. **Benchmark construction (Experiment 08).** AraPromptBench v2 has 1,000 prompts (850 from six public Arabic corpora, 150 synthetic creative prompts), split 800/200 into development and held-out test sets.
3. **Final benchmark (Experiments 09–10).** Every prompt was compressed by each method at three rates and answered by the LLM under test (13,438 API calls); APCS 1.0.0 was calibrated on development data and evaluated once on the test set.
4. **Redesign on a fresh exam (Experiment 11).** An improved selector, a cost-aware selector and a length-protecting compressor were evaluated on a new 400-prompt exam set under pre-registered hypotheses, because the test set had already been used.

Fidelity is BERTScore F1 (Zhang et al., 2020) between the LLM's answers to the compressed and original prompts, judged against a threshold derived from the model's own repeat-call noise; selectors are compared with exact McNemar tests. The set sizes follow the SDR's 80/20 plan and the available corpus data, and their adequacy can be stated directly. Given the observed rate of disagreement between selectors, the 200-prompt test set estimates accuracy within about ±6.4 points and can detect paired differences of about 11 points with 80% power; the 400-prompt exam narrows these to ±4.7 and 7 points.

1.5 Outcome

The project delivered four outcomes:

- AraPromptBench, a documented Arabic benchmark for prompt compression with provenance, licences and fixed splits;
- evidence on how LLMLingua, LLMLingua-2 and random deletion behave on Arabic, and which prompt features explain the differences;
- the APCS, an explainable Python package evaluated against no-compression, fixed-method and random-selection baselines;
- practical guidance for Arabic LLM deployments.

It also found that, on a model whose output tokens cost six times its input tokens, compression raised the total bill, because the compressor deleted the instructions that controlled answer length (Chapters 5 and 6).

1.6 Chapter Summary

Arabic pays a tokeniser tax that prompt compression could reduce, but compression has not been evaluated on Arabic and no method exists for choosing a strategy per prompt. Three research questions address this through a benchmark, an empirical analysis and a selector. Chapter 2 reviews the literature behind them.

Chapter 2. BACKGROUND AND REVIEW OF LITERATURE

2.1 Chapter Introduction

This chapter reviews why token pricing penalises Arabic (Section 2.2), five relevant bodies of work and the gap they leave (2.3), the theory behind the APCS (2.4) and the terms used throughout (2.5).

2.2 Research Background

Commercial LLM APIs bill each request by the number of tokens processed, with separate prices for input (prompt) tokens and output (completion) tokens. Prices differ by up to two orders of magnitude between providers and models (Chen, Zaharia and Zou, 2024). Output tokens are typically several times more expensive than input tokens; for the model used in this project, the ratio is 6:1 (US$0.20 per million input tokens and US$1.20 per million output tokens). Token count therefore determines cost, latency, energy use and how much of the model's context window a request uses (Wan et al., 2024). Inference energy grows with the number of tokens processed and generated (Samsi et al., 2023), so shorter exchanges are also greener.

Tokenisers do not treat languages equally. Ahia et al. (2023) measured OpenAI's API on 22 typologically diverse languages and found that some languages need up to five times more tokens than English for the same content, so speakers of those languages pay more while often receiving worse results. Petrov et al. (2023) found differences of up to fifteen times between language pairs, and showed that even byte-level tokenisers leave large gaps. Arabic is among the disadvantaged languages. In this project the tokeniser of the target API (cl100k_base) produced a mean of 4.1 tokens per Arabic word across the 1,000 AraPromptBench prompts (10th–90th percentile 3.7–4.6). In other words, a typical Arabic word is split into four pieces, each billed separately. Arabic users therefore pay what can fairly be called a *tokeniser tax*.

The mechanism is visible at word level (Table 1). Arabic attaches conjunctions (wa-, fa-), prepositions (bi-), the article (al-), the future marker (sa-) and object pronouns (-hā) to the stem, so one orthographic word can carry a whole English phrase. cl100k_base splits such words almost letter by letter, with only the article ال surviving as a unit: وبالكتاب ("and with the book") becomes six tokens, و|ب|ال|ك|ات|اب, where English needs four, one per word. Morpheme boundaries are therefore not token boundaries, which matters for any compressor that deletes tokens.

Arabic word	Morphemes (gloss)	cl100k tokens	English equivalent	Tokens
وبالكتاب	wa + bi + al + kitāb (and with the book)	6	and with the book	4
فسيكتبونها	fa + sa + yak-tubūna + hā (so they will write it)	10	so they will write it	5
والمعلمون	wa + al + muʿal-limūn (and the teachers)	8	and the teachers	3
بالمدرسة	bi + al + madrasa (at the school)	7	at the school	3

Table 1. cl100k_base fragmentation of Arabic clitic words compared with their English equivalents

Two responses to the tax exist. Model builders can extend the vocabulary with Arabic tokens: ALLaM merged a dedicated Arabic tokeniser into an English model's vocabulary (Bari et al., 2025), and AraLLaMA expanded the vocabulary progressively during training (Zhu et al., 2025). Both remedies are available only to whoever trains the model. Developers who consume a commercial API cannot change its tokeniser; they can only change what they send. For them, prompt compression is the practical lever, and that is the setting this project addresses.

Arabic LLM capability has improved quickly (Section 2.3.4), but efficiency for Arabic has received far less attention than capability; that imbalance is the background to this project.

2.3 Literature Review

2.3.1 Prompt compression methods

Li et al. (2025) survey the field and divide prompt compression into two families. **Soft-prompt** methods compress a prompt into learned continuous vectors. Gisting trains a

model to summarise an instruction into a few "gist" tokens, reaching up to 26-fold compression (Mu, Li and Goodman, 2023). AutoCompressors recursively summarise long documents into summary vectors (Chevalier et al., 2023). The In-context Autoencoder encodes a context into compact memory slots using a lightly fine-tuned copy of the LLM (Ge et al., 2024). These methods compress very effectively but need access to the model's weights or embedding layer. A developer calling a closed commercial API cannot send vectors instead of text, so soft-prompt methods are unusable in the setting of this project.

Hard-prompt methods instead output a shorter natural-language prompt that any API accepts. Selective Context removes phrases and sentences with low self-information, estimated by a small causal language model. It halves context length with only a 0.023 drop in BERTScore on English tasks (Li et al., 2023). LLMLingua (Jiang et al., 2023) refines the same idea with a coarse-to-fine procedure: a budget controller allocates compression across prompt sections, then tokens are removed iteratively according to a small model's perplexity. It reaches up to 20-fold compression on English reasoning and conversation benchmarks. LongLLMLingua adds question-aware scoring and document reordering for long retrieval contexts (Jiang et al., 2024). RECOMP compresses retrieved documents into extractive or abstractive summaries before they are added to the prompt (Xu, Shi and Choi, 2024). Nano-Capsulator rewrites prompts into shorter natural-language capsules with a fine-tuned LLM (Chuang et al., 2024). Other recent work learns which tokens to delete with reinforcement learning (Jung and Kim, 2024), prunes in-context examples for reasoning tasks (Huang et al., 2024b), or scores whole sentences with a context-aware encoder (Liskavets et al., 2025).

The two LLMLingua generations differ architecturally. LLMLingua scores each token with a small causal language model, which reads left to right and estimates how predictable the token is from the preceding context only; low-surprise tokens are deleted. LLMLingua-2 (Pan et al., 2024) reformulates compression as token classification: keep-or-drop labels are distilled from GPT-4, and a bidirectional encoder (XLM-RoBERTa-large) learns to predict them from the context on both sides of each token. Its authors argue that unidirectional entropy is poorly aligned with the compression objective. For Arabic there is a further

reason to expect the bidirectional design to help: verb–subject–object order is common in MSA alongside subject–verb–object (Habash, 2010), so a token's syntactic role often depends on words that follow it, which a left-to-right scorer cannot see when deciding whether to delete it. LLMLingua-2 is also three to six times faster, multilingual and precise in its rate control.

Evaluations of these methods are becoming more critical. Jha et al. (2024) find that simple extractive compression often beats token pruning, despite claims to the contrary. Łajewska et al. (2025) show that several state-of-the-art methods fail to preserve key details such as entities, which hurts complex tasks. Nagle et al. (2024) derive the distortion–rate function for black-box prompt compression and show a large gap between current methods and the optimum. The gap is narrowed by compression that is *query-aware* and *variable-rate*, meaning it knows the downstream task and adapts the rate to each input. This is the theoretical case for choosing the compression rate per prompt rather than fixing it globally. Table 2 summarises the alternatives against the requirements of this project.

Method family	**Representative work**	**Black-box API compatible**	**Multilingual scorer available**	**Rate control**	**Suitability here**
Soft prompts / memory slots	Mu, Li and Goodman (2023); Chevalier et al. (2023); Ge et al. (2024)	No	Model-dependent	Fixed by training	Excluded: needs model internals
Perplexity-based token deletion	Li et al. (2023); Jiang et al. (2023)	Yes	Yes, if the scorer is swapped	Approximate	Included (LLMLingua)
Question-aware long-context	Jiang et al. (2024); Xu, Shi and Choi (2024)	Yes	Partly	Approximate	Excluded: targets long retrieval contexts
Classifier-based token deletion	Pan et al. (2024)	Yes	Yes (XLM-RoBERTa)	Precise	Included (LLMLingua-2)
Generative rewriting	Chuang et al. (2024)	Yes	Needs a fine-tuned LLM	Weak	Excluded: needs training, adds an LLM call
Learned or sentence-level selection	Jung and Kim (2024); Liskavets et al. (2025)	Yes	Needs retraining for Arabic	Coarse (sentence) or learned	Excluded: no Arabic models; coarse units suit long contexts
Random token deletion	— (baseline)	Yes	Not applicable	Precise	Included as control

Table 2. Prompt compression alternatives evaluated against the project's requirements

Every one of these methods was developed and evaluated on English. LLMLingua's reference scorers are GPT-2 and LLaMA-7B. LLMLingua-2's distillation data (MeetingBank) is

English. None of the papers reports results on Arabic or any other morphologically rich language. Whether English-learned notions of token importance transfer to Arabic is therefore untested. RQ1 addresses this.

2.3.2 Adaptive and cost-aware LLM use

A second body of work shows that choosing *per query* beats committing to one global strategy. FrugalGPT combines prompt adaptation, model approximation and an LLM cascade. It matches GPT-4's accuracy at up to 98% lower cost by sending each query only as far up the cascade as needed (Chen, Zaharia and Zou, 2024). Hybrid LLM trains a router that predicts query difficulty and sends easy queries to a small model, cutting large-model calls by up to 40% without loss of quality (Ding et al., 2024). RouteLLM learns routers from human preference data and more than halves cost (Ong et al., 2025). FORC uses a meta-model to predict each candidate model's performance on an input and chooses the cheapest adequate one, reducing cost by 63% (Šakota, Peyrard and West, 2024).

These systems share the APCS's structure: inexpensive input features drive a choice among options with different cost–quality trade-offs. They differ in what they choose. Routers select which model answers an unchanged prompt; the APCS keeps the model fixed and selects how the prompt is transformed. Routing changes the price per token, whereas compression changes the number of tokens, including, as Chapter 5 shows, the number of output tokens. Both are instances of the **algorithm selection problem** (Rice, 1976), whose four components map directly onto this project: a problem space (Arabic prompts), an algorithm space (no compression and each compressor–rate pair), a feature space (length, task type, fragmentation and the other APCS features) and a performance measure (the best-balance or cheapest-faithful criterion). A selection mapping learned from features is judged against the *single best solver*, the one fixed strategy that performs best on average, which is the baseline used throughout Chapter 5.

2.3.3 Multilingual tokenisation and morphology

Why might Arabic behave differently under compression? The tokenisation literature offers mechanisms. Limisiewicz, Balhar and Mareček (2023) show that multilingual tokenisers allocate vocabulary unevenly across languages, and that the allocation predicts downstream performance gaps. Ali et al. (2024) find that tokeniser choice measurably changes both downstream accuracy and training cost. English-centric tokenisers used for multilingual models cause severe downstream degradation and up to 68% additional training cost. On morphology, Park et al. (2021) find that languages with richer inflectional morphology are harder to model, though the effect weakens when segmentation respects morpheme boundaries. Arnett and Bergen (2025) test three explanations for the gap. They find no evidence that tokenisers' alignment with morpheme boundaries explains it, some evidence for tokenisation quality, and the clearest effect from training-data size. The gap largely disappears when datasets are equated in size after adjusting for each script's encoding efficiency (its "byte premium"), so morphological type itself confers no intrinsic disadvantage. MorphBPE builds morpheme boundaries into byte-pair encoding and improves training efficiency for morphologically rich languages, including Arabic (Asgari et al., 2026).

The tax can be stated formally. For a text with W orthographic words and T sub-word tokens, tokeniser fertility and the tax relative to English are

$$\text{fertility} = \frac{T_{\text{subword}}}{W_{\text{orthographic}}}, \text{tax} = \frac{\text{fertility}_{\text{Arabic}}}{\text{fertility}_{\text{English}}}$$

Fertility is the fragmentation ratio used as an APCS feature (4.1 for cl100k_base on AraPromptBench); the tax compares equivalent content across languages, the quantity Ahia et al. (2023) and Petrov et al. (2023) measure with parallel text.

Two hypotheses follow for compression. First, if a prompt's tokens are mostly fragments of words, deleting tokens may break words. Deleting one byte-level fragment of a multi-byte Arabic character can even corrupt the text. Second, prompts with dense morphology, where many function words are fused into each orthographic word, may carry more information per word and so tolerate deletion less. The SDR turned the second hypothesis into a morphology-based selection rule. RQ2 tests both. The evidence above suggests morphology

itself may matter less than tokenisation and the amount of text, and that is the outcome Chapter 5 reports.

2.3.4 Arabic NLP and LLM evaluation

Habash (2010) describes the properties of Arabic that complicate processing: root-and-pattern morphology, clitic attachment, optional diacritics, and the coexistence of Modern Standard Arabic with regional dialects. Language-specific processing pays off. The Farasa segmenter splits words into clitics and stems quickly and accurately (Abdelali et al., 2016). AraBERT, pre-trained on Arabic with segmentation-aware preprocessing, outperforms multilingual BERT on Arabic understanding tasks (Antoun, Baly and Hajj, 2020). Both are used in this project: Farasa to measure morphological density, and AraBERT as the fidelity scorer.

Two segmenters were candidates for measuring morphology. MADAMIRA performs full morphological analysis and disambiguation, assigning lemmas, part of speech and clitic segmentation in context (Pasha et al., 2014), but it is a heavier system whose richer output is not needed for counting segments. Farasa performs segmentation only and reports accuracy comparable to MADAMIRA at a fraction of the processing time (Abdelali et al., 2016). For a selector feature computed at inference time, speed and simplicity outweigh full analysis, so Farasa was used.

Arabic LLM evaluation has matured rapidly. GPTAraEval evaluated ChatGPT on 44 Arabic tasks and found it clearly behind smaller fine-tuned models on many of them (Khondaker et al., 2023). ArabicMMLU measures knowledge with school-exam questions from Arabic-speaking countries (Koto et al., 2024). ORCA and Dolphin provide understanding and generation benchmarks respectively (Elmadany, Nagoudi and Abdul-Mageed, 2023; Nagoudi et al., 2023). Public instruction corpora now exist: CIDAR provides 10,000 culturally reviewed instruction–response pairs (Alyafeai et al., 2024), and the Aya dataset provides human-written instructions in many languages and dialects (Singh et al., 2024). Together with older task corpora (XL-Sum for summarisation (Hasan et al., 2021), the Essex Arabic Summaries Corpus (El-Haj, Kruschwitz and Fox, 2010), TyDi QA (Clark et al., 2020) and

ARCD (Mozannar et al., 2019) for question answering), these made it possible to build AraPromptBench from credible, licensed sources rather than from invented prompts.

None of these benchmarks studies efficiency. They measure whether a model *can* do a task in Arabic, not what it costs or how far a prompt can be shortened. To the author's knowledge, no Arabic prompt compression benchmark existed when this project began.

2.3.5 Evaluating compressed prompts

Three methodological issues from the literature shape how compression should be evaluated.

Fidelity measurement. BERTScore compares texts by matching contextual token embeddings, which captures paraphrase better than n-gram overlap (Zhang et al., 2020). Its behaviour depends on the encoder, so an Arabic-specific encoder may separate good from bad Arabic outputs better than multilingual BERT (Devlin et al., 2019). Experiment 01 tests this (Chapter 4). An alternative is to ask a strong LLM to judge answer quality. Such judges agree well with human preferences (Zheng et al., 2023; Chiang and Lee, 2023), but they add an extra paid call per comparison and bring biases of their own, including a preference for longer answers. For a benchmark of tens of thousands of comparisons, a deterministic embedding-based metric is more reproducible and affordable. It also matters *which* texts are compared. Comparing the compressed prompt with the original rewards keeping words, not keeping meaning. Comparing the LLM's answers to the two prompts measures what users actually experience. Selective Context and LLMLingua both report task performance on outputs for this reason (Li et al., 2023; Jiang et al., 2023).

Non-determinism. Supposedly deterministic LLM settings are not deterministic. Ouyang et al. (2025) found that ChatGPT gives different code for the same prompt even at temperature zero. Song et al. (2025) show that evaluations based on a single sample per prompt hide considerable variability. A fidelity threshold must therefore be judged against the model's own noise: if two calls with the *identical* prompt often score below a threshold, that

threshold cannot separate compression damage from chance. The SDR's fixed threshold of 0.85 was not tested this way. This project tests it (Chapter 5).

Output length. LLMs trained with reinforcement learning from human feedback tend to produce longer answers, because preference models reward length (Singhal et al., 2024). Automatic evaluators share the bias and must be corrected for length (Dubois et al., 2024). Explicit length instructions are one of the main ways to control answer length (Jie et al., 2024), and adding a token budget to the prompt cuts reasoning tokens substantially (Han et al., 2025). The implication for compression has not been drawn in the compression literature: when output costs more than input, the words that control *how much* the model writes may be the most valuable words in the prompt.

Length bias also affects the fidelity measure used here. BERTScore F1 is the harmonic mean of precision and recall over matched token embeddings, so an answer that grows after compression adds unmatched tokens, lowers precision and lowers F1 even if its core content is correct. Compressed inputs that produce expanded outputs are therefore penalised twice, once for lost content and once for extra length, whereas an LLM judge would tend to reward the longer answer (Dubois et al., 2024). Section 5.2.10 measures the size of this effect.

Two further concerns apply to evaluation data. **Contamination**: benchmark items seen during development, or during model training, inflate measured performance (Sainz et al., 2023; Deng et al., 2024). Public test sets can be traced in a model's outputs (Golchin and Surdeanu, 2024). Selectors tuned after looking at test results can be contaminated in the same way, so a design changed in response to test results must be re-evaluated on unseen data. **Synthetic data**: LLM-generated examples can fill gaps cheaply but may differ systematically from real data (Long et al., 2024; Liu et al., 2024), so their effect must be measured rather than assumed.

2.3.6 The research gap

The literature establishes that prompt compression can reduce token use substantially (Jiang et al., 2023; Pan et al., 2024); that adaptive per-input selection beats fixed strategies for cost-efficient LLM use (Chen, Zaharia and Zou, 2024; Šakota, Peyrard and West, 2024); and that tokenisation imposes a measurable cost penalty on Arabic (Ahia et al., 2023; Petrov et al., 2023). These findings have not been connected. Specifically, to the best of the author's knowledge, based on searches of the ACL Anthology and Google Scholar (October 2026):

1. no published study evaluates prompt compression methods on Arabic, so it is unknown whether English-trained compressors transfer (RQ1);
2. no study relates prompt features such as length, task type, tokeniser fragmentation or morphological density to compression outcomes in Arabic (RQ2);
3. no feature-based recommender exists for selecting a compression strategy per prompt, in Arabic or any other language (RQ3);
4. compression is evaluated by token reduction or input cost. Its effect on *output* length, and so on the total bill under output-weighted pricing, has not been measured.

The fourth gap emerged during the project rather than from the initial review. It is included because the literature on length bias (Singhal et al., 2024) and length control (Jie et al., 2024) predicts it, and because the results confirm it.

2.4 Theory

Compression as constrained optimisation. Let p be a prompt, M the LLM under test and s = (method, target rate) a compression strategy that maps p to a shorter prompt $s(p)$. Token compression ratio and output-level fidelity are

$$\mathrm{TCR}(s,p) = 1 - \frac{|s(p)|}{|p|}, F(s,p) = \mathrm{BERTScore}_{F1}(M(s(p)), M(p))$$

where |·| counts cl100k_base tokens and F uses AraBERT. For a candidate set S that always contains no compression (TCR = 0, F = 1), the best-balance strategy, the proposal's "maximum token reduction while maintaining semantic fidelity above a threshold", is

$$s^*(p) = \arg\max_{s \in S} \mathrm{TCR}(s,p) \text{ subject to } F(s,p) \geq \tau$$

Selection as algorithm selection. $s*(p)$ exists for every prompt and turns evaluation into classification. Following Rice (1976), the APCS is a mapping g from a feature vector $f(p)$ to S, judged by how often $g(f(p)) = s*(p)$ on unseen prompts. Interpretable mappings, ordered threshold rules and shallow decision trees, are preferred because they can be inspected and audited and suit small calibration sets, as the SDR required.

The cost model. A call with n_{in} prompt tokens and n_{out} completion tokens costs

$$C = \pi_{\mathrm{in}} n_{\mathrm{in}} + \pi_{\mathrm{out}} n_{\mathrm{out}}$$

If compression removes a share r of the input tokens and the answer grows by a share g, the bill falls only if $\pi_{in} \cdot r \cdot n_{in} > \pi_{out} \cdot g \cdot n_{out}$, so the break-even answer growth is

$$g^* = \frac{\pi_{\mathrm{in}}}{\pi_{\mathrm{out}}} \cdot r \cdot \frac{n_{\mathrm{in}}}{n_{\mathrm{out}}}$$

With $\pi_{out} = 6\pi_{in}$ and prompts and answers of similar length, a 10% longer answer cancels a 60% shorter prompt. Token reduction is therefore a proxy for cost only if answer length is unchanged, which is why cost is always measured from billed tokens.

Measurement noise. Because M is not deterministic, F has a noise floor, its distribution between two answers to the identical prompt, and τ is set so that pure noise rarely fails it (Section 3.4.4). In the experiments, π_{in} and π_{out} are the LLM's list prices (US$0.20 and US$1.20 per million tokens); n_{in} and n_{out} are the prompt and completion tokens the API billed for each call; $|p|$ and $|s(p)|$ are cl100k_base counts; F is AraBERT layer-9 BERTScore; and $\tau = 0.65$.

2.5 Terminology

- **Prompt compression:** reducing a prompt's token count before sending it to an LLM while trying to preserve the information it needs.

- **Target rate (keep rate):** the fraction of tokens a compressor is asked to keep (0.7, 0.5 or 0.3 in this project). *Achieved keep* is the fraction actually kept.
- **Token Compression Ratio (TCR):** 1 − compressed tokens / original tokens, counted with the cl100k_base tokeniser.
- **Fragmentation ratio:** tokens per orthographic word; the size of the tokeniser tax for a given prompt.
- **Morphological density:** mean number of Farasa segments (clitics and stems) per Arabic word.
- **Prompt-level fidelity:** BERTScore F1 between the compressed and original prompts. Diagnostic only.
- **Output-level fidelity:** BERTScore F1 between the LLM's answers to the compressed and original prompts. The primary fidelity measure.
- **Noise ceiling:** the distribution of output-level F1 between two answers to the identical prompt.
- **τ (tau):** the fidelity threshold derived from the noise ceiling (0.65 for AraBERT).
- **Best-balance label:** for a prompt, the strategy with the largest TCR whose output F1 ≥ τ, or no compression if none qualifies.
- **Cheapest-faithful label:** for a prompt, the strategy with the lowest measured API cost whose output F1 ≥ τ.
- **Length instruction:** an explicit constraint on answer length, such as "briefly", "in three sentences" or "no more than fifty words".
- **LLMLingua-2-protected:** the variant developed in this project that keeps sentences containing length instructions verbatim and compresses the rest.
- **APCS 1.0.0, APCS-v2, APCS-cost:** the calibrated rule-based selector, the redesigned decision-tree selector, and the cost-aware selector (Chapter 4).
- **Dev, test and exam sets:** the 800-prompt development split, the 200-prompt held-out split and the 400-prompt fresh exam set.

2.6 Chapter Summary

Hard-prompt compression is the only family usable with a closed API, and LLMLingua and LLMLingua-2 represent its two scoring philosophies. Adaptive selection outperforms fixed strategies elsewhere in cost-efficient LLM use, and tokenisation makes Arabic expensive, but compression has not been evaluated on Arabic, related to prompt features, wrapped in a selector, or examined for its effect on output length and total cost. Chapter 3 turns this into a research design.

Chapter 3. ANALYSIS AND DESIGN

3.1 Chapter Introduction

This chapter turns the literature and theory into a specification (Section 3.2), a research method (3.3) and design decisions for the benchmark, compressors, LLM protocol, fidelity measure, labels, APCS architecture and evaluation plan (3.4). Deviations from the SDR are justified where they arise and consolidated in Section 4.2.7.

3.2 Project Specification

The project must deliver (i) a benchmark that can measure compression on Arabic, (ii) an analysis pipeline that relates prompt features to compression outcomes, and (iii) an artefact that recommends a compression strategy for any Arabic prompt. Table 3 links each component to the research question it serves, as the SDR required.

Component	Serves	Specification
AraPromptBench v2 (1,000 prompts) and exam set (400)	RQ1, RQ3	MSA prompts from credible public corpora; four task categories; documented licences; deconfounded length bands; dev / test / exam separation with zero overlap
Compression layer	RQ1	LLMLingua, LLMLingua-2 and random deletion at target keep rates 0.7 / 0.5 / 0.3, plus no compression; deterministic and seeded
LLM and evaluation module	RQ1–RQ3	One fixed LLM with deterministic settings; output-level BERTScore with a noise-derived threshold; measured token usage and cost per call
Feature extractor	RQ2, RQ3	Character length, token count, fragmentation ratio, structural complexity, morphological density, task category (plus a length-instruction flag added in Experiment 11)
APCS decision engine	RQ3	Interpretable mapping from features to a strategy; calibrated on dev only; frozen before held-out evaluation
Interfaces	Artefact use	Python library API and command-line interface; JSON output; versioned rule files

Table 3. Project components and the research questions they serve

The non-functional requirements carried over from the SDR are reproducibility (seeds, configuration and model snapshot recorded with every result), explainability (the selector must

state *why* it chose a strategy), low inference overhead (feature extraction without a GPU or an LLM call), and evaluation integrity (held-out data used once, after the design is frozen).

Input handling is fail-safe. A non-string or empty prompt is rejected with an error, since there is nothing to compress. Text with encoding damage (undecodable bytes become Unicode replacement characters) and text in which fewer than half of the letters are Arabic fall outside the calibration domain, so the APCS returns no compression and states the reason rather than guessing. Every AraPromptBench prompt is at least 83% Arabic, so the fallback never alters an evaluated result.

3.3 Research Methods

Methodology. The project follows design science research (Hevner et al., 2004). This methodology suits a project that must both build an artefact and generate knowledge about the problem it addresses. Hevner et al.'s guidelines map directly onto the work. *Design as an artefact*: the APCS and AraPromptBench. *Problem relevance*: the Arabic tokeniser tax (Section 2.2). *Design evaluation*: held-out and fresh-exam experiments against baselines. *Research rigour*: paired tests, pre-registration and an independent audit. *Design as a search process*: the iteration from the pilot rule through APCS 1.0.0 to APCS-v2. *Research contributions*: both the artefact and the empirical findings on Arabic compression. *Communication*: this dissertation, the experiment log and a public-ready repository.

Research approach. The empirical work is quantitative and experimental. The unit of analysis is a prompt. Every prompt is processed by every strategy, which gives a fully *paired* (within-prompt) design: differences between strategies are not confounded by differences between prompts, and paired tests can be used. Sampling into the benchmark is stratified by source, category and length band with a fixed seed (42), so the dataset is reproducible and its composition controlled.

Statistical methods. The choice of tests follows the SDR and standard practice for paired NLP comparisons:

- **Method comparisons (RQ1):** Wilcoxon signed-rank test on per-prompt output F1 differences (Wilcoxon, 1945). This test is non-parametric because F1 differences are bounded and skewed. Paired bootstrap 95% confidence intervals (10,000 resamples) are given for mean differences.
- **Feature effects (RQ2):** Spearman correlations within strata; Kruskal–Wallis tests for category effects within length bands; ordinary least squares regression with standardised features and category indicators. The *unique* variance of each predictor is the drop in R^2 when it is removed from the full model.
- **Selector accuracy (RQ3):** McNemar's test on paired correct/incorrect outcomes (McNemar, 1947). Only discordant prompts, where one selector is right and the other wrong, carry information; with counts b and c, the exact test compares b with a Binomial($b + c$, 0.5) distribution, two-sided. Being exact, it uses no chi-square approximation, so no degrees of freedom or continuity correction apply. Bootstrap 95% confidence intervals are given for accuracies and differences.
- **Multiplicity:** Holm's sequentially rejective procedure across the pre-registered hypotheses of Experiment 11 (Holm, 1979).
- **Cost:** one-sided Wilcoxon test on per-prompt cost differences, plus the change in total measured cost.

Safeguards against contamination and over-fitting. Evaluation data seen during design inflates measured performance (Sainz et al., 2023; Deng et al., 2024). Four safeguards were applied. (1) The 500-prompt pilot was designated dev-only before any analysis, and AraPromptBench v2 was built with zero overlap with it, so the v2 test split is untouched by pilot-era decisions. (2) The v2 test split was read by a single script, run once, after APCS 1.0.0 was frozen. (3) When Experiment 10's results motivated a redesign, the redesigned selectors were not evaluated on the already-used test split. A new 400-prompt exam set was built instead, with zero exact or 12-word passage overlap with every earlier set. (4) Selectors, hypotheses, comparators and the analysis script for the exam were frozen and committed in a pre-registration document before any exam answer was collected (Appendix D).

Ethics. The project involves no human participants, personal data or user prompts. All corpora are public and used under their licences (Table 4), and the creative, probe and pilot prompts are synthetic. The proposal records that ethics approval was not required, and nothing in the final design changed that (Appendix C). Two indirect issues are handled explicitly: licences are tracked per prompt, and findings are reported as applying to Modern Standard Arabic and not generalised to all Arabic varieties.

3.4 Design Considerations

3.4.1 Benchmark design

AraPromptBench v2 follows the composition planned in the SDR (Table 4). Prompts are built from task corpora using fixed templates: five summarisation phrasings and three QA phrasings, rotated to avoid template artefacts. Each prompt is screened by an Arabic-character gate (at least 80% Arabic letters) and deduplicated after Unicode and diacritic normalisation, both within v2 and against the pilot. Length bands are defined in cl100k tokens: short 30–90, medium 91–250, long 251–650.

Source	Category	Licence	n (v2)	n (exam)
CIDAR (Alyafeai et al., 2024)	Instruction	CC BY-NC 4.0	250	84
Aya, Arabic subset incl. dialectal probe (Singh et al., 2024)	Instruction	Apache 2.0	100	56
XL-Sum Arabic (Hasan et al., 2021)	Summarisation	CC BY-NC-SA 4.0	150	60
EASC (El-Haj, Kruschwitz and Fox, 2010)	Summarisation	Research use	100	40
TyDi QA GoldP Arabic (Clark et al., 2020)	QA	Apache 2.0	159	64
ARCD (Mozannar et al., 2019)	QA	CC BY-SA 4.0	91	36
Synthetic creative briefs (Claude Fable 5; exam: Claude Opus 5.5)	Creative	CC BY 4.0	150	60
Total			**1,000**	**400**

Table 4. AraPromptBench v2 and exam set composition

A design weakness found in the pilot drove one deliberate change. In the pilot, almost all short prompts were QA or creative and almost all long ones were summarisation, so length and task type were confounded. The v2 design therefore populates every category × length

cell (Table 5), deliberately including 57 short summarisation prompts and 99 long QA prompts, so that RQ2 can separate the two effects statistically.

Category	Short (30–90)	Medium (91–250)	Long (251–650)	Total
Instruction	173	124	53	350
Summarisation	57	102	91	250
QA	56	95	99	250
Creative	42	73	35	150
Total	328	394	278	1,000

Table 5. AraPromptBench v2 prompts by task category and length band (cl100k tokens)

The 150 creative prompts are synthetic because public Arabic creative-writing prompts with suitable licences were not available at the required scale. Every brief caps the requested answer at 120 words or fewer, which fixed the pilot's frequent truncation of creative answers, and the effect of synthetic prompts is measured rather than assumed (Experiment 09; Section 5.2.9). The v2 prompts average 220 cl100k tokens (median 185, range 30–664).

The **exam set** (400 prompts: 140 instruction, 100 summarisation, 100 QA, 60 creative) was built with the same sources, templates and band mix, but a different seed (4242). Its 60 creative briefs follow the same format and 120-word cap. Every v2 record, every v2 QA passage and any text sharing a 12-word span with a pilot, v2 or probe prompt was excluded. Shared instruction templates were exempted because they are identical by design. An independent audit (Section 5.2.10) confirmed zero overlap.

Synthetic data provenance. Synthetic prompts were generated with the most capable model available to the author at the time of each generation step: the 500-prompt pilot set with gpt-5.4-mini (OpenAI); the 150 v2 creative prompts and the 456-prompt Experiment 09 probe pool with Claude Fable 5 (Anthropic); and the 60 exam creative briefs with Claude Opus 5.5 (Anthropic). All synthetic prompts that contribute to held-out results were therefore produced by a different provider from the LLM under test, which avoids generator–evaluator circularity. The pilot set, whose generator came from the same provider as the pilot's LLM, was used only for development and informs no held-out result.

3.4.2 Compression candidates

Three compressors and a no-compression option were selected from the alternatives in Table 2:

- **LLMLingua-2** (Pan et al., 2024) with its released XLM-RoBERTa-large classifier, representing classifier-based scoring;
- **LLMLingua** (Jiang et al., 2023), representing perplexity-based scoring. Its default GPT-2 scorer was replaced with Qwen2.5-0.5B, because the vertical slice showed that GPT-2 corrupts Arabic (Section 4.2.2);
- **random deletion** at the same keep rate, seeded per prompt: the control that any intelligent method must beat;
- **no compression**, which is always available and always passes the fidelity constraint.

Each compressor runs at target keep rates of 0.7, 0.5 and 0.3, the rates fixed in the SDR. Experiment 11 adds **LLMLingua-2-protected** at the same rates (Section 4.2.6). LongLLMLingua, named in the proposal, was replaced by LLMLingua-2 at the SDR stage. LongLLMLingua's question-aware machinery targets long multi-document retrieval contexts that AraPromptBench does not contain, while LLMLingua-2 gives the scientifically more useful contrast of classifier against perplexity scoring.

Qwen2.5-0.5B was chosen as LLMLingua's scorer on four criteria: Arabic among its training languages, a standard causal architecture that LLMLingua loads without modification, a size small enough to score every token quickly on a 12 GB GPU, and acceptable Arabic tokenisation. A later comparison on 200 dev prompts (Experiment 13) confirms the choice was not costly: Qwen2.5 splits Arabic into 2.27 tokens per word, against 2.28 for the Llama 3 family (including Llama-3.2-1B, twice the size), 5.32 for LLMLingua's default Llama-2-7B and 5.86 for GPT-2, while Cohere's Aya models (8 billion parameters for Aya 23 and Aya Expanse) are too large for per-token scoring on the project hardware. Arabic-centric models fragment Arabic less and are left for future work (Section 6.6).

3.4.3 LLM under test and calling protocol

One commercial LLM is used throughout the final benchmark, with the most deterministic settings it accepts: temperature 0 and reasoning effort "none", so no hidden reasoning tokens are billed. The completion limit is 1,024 tokens; the SDR's 512 truncated 9.6% of pilot answers (401 of 4,173). The SDR specified gpt-4o-mini, which the pilot used. The final benchmark moved to gpt-5.6-luna, a newer small model from the same provider with separate input and output prices, so that the noise ceiling, answer-length behaviour, costs and calibration describe the generation of low-cost models deployed now. All reported results come from one model snapshot in one run, with every call's billed token counts recorded and the call order shuffled with a fixed seed so that drift cannot align with a method or category. Temperature 0 still does not make a hosted model deterministic. Greedy decoding picks the most probable token, but the probabilities vary slightly between runs: server load changes how many requests are batched together, and GPU kernels that are not batch-invariant return numerically different results for different batch sizes, which can flip a near-tie between tokens and change the rest of the answer (He, 2025). Section 3.4.4 measures this variation and builds the threshold on it.

3.4.4 Fidelity measurement

Three design decisions govern fidelity.

Output level, not prompt level. Fidelity is measured between the LLM's answers to the compressed and original prompts, not between the prompts. The pilot showed why: at prompt level, random deletion scores *higher* than LLMLingua-2, because BERTScore rewards keeping words verbatim. At output level the ranking reverses (Section 5.2.2). Prompt-level F1 is retained only as a diagnostic.

Scorer. The SDR named multilingual BERT as the primary scorer and AraBERT-v02 as the check. Experiment 01 compared them on 4,000 controlled perturbation pairs. Both rank pairs almost identically (Spearman 0.98), but AraBERT has 50% more dynamic range (0.51 vs 0.34 between identical and unrelated texts). Under multilingual BERT, randomly deleting

30% of a prompt still passes the SDR's 0.85 threshold 86% of the time; under AraBERT, 9.8% of the time. AraBERT therefore became primary (layer 9, no rescaling), consistent with Antoun, Baly and Hajj (2020), and multilingual BERT the sensitivity check.

Threshold from the noise ceiling. τ is derived in three steps. (1) Each of the 800 dev prompts was sent twice, uncompressed, giving answers $a_1(p)$ and $a_2(p)$. (2) Their agreement $F_{\text{rep}}(p)$ = BERTScore F1($a_1(p)$, $a_2(p)$) forms the noise distribution. (3) τ is its 0.4th percentile, so that 99.6% of pure-noise pairs pass, rounded to the nearest 0.05:

$$\tau = \text{round}_{0.05}\left(Q_{0.004}\{F_{\text{rep}}(p)\}\right) = \text{round}_{0.05}(0.640) = 0.65$$

The same procedure gives 0.75 for multilingual BERT (quantile 0.731). Section 5.2.1 shows why the SDR's fixed 0.85 could not be kept.

3.4.5 Labels and metrics

The primary label is **best balance** (Section 2.4): the candidate with the largest TCR whose output F1 ≥ τ, chosen from no compression and LLMLingua-2 at 0.7, 0.5 and 0.3. The SDR's original metric, the share of prompts on which the selector picks *any* Pareto-optimal candidate, was implemented and found to be degenerate. Because no compression always has the highest fidelity, it is always Pareto-optimal, so the trivial policy "never compress" scored 88.5% on the test split, higher than any real selector (Section 5.2.6). Best-balance accuracy keeps the proposal's own definition of "best balance" and removes this degeneracy, so it was adopted as the primary RQ3 metric. Pareto-hit rate is reported as a footnote.

A second label, **cheapest faithful**, is the candidate with the lowest *measured* call cost whose F1 ≥ τ, chosen from ten candidates. It was introduced in Experiment 11 to evaluate the cost-aware selector. Secondary metrics are mean TCR, mean output F1, the rate of answers below τ, the change in total measured cost relative to never compressing, and, for QA prompts, **QA correctness**: whether the answer contains the gold answer string.

3.4.6 APCS architecture

Figure 1 shows the architecture as built. The APCS is a thin, dependency-light layer: a feature extractor that needs only a tokeniser (plus an optional Java segmenter for morphology), a decision engine that reads its rules from a versioned JSON file, and an optional compression step that applies the recommendation through LLMLingua. The evaluation pipeline is outside the artefact. It produces the evidence from which the rules are calibrated.

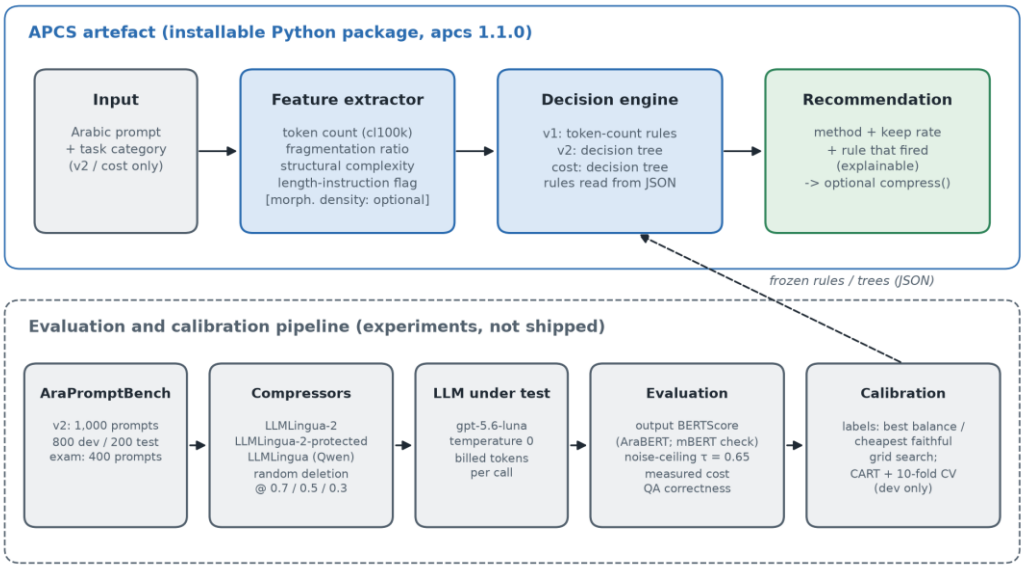


Figure 1. Architecture of the APCS artefact and the evaluation pipeline that calibrates it

The decision engine was designed in two generations. **APCS 1.0.0** keeps the SDR's form, an ordered list of guarded threshold rules: calibration searches a grid of token-count thresholds $T1 < T2$ and rates for the medium and long branches and keeps the rule with the highest best-balance accuracy on dev. The SDR's draft rules also used structural complexity, morphological density and a creative-category clause, to be retained only if the dev evidence supported them; it did not (Section 5.2.4). **APCS-v2** replaces hand-written guards with a CART decision tree learned with scikit-learn (Pedregosa et al., 2011) over all features plus task category. Splits minimise Gini impurity, the library default; the tree is limited in advance by maximum depth (1–5) and minimum leaf size (10, 20 or 40), chosen by 10-fold stratified cross-validation on dev, with no post-hoc cost-complexity pruning and a fixed random seed. A depth-four tree is still a short list of readable rules (Appendix D), meeting the SDR's explainability requirement. **APCS-cost** uses the same learner trained on cheapest-faithful labels, with the protected compressor and LLMLingua among its candidates.

3.4.7 Evaluation plan and hypotheses

Evaluation proceeds in three stages, each on data the previous stage never saw.

1. **Development (800 prompts):** RQ1 and RQ2 analyses, τ derivation and selector calibration.
2. **Held-out test (200 prompts):** a single evaluation of APCS 1.0.0 against the SDR's baselines: no compression, always LLMLingua-2 (each rate), always LLMLingua (each rate, on labels that admit LLMLingua) and random selection.
3. **Fresh exam (400 prompts):** a pre-registered evaluation of three hypotheses, Holm-corrected:

- **H1.** APCS-v2 achieves higher best-balance accuracy than always using LLMLingua-2 @ 0.5, the best fixed strategy on dev (exact McNemar).
- **H2.** APCS-v2 achieves higher accuracy than APCS 1.0.0 (exact McNemar).
- **H3.** APCS-cost lowers the measured bill relative to never compressing (one-sided Wilcoxon on per-prompt cost differences, and a negative total cost change).

The replication accuracy of APCS 1.0.0 on the exam, the protected compressor's effect and QA correctness were declared as secondary analyses.

3.5 Chapter Summary

The design specifies a deconfounded benchmark, three compressors, a deterministic LLM protocol, output-level fidelity with an Arabic scorer and a noise-derived threshold, two labels, a two-generation APCS and a three-stage evaluation ending in pre-registered hypotheses. Chapter 4 describes its implementation.

Chapter 4. IMPLEMENTATION (REALISATION)

4.1 Chapter Introduction

This chapter describes the implementation (Section 4.2), including where evidence changed the design, consolidates every change from the SDR (4.2.7) and presents the IT artefact (4.3).

4.2 Details of the Implementation

4.2.1 Environment and tooling

All work was done in Python 3.11.9 on a Windows 11 workstation with an NVIDIA RTX 5070 GPU (12 GB), which ran the compressors and BERTScore locally. Only LLM calls went to the API. Table 6 lists the main libraries. Every experiment lives in its own folder containing the scripts, a README stating the question and method, a `results/` folder with raw outputs and configuration, and a FINDINGS file with results, decision and caveats. Git and a GitHub repository recorded each milestone, with one commit per milestone. A chronological experiment log indexes all eleven experiments and one supplementary measurement (Appendix E).

Purpose	Library (version)
Compression	llmlingua 0.2.2 (LLMLingua, LLMLingua-2)
Models and scoring	torch 2.11 (CUDA 12.8), transformers 5.13, bert-score 0.3.13
Tokenisation	tiktoken 0.13 (cl100k_base)
Morphology	farasapy 0.1.1 (Farasa, Java 21)
LLM API	openai 2.44 (asynchronous client)
Analysis	pandas 3.0, scipy 1.17, statsmodels 0.15, scikit-learn 1.9
Testing	pytest

Table 6. Implementation environment

Two platform problems had to be solved. First, the GPU's Blackwell architecture required PyTorch builds for CUDA 12.8. Second, Farasa's interactive mode silently corrupted Arabic text passing through the Windows process pipe. It was run in standalone mode instead, which needs a Java runtime but returns correct segmentations.

GPU memory was managed by staging. The 12 GB card cannot comfortably hold the compressors and both BERTScore encoders at once, so compression (Qwen2.5-0.5B and XLM-RoBERTa-large, loaded together) and scoring run as separate scripts. Scoring loads one encoder at a time and processes answer pairs in batches of 32 to 64. Answers longer than AraBERT's 512 word-piece window are truncated by the scorer, and their share is reported (Section 4.2.4).

4.2.2 Pilot phase and the design changes it triggered

Seven pilot experiments on the 500-prompt pilot set tested each design assumption before money was spent on the final benchmark. The pilot set consists of 500 Modern Standard Arabic prompts, 125 per category, generated with gpt-5.4-mini (OpenAI). Because it was used only for development, its synthetic origin does not affect any held-out result. Table 7 summarises what each found and what it changed.

Exp.	Question	Finding	Design consequence
01	Which BERTScore encoder?	AraBERT and mBERT rank pairs alike (ρ = 0.98), but AraBERT has 50% more dynamic range; under mBERT, random 30% deletion passes 0.85 in 86% of cases	AraBERT primary, mBERT sensitivity check (roles swapped from the SDR)
02	Do the compressors run on Arabic?	LLMLingua with its default GPT-2 scorer crashed on all long prompts and corrupted characters in 38 of 45 outputs; with Qwen2.5-0.5B it ran cleanly but left short prompts uncompressed; LLMLingua-2 hit all target rates	GPT-2 scorer dropped; Qwen2.5-0.5B adopted; LLMLingua's length gate recorded as a property of the method
03	Local benchmark at scale	5,000 compressions, 0 errors; at prompt level random deletion beats LLMLingua-2 ($p < 1e{-}22$)	Prompt-level F1 judged a mechanical artefact; fidelity moved to output level
04	Output-level evaluation	4,173 API calls (US$0.66); ranking flips: LLMLingua-2 beats random deletion at every rate ($p \leq 4e{-}9$); 9.6% of answers truncated at 512 tokens	Two-level design vindicated; completion limit raised to 1,024
05	Feature analysis	Length dominates (ρ ≈ +0.4 with LLMLingua-2 fidelity); morphological density weak (absolute $\rho \leq 0.19$); length and category confounded	Morphology rule not adopted; v2 dataset deconfounded by length band
06	First APCS	Rule "tokens < 120 → none, else LLMLingua-2 @ 0.5": 41.2% best-balance accuracy in-	Best-balance accuracy adopted as the primary metric; package scaffolded

Exp.	Question	Finding	Design consequence
		sample; Pareto-hit metric degenerate	
07	Protocol checks	Only 13% of repeat calls identical; 31% of identical-prompt pairs score below 0.85	SDR threshold retired; noise-ceiling τ adopted

Table 7. Pilot experiments and the design decisions they produced

Experiment 02 is direct evidence for RQ1: LLMLingua's published default configuration does not transfer to Arabic. Each Arabic letter occupies two bytes in UTF-8. GPT-2's byte-level tokeniser, trained mostly on English, has few merges for Arabic byte pairs, so it inflates Arabic two- to three-fold and often splits one letter's two bytes across tokens: وبالكتاب becomes eight GPT-2 tokens, with the letter ك split into the raw bytes 0xD9 and 0x83. Long summarisation prompts therefore exceeded GPT-2's 1,024-token context, and when the compressor deleted one byte of a split letter, the remaining byte was an invalid UTF-8 sequence rendered as □ (زيادة became □يادة) in 38 of 45 outputs. Qwen2.5-0.5B fixed the corruption, but a length gate remains: short prompts come back almost untouched whatever rate is requested, even with the sentence- and context-level filters disabled. BLOOM-560m was rejected as incompatible with the installed packages.

One defect was found later in the pilot code. The morphological-density feature counted Farasa's punctuation and numeral tokens as words. It was corrected so that only tokens containing an Arabic letter are counted, unit-tested, and the Experiment 05 analysis re-run. Rankings were nearly unchanged ($\rho = 0.94$ between old and new values), but the association with fidelity became slightly stronger and statistically significant (absolute $\rho \leq 0.19$), still far weaker than length. An erratum (05e) records the correction and the softened conclusion.

4.2.3 Building AraPromptBench v2

AraPromptBench v2 was built by the dataset stage of the pipeline (Table 8). Implementation forced two compositional deviations from the SDR: the ARCD pool ran out at 91 prompts after deduplication and answer-preserving truncation (QA passages are cut around the answer span), so QA was topped up from TyDi QA; and Aya's "mixed" rows were all instruction-like, making the categories 350/250/250/150 rather than 250 each. XL-Sum, TyDi QA

and ARCD no longer load through the datasets library's script loaders and were read from the Hugging Face Hub's parquet conversions. The author reviewed a 5% stratified sample before any benchmark call.

The exam set was built by `build_exam.py` with the same machinery and a different seed. Its first version contained five prompts that partially overlapped earlier passages. A 12-word shingle filter was added and the set rebuilt; `audit_exam.py` then confirmed zero overlap. A 20-prompt sample was reviewed by the author before the exam run.

4.2.4 The benchmark pipeline

Table 8 summarises the pipeline. Each stage is a separate script that reads the previous stage's output, so any stage can be re-run alone.

Stage	Script(s)	Output	Key details
1. Dataset	build_sourced.py, assemble_v2.py; exam: build_exam.py, audit_exam.py	AraPromptBench v2 and exam set	Seeded sampling, band quotas, deduplication, split, overlap audit
2. Local compression	run_local.py	14,560 compressed variants with TCR and prompt-level F1	LLMLingua (Qwen), LLMLingua-2, random deletion × 3 rates
3. API calls	run_api.py	13,438 answers with billed token counts	Deduplicated by text hash, seeded shuffle, resumable
4. Scoring and analysis	score_and_analyze.py, rq2_final.py, cost_analysis.py	Output F1, τ, RQ1–RQ2 and cost results	Both scorers; dev only
5. Calibration and test	apcs_final.py	APCS 1.0.0 rules; test results	Test set read once
6. Redesign and exam	design_selectors.py, run_exam.py, evaluate_exam.py	Frozen trees; exam results (3,828 calls)	Pre-registered; exam read once

Table 8. Experimental pipeline: stages, scripts and outputs

The final API run made 13,438 calls with no failures, one model snapshot throughout and a measured cost of US$2.49. 79 answers (0.6%) reached the 1,024-token limit, and one answer, to a random-deletion variant, came back empty and is scored 0.

Two implementation details protect validity. AraBERT's input window is 512 word-pieces. 2.6% of dev answer pairs exceed it, so every RQ1 comparison is repeated without them; the differences change by at most 0.002. The cost of each call is computed from the token counts the API returned for that call, priced at US$0.20 and US$1.20 per million input and output tokens: *cost* = prompt tokens × $0.20/10^6$ + completion tokens × $1.20/10^6$. The cost

of a policy is the sum over prompts of the cost of the call for the strategy it chose. Cost is never estimated from TCR.

4.2.5 Calibrating APCS 1.0.0

`apcs_final.py` computes best-balance labels on the 800 dev prompts, searches a grid of 7 × 6 token-count thresholds and 3 × 3 rates, and writes the winning rule to the package. The rule is:

- token count < 80 → no compression;
- 80 ≤ token count < 250 → LLMLingua-2 @ 0.5;
- token count ≥ 250 → LLMLingua-2 @ 0.3.

Its dev accuracy is 38.0%. The script was written to ship a single global rule, as the SDR specified. Per-category rules were computed for reference only: they raised dev accuracy by 5.6 points in-sample, a gain that, with 120–280 dev prompts per category, could easily be over-fitting. The signal they hinted at was followed up properly, on fresh data, in Experiment 11. The script then read the 200 test prompts once, evaluated APCS 1.0.0 and all baselines, and wrote the result. A later bug fix to the LLMLingua comparison in `score_and_analyze.py` re-scored the same stored answers deterministically. The APCS test evaluation was not re-run.

4.2.6 The Experiment 11 redesign

Experiment 10 left two clear leads (Chapter 5): task category carried more signal than length, and LLMLingua-2 made answers longer by deleting length instructions. Experiment 11 acted on both. All design work used dev data only.

Length-protected compression (`protect.py`). One regular expression, fixed in Experiment 10's mechanism test and reused unchanged, identifies length instructions by eight literal markers: بإيجاز ("briefly"), موجز ("concise"), جملتين ("two sentences"), ثلاث جمل ("three sentences"), جملة واحدة ("one sentence"), فقرة واحدة ("one paragraph"), لا يتجاوز ("does not exceed") and كلمة ("word", as in "fifty words"). Markers match as substrings, so attached clitics

(وبإيجاز) still match. They cover the templates' phrasings and matched 625 of the 1,400 v2 and exam prompts; variants outside the list, such as في حدود ("within", 10 prompts), are missed unless كلمة also appears (Experiment 13). The prompt is split into segments at sentence punctuation, colons and line breaks; segments containing a marker are kept verbatim, the rest are compressed with LLMLingua-2 at the target rate, and the pieces are reassembled in order. Prompts without a marker are compressed exactly as plain LLMLingua-2 would compress them. On dev the variant removed most of the cost penalty and raised fidelity at the price of a lower TCR. Protecting question marks in QA prompts was tested and rejected, because dropping them did not reduce QA correctness.

QA correctness (`qa_metrics.py`). For QA prompts, an answer is counted correct if it contains any gold answer as a whole-word sequence, after Arabic normalisation: diacritics, tatweel and punctuation removed, and alef, ya and taa-marbuta variants unified. This gives a task-accuracy check that does not depend on BERTScore.

APCS-v2 and APCS-cost (`design_selectors.py`) were trained on the 800 dev prompts with the settings of Section 3.4.6. APCS-v2 uses best-balance labels and the same four candidates as APCS 1.0.0, so the comparison isolates the selector; its chosen tree (depth 4, minimum leaf 20) reached 43.5% cross-validated accuracy (SD 5.2), against 36.0% for the APCS 1.0.0 rule on the same folds. APCS-cost uses cheapest-faithful labels over ten candidates; its tree (depth 3, minimum leaf 40) cut the cross-validated bill by 1.9%. Both trees were frozen as JSON and pre-registered with `evaluate_exam.py`, which was first run on dev data only as a code check. `run_exam.py` then compressed, answered and scored the exam (3,828 calls, no failures, US$0.61), and `evaluate_exam.py` ran once.

The learned APCS-v2 tree is easy to read. It first separates creative prompts from the rest. Creative prompts are left uncompressed below 86 tokens and otherwise compressed lightly. For other categories it uses token count, with QA treated separately: long QA prompts are compressed *less*, because their answers depend on details that heavy compression removes. A fragmentation-ratio split leaves short, heavily fragmented prompts uncompressed. APCS-cost is simpler still. It leaves every prompt of up to 127 tokens uncompressed. Above that, prompts without a length instruction stay uncompressed unless they

are QA, which get LLMLingua-2 @ 0.5. Prompts with a length instruction get the protected variant @ 0.3, except creative prompts, which get light plain compression (@ 0.7). The selector has in effect learned that compression pays only when the prompt is long and its answer length is either anchored by a protected instruction or naturally short, as in QA.

4.2.7 Changes from the Specification and Design Report

The rubric expects any change from the SDR to be identified and justified. Table 9 lists every material change, the evidence for it and the experiment in which it was decided and recorded in the experiment log. Each was decided before the data it would be evaluated on was examined.

SDR specification	As implemented	Evidence and justification	Decided and logged in
LLM: gpt-4o-mini, max_tokens 512	gpt-5.6-luna, temperature 0, reasoning "none", 1,024 completion tokens (pilot used gpt-4o-mini)	A newer model from the same provider gives results that are more relevant and accurate for current deployments; 512 truncated 9.6% of pilot answers (Exp 04). Same provider and billing structure; one snapshot throughout	Final benchmark set-up (Exp 10)
BERTScore: mBERT primary, AraBERT check	AraBERT primary, mBERT check	Exp 01: AraBERT 50% more dynamic range; mBERT passes random 30% deletion at 0.85 in 86% of cases	Pilot (Exp 01)
Fidelity threshold F1 ≥ 0.85	$\tau = 0.65$ from the model's repeat-call noise ceiling (0.75 for mBERT)	Exps 07 and 10: about 31% of *identical-prompt* answer pairs fall below 0.85, so it cannot separate compression damage from noise	Pilot (Exp 07); τ fixed in Exp 10
LLMLingua with its reference scorer	LLMLingua with Qwen2.5-0.5B	Exp 02: GPT-2 crashes and corrupts Arabic; BLOOM incompatible	Pilot (Exp 02)
Primary metric: Pareto-hit rate	Best-balance accuracy (proposal's own definition); Pareto-hit as footnote	Exp 06 and Table 15: "never compress" always Pareto-optimal, scores 88.5%	Pilot (Exp 06)
Rule engine incl. morphology, structure and creative clauses	APCS 1.0.0: token-count rule; APCS-v2: tree over all features + category	Exps 05 and 10: morphology not predictive when other features are controlled; rules kept only where dev evidence supported them	Exp 05; calibration Exp 10; redesign Exp 11
Creative prompts author-written	Synthetic briefs generated with Claude Fable 5 (different provider from the LLM under test)	Scale and licensing; effect measured in Exp 09	Dataset construction (Exp 08)
Categories 250 × 4	350 / 250 / 250 / 150	ARCD pool exhausted; Aya rows instruction-like (Exp 08)	Dataset construction (Exp 08)
Single held-out evaluation	Held-out test (Exp 10) **plus** fresh pre-registered exam (Exp 11)	Redesign motivated by test results could not be evaluated on the same test without contamination	Exp 11 pre-registration
Recommendation of (method, rate) only	Adds APCS-cost and the length-protected compressor	Exp 10: compression raised the total bill; the SDR's cost-saving objective required a cost-aware design	Exp 11 redesign

Table 9. Changes from the SDR and their justification

None of these changes alters the aim or the research questions. Three of them (the scorer swap, the threshold and the primary metric) were corrections of the SDR's evaluation design that the pilot showed to be necessary. The others kept the study current (the newer LLM), responded to data availability, or followed findings (the cost effect) that the original design could not have anticipated.

4.3 The IT Artefact

The APCS is distributed as an installable Python package (`apcs`, version 1.1.0, MIT licence) in the project repository. Its core dependency is only `tiktoken`. LLMLingua (for applying a recommendation) and farasapy (for morphological density) are optional extras, so recommendations are fast and need no GPU. Table 10 shows the structure.

Module	Responsibility
`features.py`	Normalises the prompt (Unicode NFC) and computes the feature vector: character length, cl100k token count, word count, fragmentation ratio, structural complexity, optional morphological density, and the length-instruction flag
`selector.py`	Decision engine. `APCSSelector("v1")` applies the APCS 1.0.0 rules from `rules_default.json`; `"v2"` and `"cost"` walk the frozen decision trees in `selectors/*.json`. Returns a `Recommendation` (method, rate, the guards that fired, the features and the rules version)
`protect.py`	The length-protected LLMLingua-2 compressor
`cli.py`	Command-line interface: `apcs "<prompt>"` or `apcs --file`, with `--selector`, `--category` and `--json`
`rules_default.json`, `selectors/`	Versioned, frozen calibration data, including the dev set it was trained on, τ and the readable rules

Table 10. Structure of the APCS package

The decision engine treats rules as data: recalibrating for a different LLM or price ratio means producing a new JSON file, not changing code. Each recommendation states the rule that produced it, which meets the explainability requirement. For example, for a 191-token summarisation prompt from the dev set:

```
$ apcs --selector v2 --category summarisation --file prompt.txt
recommendation : llmlingua2 @ rate 0.5
rule fired     : cat_creative <= 0.5 and token_count > 104.5 and cat_qa <= 0.5 and token_count
<= 243
features       : tokens=191 words=41 frag=4.6585 struct=5
rules version  : apcs_v2

$ apcs --selector cost --category summarisation --file prompt.txt
recommendation : llmlingua2_protected @ rate 0.3
rule fired     : token_count > 127.5 and has_length_instruction > 0.5 and cat_creative <= 0.5
```

The two selectors disagree in an informative way. The prompt begins "write a summary *in one paragraph*", a length instruction. APCS-v2, which maximises token reduction, picks

plain LLMLingua-2. APCS-cost picks the protected variant, which keeps the instruction and therefore the answer's length. In library form, the same call is:

```
from apcs import APCSSelector
rec = APCSSelector("cost").recommend(prompt, category="summarisation")
compressed = APCSSelector("cost").compress(prompt, category="summarisation")
```

Testing. The package has 18 unit tests. They check feature extraction (empty input rejected, Farasa's punctuation excluded from morphological density), the APCS 1.0.0 rule branches, custom rule files, serialisation, the tree selectors' category handling and their treatment of a missing optional feature. One integration test feeds all 400 exam prompts through the package's own feature extraction and confirms that the shipped APCS-v2 and APCS-cost trees choose *exactly* the strategy recorded in the pre-registered exam evaluation. The artefact a user installs is therefore the one that was evaluated.

Documentation. The package README serves as the user manual, covering installation, the command-line and library interfaces, the three selectors and recalibration. An example notebook (`apcs/examples/apcs_example.ipynb`) walks through single and batch recommendations, applying a recommendation and custom calibration. Together they deliver the manual and example notebook promised in the proposal.

Overhead. A supplementary measurement (Experiment 12; 100 dev prompts, rate 0.5) timed each step on the project workstation. The APCS recommendation itself takes about 0.05 ms per prompt, so feature extraction and tree evaluation are not a runtime bottleneck. Applying a recommendation does take time: LLMLingua-2 needs about 30 ms per prompt on the consumer GPU and about 0.4 s on CPU, because it runs a 560-million-parameter encoder; the protected variant costs the same. For prompts of a few hundred tokens, this is of the same order as, or larger than, the prompt-processing time that compression removes. Compression should therefore be justified by cost or context-window limits rather than by speed. End-to-end API latency was not measured.

Reproducibility and verification. Every result file is accompanied by its configuration (model snapshot, parameters, prices, seeds). Before writing began, an independent audit script that imports no experiment code recomputed every headline number in this dissertation from the raw data. It covered the dataset sizes and overlaps, the 13,438 and 3,828

stored responses and their costs, the RQ1–RQ3 statistics and the exam hypotheses, and all 117 checks passed (Section 5.2.10).

4.4 Chapter Summary

The design was implemented as a staged pipeline. The pilot changed the scorer, threshold, LLMLingua scorer, completion limit and primary metric, and the redesign added a length-protected compressor and two decision-tree selectors evaluated on a pre-registered exam. The artefact is a small, tested package whose shipped selectors reproduce the evaluated ones exactly.

Chapter 5. RESULTS AND EVALUATION

5.1 Chapter Introduction

This chapter evaluates the compression methods and the APCS against the research questions: measurement validity (Section 5.2.1), RQ1 and RQ2 on the 800 dev prompts with the cost finding (5.2.2–5.2.5), the APCS on held-out data (5.2.6–5.2.8), robustness (5.2.9–5.2.10) and the answers (5.2.11). Unless stated otherwise, fidelity is output-level AraBERT BERTScore F1 with $\tau = 0.65$, and costs come from billed tokens.

5.2 Evaluation of IT Artefact

5.2.1 Measurement validity: the noise ceiling

Before compression can be judged, the measuring instrument must be shown to separate compression damage from chance. Sending each uncompressed dev prompt twice to the LLM, at temperature 0 with no reasoning, produced identical text only **29.6%** of the time. The median F1 between the two answers was 0.930, and **31.8% of identical-prompt pairs scored below 0.85** (Figure 2). The pilot model gave almost the same figure (31.2%), so the effect is not specific to one model. This replicates the non-determinism reported by Ouyang et al. (2025) and Song et al. (2025), and has a direct consequence. The SDR's threshold of 0.85 would have classified almost a third of *unaltered* prompts as damaged, and any compressor would have "failed" on many prompts through no fault of its own.

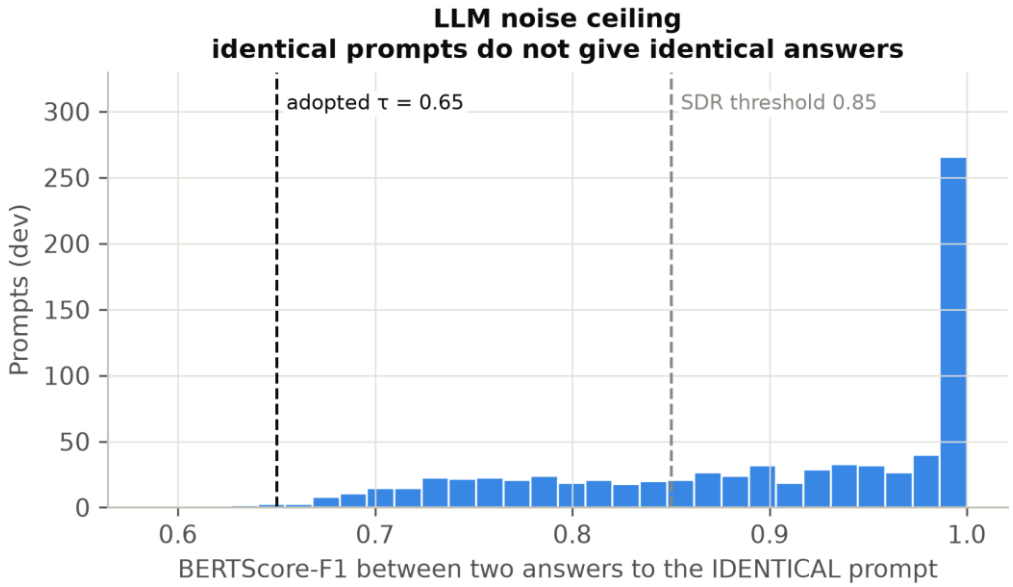


Figure 2. Repeat-call noise ceiling on the dev set: distribution of output F1 between two answers to the identical prompt, with the SDR threshold (0.85) and the adopted τ (0.65)

With τ at the 0.4th percentile of this distribution (Section 3.4.4), a compressed prompt scoring below it has almost certainly been damaged. The ceiling differs sharply by task (Table 11). QA answers restate facts and repeat exactly two times in three, while creative answers almost never repeat and 95.8% of identical-prompt creative pairs fall below 0.85. Raw F1 therefore flatters deterministic tasks and penalises creative writing, which Section 5.2.3 corrects for. A single global τ is also lenient for QA, whose noise rarely falls below 0.77, and slightly strict for creative writing.

Category	Identical answers	Median F1	Below 0.85	0.4th percentile
QA	66.5%	1.000	5.0%	0.768
Summarisation	20.0%	0.947	9.5%	0.713
Instruction	22.1%	0.881	39.3%	0.613
Creative	1.7%	0.743	95.8%	0.636
All dev prompts	29.6%	0.930	31.8%	0.640

Table 11. Repeat-call noise ceiling by task category (dev, AraBERT)

5.2.2 RQ1: effectiveness of compression methods on Arabic

Figure 3 shows the trade-off between tokens removed and output fidelity for each method on the 800 dev prompts. **LLMLingua-2 dominates random deletion at every rate**: at the same achieved TCR it gives higher fidelity, and Table 12 shows the paired differences are highly significant.

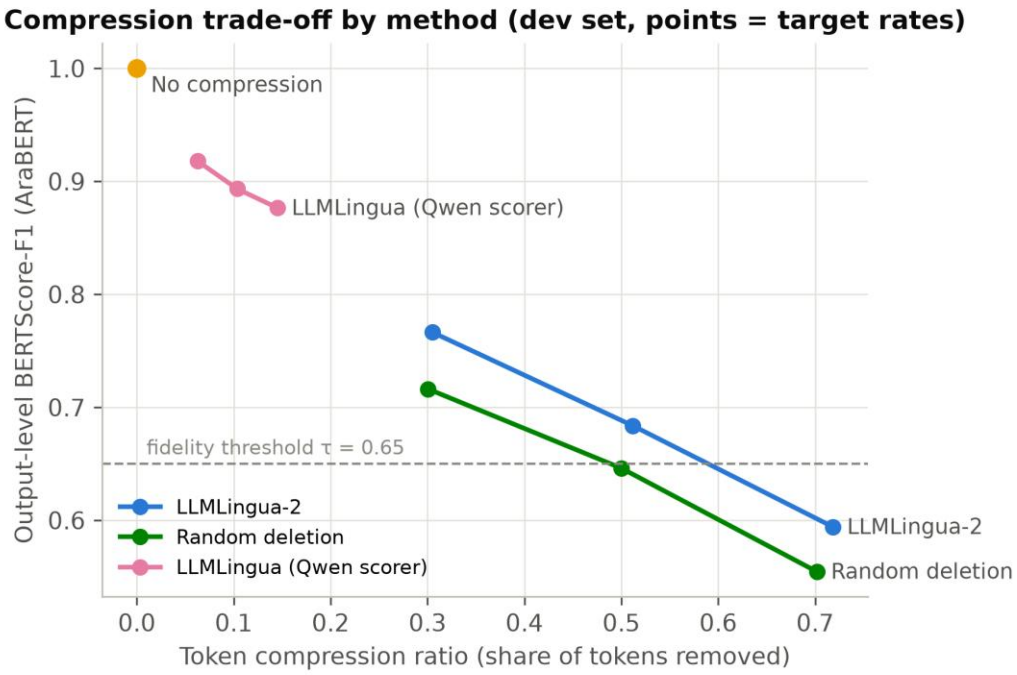


Figure 3. Compression trade-off by method on the dev set: mean TCR against mean output F1 at target rates 0.7, 0.5 and 0.3

Target rate	LLMLingua-2	Random deletion	Difference [95% CI]	Wilcoxon p
0.7	0.766	0.716	**+0.050** [0.041, 0.059]	2×10^{-29}
0.5	0.684	0.646	**+0.038** [0.029, 0.046]	6×10^{-20}
0.3	0.594	0.554	**+0.040** [0.032, 0.047]	6×10^{-25}

Table 12. Output-level fidelity of LLMLingua-2 vs random deletion (dev, n = 800, paired)

The advantage is real but modest, about 0.04–0.05 F1. LLMLingua-2's classifier, trained on English distillation data, carries a useful notion of token importance into Arabic, but it is far from lossless. At rate 0.5, mean fidelity (0.684) is only just above τ, and at 0.3 it falls below τ on average. Arabic prompts tolerate light compression; heavy compression damages most answers.

Three dev prompts show what the difference means in practice (rate 0.5). An instruction asked for one example of how renewable energy can be used. LLMLingua-2 kept فكر مثال استخدام الطاقة المتجددة ("think example use renewable energy") and the answer gave one example (F1 0.82); random deletion kept التي يمكن استخدام الطاقة المتجددة فيها ("which renewable energy can be used in"), losing "one example", and the model listed six uses (0.54). A summarisation prompt about coal lost the word الفحم ("coal") under random deletion, and the model summarised building stone instead (0.55, against 0.82). A QA prompt about Emirates'

founding year kept its question under LLMLingua-2 and received a one-line answer; random deletion removed "answer the question", and the model added unrequested facts (0.58, against 0.79). LLMLingua-2's output is telegraphic but keeps the task-defining words, whereas random deletion keeps fluent fragments and loses them.

The ranking flip. At *prompt* level the order reverses. Random deletion scores higher than LLMLingua-2 at every rate (−0.014, −0.026 and −0.028; all $p < 10^{-14}$; Figure 4). BERTScore rewards verbatim retention of surface words, and random deletion keeps whole words scattered evenly through the prompt, while LLMLingua-2 deletes function words and morphological fragments that it rightly judges uninformative. An evaluation that compared prompts, as many compression studies effectively do when they report prompt similarity, would have ranked the methods backwards. This finding, first observed in the pilot (Experiments 03 and 04) and replicated on v2, is a methodological contribution: **for Arabic, compression must be evaluated on the LLM's outputs, not on the compressed prompts.**

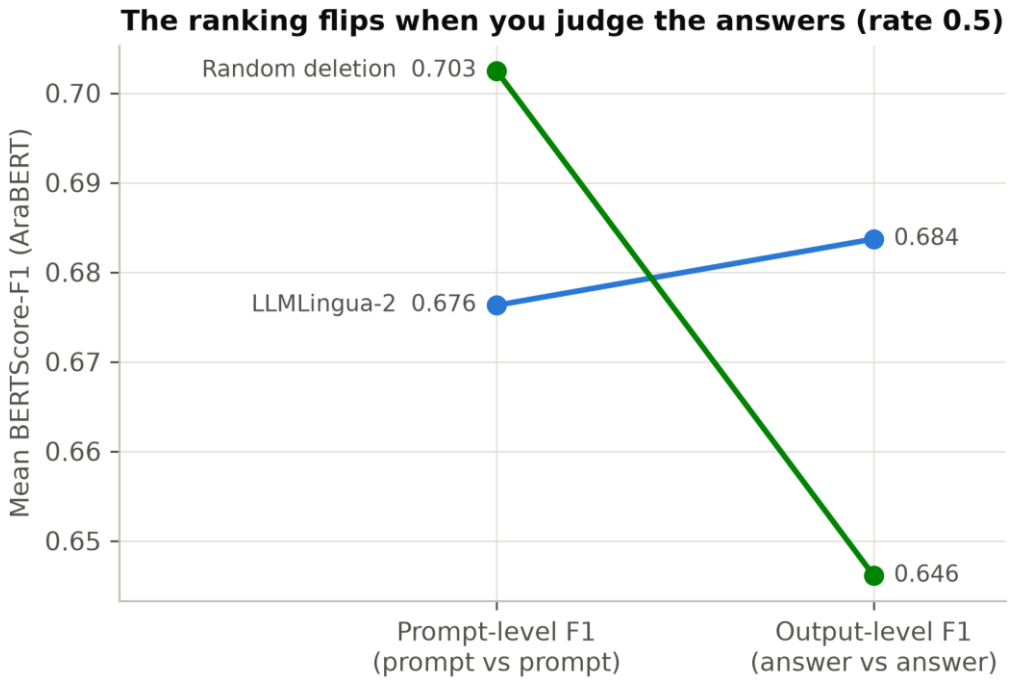


Figure 4. The ranking flip: LLMLingua-2 minus random deletion at prompt level (negative) and output level (positive)

LLMLingua versus LLMLingua-2. LLMLingua with the Qwen scorer behaves differently. It refuses to compress most prompts: 50–83% are returned untouched, depending on category. Its achieved keep is strongly tied to prompt length (Spearman ρ between tokens and

keep from −0.71 to −0.79). This is the intrinsic length gate first seen in Experiment 02. At the same *target* rate the two methods are therefore not comparable, because LLMLingua simply compresses less. When each prompt that LLMLingua did compress is paired with the LLMLingua-2 variant closest in *achieved* keep, LLMLingua-2 is better by +0.030 [0.014, 0.045] at keep ≈ 0.52 and +0.041 [0.025, 0.056] at keep ≈ 0.66. The two do not differ at the lightest setting, where LLMLingua kept more text. Classifier-based scoring therefore transfers to Arabic better than perplexity-based scoring, consistent with Pan et al.'s (2024) argument that unidirectional entropy is poorly aligned with compression.

In summary for RQ1: published English defaults do not transfer (GPT-2 scoring corrupts Arabic); with a multilingual scorer, perplexity-based compression barely compresses short Arabic prompts; and classifier-based LLMLingua-2 is the most effective method, beating random deletion by a modest, consistent margin.

5.2.3 Category fragility and task accuracy

Dividing each category's mean LLMLingua-2 @ 0.5 fidelity by its own median noise ceiling gives the share of attainable fidelity that survives compression: creative 83.7%, summarisation 76.7%, instruction 75.4% and **QA 70.6%**. On raw F1, creative answers looked the most damaged. Once their natural variability is accounted for, the order reverses: QA answers, which depend on specific facts that compression can delete, lose the most. Creative answers vary anyway, so compression costs them relatively little.

QA correctness on the exam confirms this with a measure independent of BERTScore. Uncompressed prompts produced answers containing the gold answer in **57%** of QA cases. Compression lowered this to 46–53% with LLMLingua, 32–46% with LLMLingua-2 (decreasing with rate), 31–47% with the protected variant and 21–31% with random deletion. The methods rank in the same order as on BERTScore, which supports the validity of the similarity measure, and the size of the drop shows that heavy compression is unsafe for factual QA.

The drop has a direct mechanism: compression deletes the answer. All QA prompts are extractive, so the gold answer appears in the passage. When it survived compression, answers were correct 71.5% of the time; when it was deleted, 17.1% (1,200 compressed exam variants; Experiment 13). LLMLingua-2 kept the answer span in 55%, 35% and 23% of prompts at rates 0.7, 0.5 and 0.3, random deletion in 31%, 16% and 10%, and LLMLingua, which barely compresses, in 60–66%. Correctness falls in the same order as answer retention, so a compressor that knew which span the question targets would avoid most of the loss.

5.2.4 RQ2: which prompt features matter

Figure 5 shows the rank correlation of each feature with output fidelity at rate 0.5. For LLMLingua-2, longer prompts survive compression better (ρ = +0.28): they contain more redundancy for the compressor to remove. Structural complexity correlates positively (+0.19), fragmentation weakly negatively (−0.12), and morphological density not at all (+0.01). For LLMLingua the length correlation is strongly *negative* (−0.67), because its gate leaves short prompts untouched and therefore perfectly faithful.

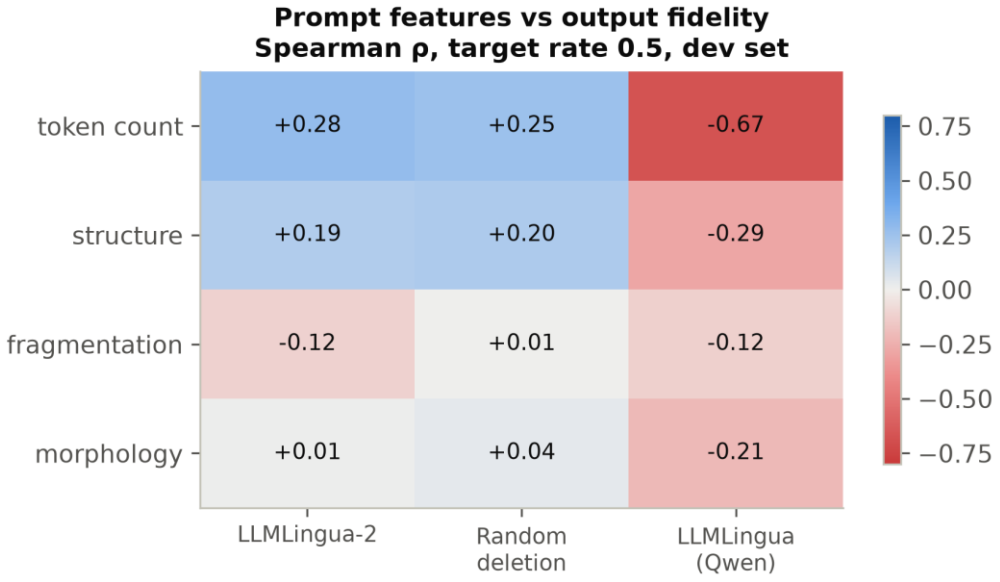


Figure 5. Spearman correlation between prompt features and output fidelity at target rate 0.5 (dev)

Because v2 was deconfounded by design (Section 3.4.1), length and task type can be separated:

- **Length helps within every category.** Spearman correlations with output F1 are creative +0.33, summarisation +0.27 and instruction +0.23 (all $p < 0.001$), and QA +0.13 ($p = 0.07$).
- **Category matters within every length band.** Kruskal–Wallis $p = 0.011$ (short), 10^{-13} (medium) and 3×10^{-8} (long). Summarisation prompts are the most tolerant, creative the least.
- **Joint model.** An OLS regression of output F1 on all features and category explains $R^2 = 0.148$. Removing category reduces R^2 by **6.5 points** and removing length by **3.4 points**. Among standardised features, log length ($\beta = +0.027$, $p = 3 \times 10^{-8}$) and fragmentation ($\beta = -0.012$, $p = 0.007$) are significant; structure ($\beta = -0.003$) and **morphological density ($\beta = +0.002$, $p = 0.65$) are not.**

Predictor	Standardised β	p	Unique R^2
Task category (3 indicators)	—	—	**0.065**
log token count	+0.027	3×10^{-8}	**0.034**
Fragmentation ratio	−0.012	0.007	—
Structural complexity	−0.003	n.s.	—
Morphological density	+0.002	0.65	—

Table 13. OLS model of LLMLingua-2 @ 0.5 output fidelity (dev, $R^2 = 0.148$)

Two conclusions follow. First, **task type and length both matter, and task type slightly more.** This revises the pilot's conclusion that length dominated. In the pilot, short prompts were almost all QA or creative, so part of the apparent length effect was a task-type effect. Second, **morphological density, the feature on which the SDR built its Arabic-specific rule, does not predict compression outcomes once length and category are controlled.** This agrees with Arnett and Bergen (2025), who find that morphological type itself confers no intrinsic disadvantage. What does matter is the tokeniser: fragmentation has a small but significant negative effect, as Limisiewicz, Balhar and Mareček (2023) and Ahia et al. (2023) would predict. The low overall R^2 is itself informative: most of the variation in fidelity is prompt-specific and not captured by these surface features, which caps the accuracy any feature-based selector can reach.

Why does morphology fail despite the theoretical expectation? Experiment 13 points to three reasons. Its spread is small: density averages 1.69 segments per word with a coefficient of variation of only 10.5%, because MSA prompts share similar clitic rates. Much of that variation is already captured by fragmentation (Spearman 0.59), which measures the

tokeniser's response to morphology rather than morphology itself; the variance inflation factors (1.35–1.54) show this is redundancy, not harmful collinearity. And compression acts on tokens, not morphemes, so what matters is how the tokeniser splits words, which fragmentation measures directly.

5.2.5 The cost finding: fewer input tokens, a bigger bill

The proposal's fourth metric was cost saving. Table 14 compares the input tokens saved with the change in the *total* measured bill on the dev set.

Policy	Input tokens saved	Total cost change	Median answer length vs uncompressed
LLMLingua-2 @ 0.7	29.7%	**+4.6%**	×1.06
LLMLingua-2 @ 0.5	49.1%	**+10.9%**	×1.19
LLMLingua-2 @ 0.3	67.7%	**+24.3%**	×1.52
LLMLingua @ 0.7 / 0.5 / 0.3	8.4–23.9%	−2.2 to −3.0%	×1.00
Random deletion @ 0.3	66.1%	−27.0%	×1.00 (fidelity 0.55)

Table 14. Input-token saving vs total measured cost change (dev, output priced at 6× input)

LLMLingua-2 removed up to two-thirds of the input tokens and still raised the bill by 5–24% (Figure 6), because answers grew. The break-even condition of Section 2.4 makes the threshold concrete. On dev, uncompressed calls billed on average 109 input and 127 output tokens, so with $\pi_{out} = 6\pi_{in}$

$$g^* = \frac{1}{6} \cdot r \cdot \frac{109}{127} \approx 0.143r$$

At rate 0.5, where 49.1% of input tokens were removed, the bill falls only if total answer length grows by less than 7.0%; it grew by 19.5%. The corresponding figures are 4.3% against 9.5% at rate 0.7 and 9.7% against 37.4% at rate 0.3. Under 6:1 pricing, modest answer growth wipes out large input savings.

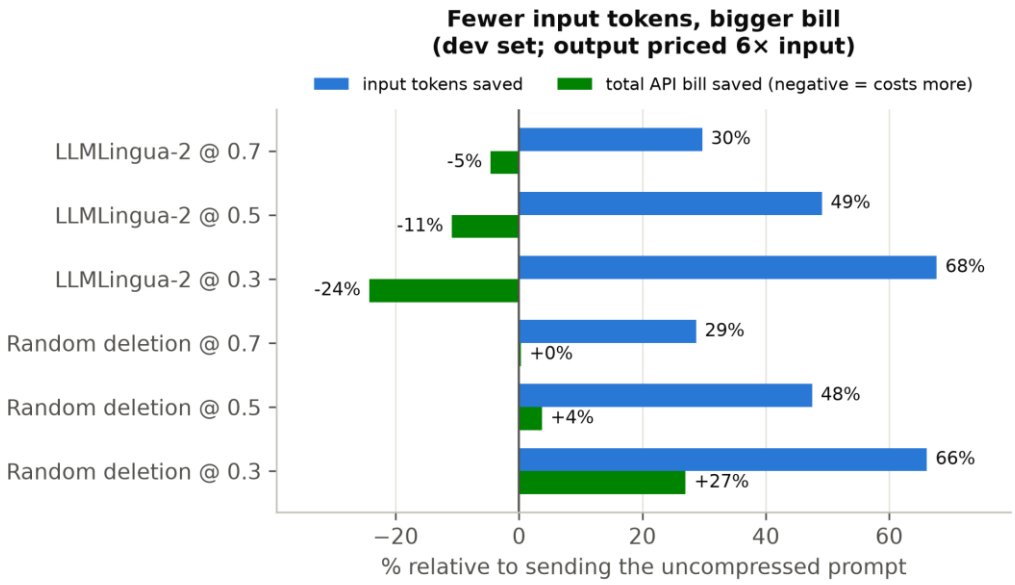


Figure 6. Input tokens saved vs total API bill saved per policy (dev; negative bill saving = costs more)

The mechanism was tested, not assumed. 347 dev prompts contain an explicit length instruction ("answer briefly", "in three sentences", "no more than fifty words"). LLMLingua-2 deleted that instruction in 58%, 39% and 21% of cases at rates 0.3, 0.5 and 0.7. When it was deleted, answers were **1.3–2.4 times longer**; when it was kept, about 1.1 times (Mann–Whitney $p \leq 2 \times 10^{-5}$ at every rate). The classifier scores words such as "briefly" or "three" as low-information, and in a sense they are: they carry little *content*. But they are exactly the tokens that control *how much* the model writes, which is consistent with the length-control literature (Jie et al., 2024; Han et al., 2025). Random deletion removes length instructions just as often, but its garbled prompts produce shorter, lower-quality answers, so it "saves" money only by destroying fidelity. Appendix D gives a worked example: on one exam QA prompt, plain compression deleted "answer briefly", the answer grew from 23 to 85 tokens and the call cost nearly three times as much, while the protected variant kept the answer short.

To the author's knowledge this is a new finding: for any model whose output tokens cost several times its input tokens, **compression ratio is not a proxy for cost saving**, and a compressor can raise the bill it was meant to reduce.

5.2.6 RQ3 (stage 1): APCS 1.0.0 on the held-out test split

APCS 1.0.0 (Section 4.2.5) was evaluated once on the 200 untouched test prompts. The test labels were close to uniform (LLMLingua-2 @ 0.3: 29%, @ 0.5: 28.5%, none: 22%, @ 0.7: 20.5%), which makes this a hard four-way choice in which the best fixed strategy can reach at most 29% (chance level is 25%).

Policy	Accuracy [95% CI]	Mean TCR	Mean out-F1	Below τ	Cost vs none	Pareto-hit
APCS 1.0.0	**31.0%** [24.5, 37.5]	0.43	0.750	39.0%	+34.2%	79.5%
always LLMLingua-2 @ 0.3	29.0%	0.72	0.595	71.0%	+37.8%	86.5%
always LLMLingua-2 @ 0.5	28.5%	0.51	0.676	45.0%	+19.0%	66.5%
random selection	27.0%	0.39	0.751	35.5%	+14.3%	75.5%
never compress	22.0%	0.00	1.000	0.0%	0.0%	**88.5%**
always LLMLingua-2 @ 0.7	20.5%	0.30	0.752	22.5%	+9.6%	64.0%
always LLMLingua (best rate)‡	6.0%	0.11	0.883	9.5%	−1.2%	78.0%

Table 15. Selector and baseline performance on the held-out test split (n = 200)

‡ Compared on extended labels that admit LLMLingua candidates; under the primary labels it can never be correct.

APCS 1.0.0 had the highest accuracy and was **significantly better** than never compressing (+9.0 points, 95% CI [1, 17], McNemar p = 0.038), always LLMLingua-2 @ 0.7 (+10.5, p = 0.048) and always LLMLingua at every rate ($p < 10^{-8}$). It was **not significantly better** than always LLMLingua-2 @ 0.5 (+2.5, p = 0.61), always @ 0.3 (+2.0, p = 0.75) or random selection (+4.0, p = 0.45). The multilingual BERT re-run, with its own τ and recalibrated rule, gave the same picture (34% accuracy). APCS 1.0.0 delivered a sensible balance: the TCR class of random selection, far fewer fidelity violations than always compressing at 0.3, and per-category accuracy between 29% and 34%. But measured by the primary metric, a one-feature rule was only marginally better than the best fixed strategy. The drop from 38.0% on dev to 31.0% on test is the expected optimism of in-sample calibration.

The Pareto-hit column shows why the SDR's metric was replaced. "Never compress" scores 88.5%, the highest of any policy, because its fidelity of 1.0 can never be dominated. A metric that rewards doing nothing cannot evaluate a compression selector.

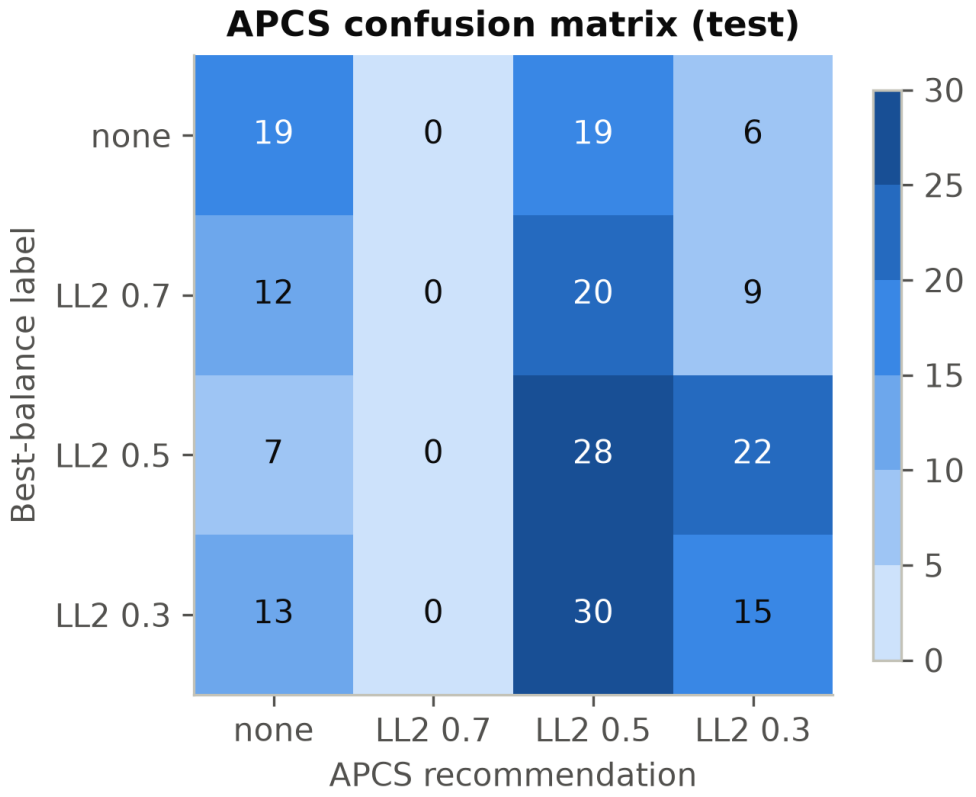


Figure 7. Confusion matrix of APCS 1.0.0 recommendations against best-balance labels (test)

The confusion matrix (Figure 7) shows where the rule fails. It never recommends LLMLingua-2 @ 0.7, although that was the best choice for 20.5% of test prompts. It also recommends @ 0.5 for many prompts whose best choice was no compression or @ 0.3. Token count alone cannot tell these apart. RQ2 had already indicated what was missing, namely task category, and the cost analysis indicated that the target itself (token reduction) was misaligned with the bill. Both were followed up in Experiment 11, on fresh data.

5.2.7 RQ3 (stage 2): the redesigned selectors on the fresh exam

All three selectors were evaluated once on the 400 never-seen exam prompts under the pre-registered protocol. Table 16 gives the results and Figure 8 the accuracies.

Hypothesis	Result	Holm p	Verdict
H1: APCS-v2 > always LLMLingua-2 @ 0.5	38.5% vs 32.8%, **+5.8 points** [0.8, 10.5]	0.028	**Supported**
H2: APCS-v2 > APCS 1.0.0	38.5% vs 31.2%, **+7.2 points** [2.5, 12.0]	0.008	**Supported**
H3: APCS-cost bill < never compress	total **−1.0%**; per-prompt one-sided Wilcoxon	0.001	**Supported** (small effect)

Table 16. Pre-registered hypotheses on the fresh exam (n = 400; exact two-sided McNemar for H1 and H2, conservative for directional hypotheses)

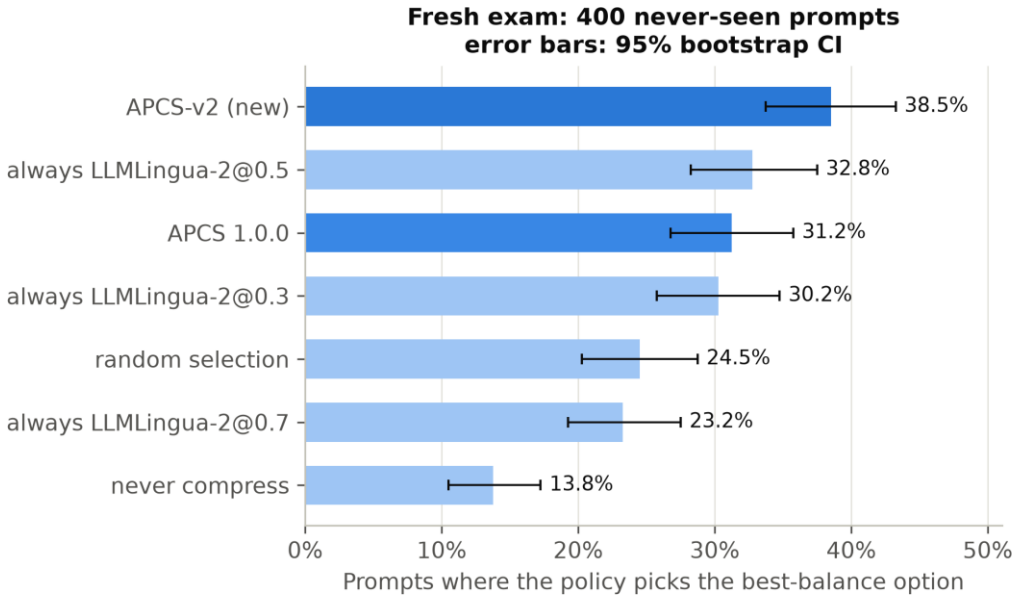


Figure 8. Best-balance accuracy of each policy on the fresh exam, with 95% bootstrap confidence intervals

Policy	Accuracy [95% CI]	Mean TCR	Mean out-F1	Below τ	Cost vs none	QA correct
APCS-v2	**38.5%** [33.8, 43.3]	0.48	0.709	34.2%	+15.9%	41%
always LLMLingua-2 @ 0.5	32.8% [28.3, 37.5]	0.51	0.683	39.8%	+12.1%	40%
APCS 1.0.0	31.2% [26.8, 35.8]	0.43	0.757	34.5%	+19.7%	40%
always LLMLingua-2 @ 0.3	30.2%	0.72	0.602	69.8%	+27.1%	32%
random selection	24.5%	0.38	0.765	30.5%	+7.5%	42%
always LLMLingua-2 @ 0.7	23.2%	0.31	0.765	16.0%	+8.4%	46%
APCS-cost	19.5%†	0.22	**0.880**	**9.2%**	**−1.0%**	42%
always LLMLingua @ 0.3	—	0.17	0.862	17.5%	−4.0%	46%
never compress	13.8%	0.00	1.000	0.0%	0.0%	57%

Table 17. All policies on the fresh exam (n = 400)

† APCS-cost optimises a different target. On its own cheapest-faithful labels it scores 35.7%, the highest of any policy.

H1 and H2. APCS-v2 was the most accurate policy and significantly better than every alternative. For reference, chance level for this four-way choice is 25%, and random selection scored 24.5%. Beyond the pre-registered comparisons, it beat always @ 0.3 by +8.3 points ($p = 0.027$), always @ 0.7 by +15.2 ($p < 10^{-4}$), random selection by +14.0 ($p < 10^{-4}$) and never compressing by +24.7 ($p < 10^{-12}$); these secondary tests are uncorrected. It also had fewer fidelity violations than the best fixed strategy (34.2% vs 39.8%). The by-category breakdown shows where the gain comes from. For APCS-v2, APCS 1.0.0 and always @ 0.5 respectively, accuracy was creative 45 / 23 / 20%, summarisation 53 / 46 / 39%, instruction 30 / 24 / 29% and QA 32 / 31 / 40%. Treating creative and summarisation prompts differently produces the gain. On QA, a fixed 0.5 rate still does better than either selector, a limitation discussed in Chapter 6.

The QA weakness traces to two tree branches (Experiment 13). On dev, very long QA passages were best compressed lightly, so the tree learned "QA over 433 tokens → LLMLingua-2 @ 0.7"; on the exam that branch was right for 1 of 16 prompts, whose labels favoured 0.5 or 0.3, while a fixed 0.5 was right for 7. Short QA prompts with many structural markers (under 105 tokens, structural complexity above 6.5) were sent to 0.7 and were right 21% of the time, against 36% for 0.5. Both rules rest on small dev leaves and did not generalise; the remaining QA branch (105–433 tokens → 0.5) matched the fixed strategy exactly.

Replication. APCS 1.0.0 scored 31.2% on the exam against 31.0% on the test split. Its held-out performance replicated almost exactly on independent data, which supports the reliability of both evaluations.

H3 and the accuracy–cost tension. APCS-cost met its pre-registered criterion: the per-prompt cost difference was significantly negative and the total fell by 1.0%. It did so while keeping fidelity high (mean 0.880; only 9.2% of answers below τ, the lowest of any compressing policy). The effect, however, is small. The bootstrap interval for the *mean* per-prompt saving spans zero, so the saving is consistent in direction but modest in total, and is reported as such. APCS-v2, which optimises the proposal's best-balance target, raised the bill by 15.9%. A simple fixed LLMLingua @ 0.3 saved more (−4.0%), but with nearly

twice APCS-cost's rate of low-fidelity answers. **Accuracy against the best-balance label and cost pull in different directions**, and no single selector maximises both.

5.2.8 The length-protected compressor

The protected compressor was evaluated on the 180 exam prompts that contain a length instruction (Table 18; Figure 9).

Target rate	Cost: plain → protected	Output F1: plain → protected	TCR: plain → protected
0.7	+7.2% → +0.8%	0.762 → 0.803	0.30 → 0.22
0.5	+17.6% → **−5.3%**	0.677 → 0.747	0.51 → 0.36
0.3	+47.6% → **−7.9%**	0.597 → 0.687	0.71 → 0.51

Table 18. Plain vs length-protected LLMLingua-2 on exam prompts with a length instruction (n = 180)

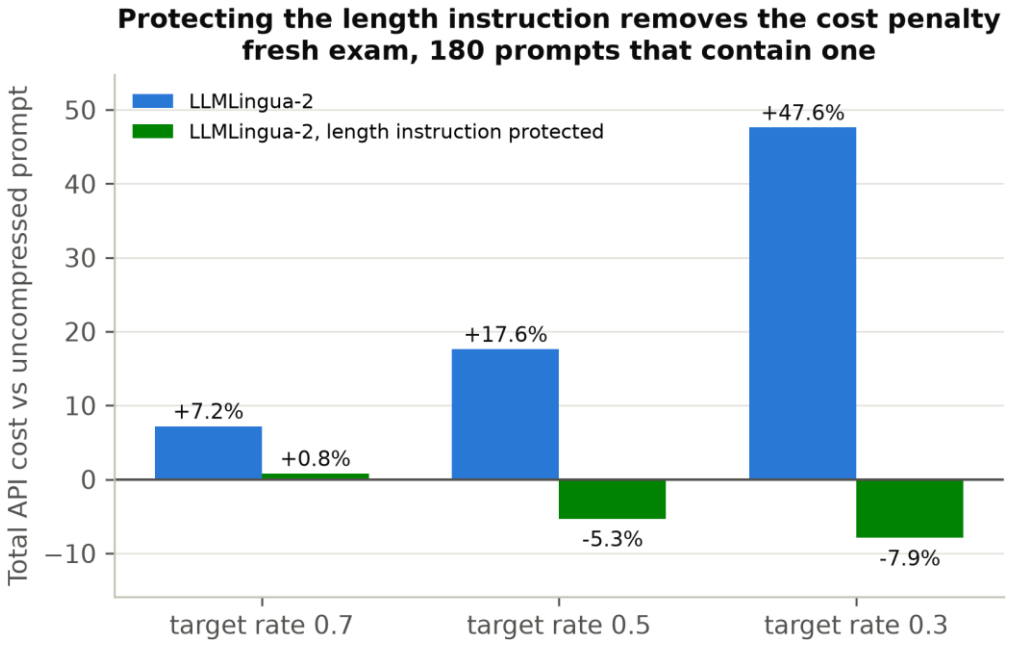


Figure 9. Total API cost relative to the uncompressed prompt, plain vs protected LLMLingua-2 (exam prompts with a length instruction)

Keeping the length instruction verbatim turned the largest cost surge, +47.6% at rate 0.3, into a net saving of 7.9%, and +17.6% at rate 0.5 into −5.3%. It also raised fidelity by 0.04–0.09, at the price of compressing less. The same pattern had been found on dev before the exam was run, so this is a replicated result. It shows that under output-weighted pricing the largest lever is the compressor, not the selector: protecting the few tokens that control answer length is what turns compression into a saving.

5.2.9 Synthetic data sensitivity

Experiment 09 tested whether the 15% synthetic share of v2 could have biased the results, in response to a question from the dissertation advisor. (a) Calibration: APCS was recalibrated on 200 bootstrap mixes at 15%, 30% and 50% synthetic share, with category and length quotas held fixed. The learned rule barely changed: the no-compression threshold moved by one grid step (40 → 60 tokens), the chosen rates never changed, and accuracy moved by 0.6 points (0.380 → 0.374). (b) Fidelity: a separate pool of 456 synthetic instruction, summarisation and QA prompts, generated with Claude Fable 5, was compared with real prompts of the same category, weighted to the same length mix. In eight of nine category × rate cells the 95% interval excluded zero. Synthetic instruction and summarisation prompts lost 0.02–0.05 more fidelity under compression, and the QA differences changed sign with rate (Figure 10).

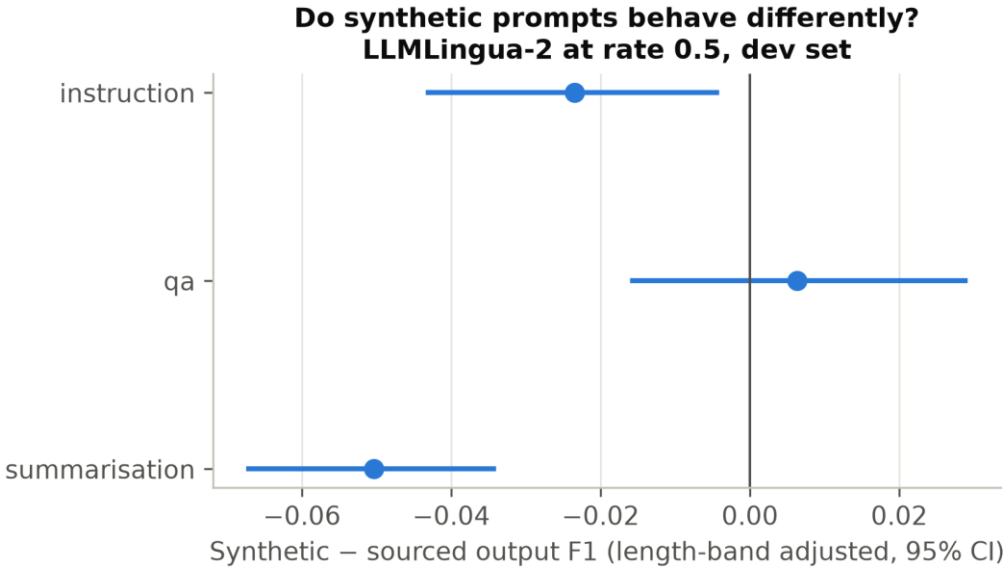


Figure 10. Fidelity difference between synthetic and real prompts by category and rate, with 95% bootstrap intervals

Synthetic prompts are therefore safe for *calibrating* a selector but **not a drop-in substitute for real prompts when measuring fidelity**: they lost 0.02–0.05 more F1 than real prompts of the same category and length, the same order as the 0.04–0.05 method differences in Table 12. Generated briefs are also more uniform than human requests in topic, register and phrasing, and all come from one model family, so they may understate the variety a deployed system faces. The headline method comparisons therefore rest on the 850 real

prompts plus the 150 synthetic creative prompts confined to one category, and creative-category results carry this caveat (Long et al., 2024; Liu et al., 2024).

5.2.10 Robustness and verification

The main results were checked in five ways.

- **Scorer.** Every RQ1 comparison holds under multilingual BERT (LLMLingua-2 beats random deletion by +0.017 to +0.033, all $p < 10^{-14}$), and the mBERT APCS evaluation reaches the same conclusions.
- **Scorer window.** Removing the 2.6% of pairs that exceed AraBERT's 510-word-piece window changes the RQ1 differences by at most 0.002.
- **Length bias in the metric.** Answers that grew by more than 50% after compression (36% of LLMLingua-2 @ 0.5 cases) scored 0.634, against 0.712 for the rest (Spearman −0.47 between absolute log length change and F1), so part of the measured fidelity loss reflects length change, as Section 2.3.5 predicts. Method comparisons are unaffected, since every method is scored the same way, but absolute F1 understates content preservation when answers grow.
- **Replication across data.** The ranking flip, the method ordering, the cost penalty and APCS 1.0.0's accuracy all replicated from pilot to v2 and from test to exam. The size of the cost penalty varies between prompt sets: LLMLingua-2 @ 0.5 adds 10.9% on dev, 19.0% on test and 12.1% on the exam. Its direction never changes.
- **Independent audit.** A script that imports none of the experiment code recomputed every number cited in this chapter from the raw files. It re-joined all 13,438 and 3,828 stored responses to the results tables, recomputed costs from token counts, re-applied the frozen selectors and re-ran the statistical tests. All 117 checks passed. The audit found three documentation errors in the experiment log, which were corrected, and none in the data or results.

5.2.11 Answering the research questions

RQ	Answer	Key evidence
RQ1: effectiveness on Arabic	**Partly effective.** English defaults fail (GPT-2 scoring corrupts Arabic). Classifier-based LLMLingua-	Table 12; Figures 3 and 4

RQ	Answer	Key evidence
	2 transfers best, beating random deletion by 0.04–0.05 F1 at every rate, and beats compression-matched LLMLingua. Light compression is tolerable; heavy compression damages most answers and lowers QA correctness from 57% to as little as 32%. Prompt-level evaluation ranks methods backwards.	
RQ2: feature effects	**Task type and length matter; morphology does not.** Category explains 6.5% and length 3.4% of unique variance; fragmentation has a small negative effect; morphological density is not significant. QA is the most fragile category once noise is accounted for. Prompt-level features explain only 15% of the variation.	Table 13; Figure 5
RQ3: can a selector choose accurately?	**Yes, with a modest effect.** A tree over task category and length (APCS-v2) beats the best fixed strategy on unseen prompts (+5.8 points, Holm $p = 0.028$) and the rule-based APCS 1.0.0 (+7.2, $p = 0.008$). Absolute accuracy (38.5%) remains limited by the noisy, near-uniform labels. A cost-aware variant lowers the bill only slightly (−1.0%) while keeping fidelity high.	Tables 15–17; Figure 8
Additional finding	**Compression can raise the bill.** Under 6:1 output pricing, LLMLingua-2 deletes length instructions, answers grow 1.3–2.4×, and total cost rises 5–24%. Protecting length instructions reverses this (−5 to −8%) and raises fidelity.	Tables 14 and 18; Figures 6 and 9

Table 19. Answers to the research questions

5.3 Chapter Summary

A third of identical-prompt answer pairs fail the SDR's 0.85, so a noise-derived τ was used. LLMLingua-2 is the most effective compressor for Arabic, and output-level evaluation is essential. Task type and length, not morphology, drive outcomes. APCS-v2 beat every fixed strategy on the pre-registered exam. Under 6:1 pricing, compression raised the bill, and protecting length instructions turned that into a saving.

Chapter 6. CONCLUSIONS

6.1 Chapter Introduction

This chapter reflects on the project: lessons in three domains and the project plan (Section 6.2), strengths and weaknesses (6.3), academic and business implications (6.4–6.5) and future research (6.6).

6.2 Lessons Learned

Domain-specific NLP insights. Arabic's rich morphology, the starting hypothesis behind the SDR's decision rules, did not decide compressibility; task type and length did, and the tokeniser mattered only through fragmentation. Morpheme boundaries are not token boundaries (Section 2.2), so linguistic structure reaches a token-deletion compressor only through the tokeniser, and an English-trained default (GPT-2) can even corrupt Arabic text. Linguistic intuition is a good source of hypotheses, but it must be tested against the tokeniser actually in use.

LLM system economics. Cheaper input is not a cheaper call. When output costs six times input, words that carry almost no content, such as "briefly", are among the most valuable in the prompt, and an answer only 7% longer cancels halving the prompt. Any prompt intervention, whether compression, translation or templating, should be judged on the whole exchange and the whole bill, and the selector is a smaller lever than the compressor itself.

Empirical evaluation methodology. The instrument must be validated before it is used: a fixed 0.85 threshold would have failed a third of unaltered prompts, and prompt-level similarity would have ranked the methods backwards. Development-only pilots, one documented experiment per question, pre-registration before a one-use exam and an independent audit before writing each proved their value; the audit found three documentation errors and none in the results, and the morphology defect was corrected openly as an erratum.

About the project plan. Table 20 summarises the SDR's plan and what each phase delivered.

Phase (SDR plan)	Planned	Outcome	Note
Dataset construction	18 May – 14 Jun	Pilot set (500) and AraPromptBench v2 (1,000) built and frozen	v2 deconfounded by length band, informed by the pilot
Framework implementation	15 Jun – 12 Jul	Pipeline built incrementally through Exps 01–07	Resumable and deduplicating
Experimental benchmarking	13 Jul – 9 Aug	Full benchmark: 13,438 calls, one model snapshot	Run on gpt-5.6-luna for current results
APCS design and implementation	10 Aug – 6 Sep	Pilot package, APCS 1.0.0, then redesign (APCS-v2, APCS-cost)	Redesign added beyond the plan
Artefact evaluation	7 – 27 Sep	Held-out test evaluation and fresh pre-registered exam	Exam added beyond the plan
Dissertation writing	21 Sep – 18 Oct	Draft written and revised	—

Table 20. SDR project plan and outcomes

The project followed the phase order of the SDR plan, and each phase produced its planned deliverable: the benchmark, the pipeline, the experimental results, the APCS and its held-out evaluation. The evaluation phase was extended beyond the plan with a second, pre-registered evaluation on a fresh exam set, added because the redesigned selectors could not be evaluated fairly on the already-used test split. Two features of the plan proved their worth: running a pilot before the final benchmark, which allowed the evaluation design to be corrected cheaply, and the resumable, deduplicating pipeline, which ran the full 13,438-call benchmark in about an hour for US$2.49. With hindsight, the plan could also have scheduled human evaluation of answer quality, which would have strengthened the fidelity measurements.

Personal growth as a researcher. This project changed how I approach evidence. I began by trusting the instruments I had specified: the SDR's threshold of 0.85 looked reasonable until a simple repeat-call test showed that a third of answers to *identical* prompts failed it. Since then, my first question about any evaluation has been how much the measurement varies on its own, before asking whether a method works.

I also learned to let data overrule my intuition. The idea at the heart of my SDR was that Arabic's rich morphology would decide how well prompts compress, and the results showed that it did not. Reporting that clearly, rather than searching for an analysis that would rescue it, was uncomfortable at first, but I now see negative results as findings in their own right.

The cost result taught a related lesson. I expected compression to save money, and when the bill went up, the satisfying part was not the surprise but tracing it to a mechanism (deleted length instructions) and then fixing it. It made me look at systems as a whole rather than optimising the one number in front of me.

The fresh exam taught me discipline. Writing down my hypotheses and freezing the selectors before seeing a single result removed any temptation to adjust them afterwards, and it is the reason I trust the positive results. Finding a defect in my own morphology measure and filing an erratum taught me that admitting a mistake openly strengthens a piece of work rather than weakening it.

Technically, I gained practical skills in Arabic NLP tools, paired statistical testing and building reproducible pipelines on a budget. Just as importantly, I learned to verify my own work systematically, which is why the project ends with an independent audit of every number. If I started again, I would add human evaluation of answer quality from the outset and measure total cost from the very first pilot, so that effects such as the cost penalty are visible from the start.

6.3 Strengths and Weakness of the Project

Strengths.

- **Rigour of evaluation.** The paired design, noise-derived threshold, two independent held-out evaluations, pre-registration, Holm correction and an independent audit give the conclusions a strong evidential basis. Negative and partial results (APCS 1.0.0 not beating the best fixed strategy; morphology not predictive) are reported as clearly as the positive ones.
- **Credible, documented data.** 850 of the 1,000 benchmark prompts come from six published, licensed corpora, with per-prompt provenance, deliberate deconfounding of length and task type, and a measured rather than assumed treatment of the synthetic share.

- **Replication.** The ranking flip, method ordering, cost penalty and APCS 1.0.0 accuracy each replicated across independent prompt sets, the last within 0.2 points.
- **A finding of practical value.** The cost penalty, its mechanism and its remedy (the protected compressor) are concrete, testable and immediately useful.
- **An honest, usable artefact.** The APCS is small, explainable, tested, and provably identical to the evaluated selectors.

Weaknesses.

- **One LLM.** All final results come from one commercial model. The method ordering is likely to generalise, because it replicated from the pilot model to the final one, but the cost results depend on that model's verbosity and its 6:1 price ratio.
- **Automatic fidelity measure.** BERTScore is a proxy. QA correctness provides an independent check for one category, but no human judgement of answer quality was collected, and creative quality in particular is poorly captured by similarity to a single reference answer.
- **Noisy labels.** Best-balance labels depend on single answers that vary from call to call. With labels this noisy and near-uniform, the attainable accuracy of any selector is limited. Part of the gap between 38.5% and 100% is irreducible noise, not selector error.
- **Modest effect sizes.** APCS-v2's advantage over the best fixed strategy (+5.8 points) is significant but modest, and APCS-cost's saving (1%) is small. A selector is not a large lever on its own.
- **QA.** On QA prompts a fixed 0.5 rate beat both selectors on the exam, so the selectors' features do not yet capture what makes a QA prompt compressible.
- **Coverage.** Dialectal Arabic is represented only by a 2% probe, prompts are at most about 650 tokens long, and the creative category is synthetic.
- **Possible training-data exposure.** The source corpora are public, so the LLM may have seen some passages during training (Golchin and Surdeanu, 2024). This could affect absolute QA correctness, but it should not bias the paired comparisons between compression methods, which all use the same prompts.

- **Latency not measured end to end.** The compressor's own time was measured (Section 4.3), but API response latency was not, so no claim is made that compression makes requests faster.
- **Tokeniser-dependent costs.** Answer growth was measured in one model's billed tokens. Arabic answers cost more tokens per word than English ones, so the same length change has a larger cost effect in Arabic, and the size of the penalty would differ under another provider's tokeniser.

6.4 Academic Application and Limitations

The project contributes to research on efficient LLM use in four ways. First, it provides what is, to the author's knowledge, the **first systematic evaluation of prompt compression on Arabic**, establishing that classifier-based compression transfers and perplexity-based compression with English defaults does not. This fills the first gap identified in Section 2.3.6. Second, it shows that **output-level evaluation is necessary**: prompt-level similarity ranks methods backwards. This methodological point applies to compression research in any language, and particularly to morphologically rich ones, where surface-word retention and meaning retention diverge. Third, it provides **evidence on what drives compressibility**: task type and length matter, while morphological density, once controlled, does not. This is consistent with recent findings that morphological type itself confers no intrinsic disadvantage in language modelling (Arnett and Bergen, 2025). Fourth, it identifies the **output-length side-effect of compression**, which connects the compression literature to the literature on length bias and length control (Singhal et al., 2024; Jie et al., 2024), and shows how the cost model must change when output is priced above input.

AraPromptBench and its exam set are reusable assets. The documented protocol (noise ceiling, output-level scoring, paired tests, pre-registered fresh evaluation) is a template that can be transferred to other languages.

The academic limitations follow from the scope set in Section 1.3. The conclusions apply to MSA, to prompts of a few hundred tokens, to extractive hard-prompt compression and

to one LLM. They do not cover long-context retrieval, soft-prompt methods or dialects. The RQ2 regression explains 15% of variance, so the features studied describe only part of what makes a prompt compressible. The finding that morphology does not matter is conditional on the morphology measure used, average Farasa segments per word, and a richer measure might detect an effect this one missed.

6.5 Business Application and Limitations

For software architects building Arabic LLM applications, the findings give concrete deployment rules:

1. **Measure the whole bill.** Log billed input and output tokens per call and judge compression by total cost on real traffic, never by tokens saved; under output-heavy pricing, naive compression costs more.
2. **Enable length protection** whenever prompts carry answer-length constraints such as "in three sentences" or "no more than fifty words". Detecting them with a pattern list and keeping those sentences verbatim turned a 47.6% cost increase into a 7.9% saving.
3. **Bypass compression** for short prompts (below roughly 80–130 tokens) and for factual QA pipelines, where deleting the answer span cut correctness from 57% to 32% with LLMLingua-2 at rate 0.3; compress long summarisation and instruction prompts.
4. **Pick the selector by objective:** APCS-v2 for maximum safe token reduction, for example to fit a context window, and APCS-cost for the lowest bill at high fidelity. Neither reduces latency: compression adds about 30 ms per prompt on a GPU (Section 4.3).
5. **Use a classifier-based compressor** (LLMLingua-2), never perplexity-based compression with English scorer models on Arabic, and recalibrate the APCS when the model, price ratio or prompt mix changes.

The value of these measures depends on scale: the whole 1,000-prompt benchmark cost a few dollars, so savings matter for high-volume services. Prices and models change

quickly; the method for deciding whether compression pays is more durable than any specific number in this dissertation.

Beyond Arabic. The core findings are not specific to Arabic and are likely to apply in other settings. Any language that pays a tokeniser tax (Ahia et al., 2023; Petrov et al., 2023) faces the same incentive to compress and the same evaluation pitfalls. The output-length effect depends on pricing, not on language, so it applies to English deployments too. Routing systems that choose between models (Ding et al., 2024; Ong et al., 2025) should account for answer length in the same way. The general lesson is that, in systems where several cost components interact, optimising one component (input tokens) in isolation can worsen the total.

6.6 Recommendations / Prospects for Future Research / Work

1. **Open-weight and local models.** Repeat the benchmark on open models with different tokenisers and verbosity, including the Arabic-centric ALLaM (Bari et al., 2025) and AraLLaMA (Zhu et al., 2025) and large multilingual models such as Qwen2.5-72B, to separate compressor effects from model effects. Self-hosting also removes per-token billing and changes the economics entirely.
2. **Cost-aware sentence-level pruning.** Train a pruning model whose objective is the measured cost of a call rather than token count, using each target model's token-to-cost conversion, so that it learns which sentences control answer length.
3. **Length-instruction-aware compressors.** Fine-tune an open-source LLMLingua-2-style classifier (Pan et al., 2024) with length-controlling tokens labelled "keep", replacing the rule-based protection, and add Arabic distillation data; Arabic-centric scorers, which fragment Arabic less, could also be tested for LLMLingua.
4. **Better labels and human evaluation.** Use several answers per prompt to reduce label noise, and validate BERTScore with human judgement or a length-debiased LLM judge (Zheng et al., 2023; Dubois et al., 2024), especially for creative and summarisation tasks.

5. **Answer-aware QA features.** Features that locate the span a question targets would let the selector protect it, addressing the QA loss traced in Section 5.2.3.
6. **Dialects and longer contexts.** Extend AraPromptBench to major dialects and to long retrieval contexts, where query-aware compression (Jiang et al., 2024; Nagle et al., 2024) matters most.
7. **Release** AraPromptBench, subject to each source corpus's licence, with the APCS package and its example notebook.

6.7 Chapter Summary

Existing compression methods are partly effective on Arabic, with LLMLingua-2 the best; task type and length, not morphology, govern outcomes; and a feature-based selector beats every fixed strategy on unseen prompts by a modest margin. The project delivered AraPromptBench, a reproducible protocol and the APCS in three validated variants. Its most practical finding is that compression can raise the cost of a call by changing the answer, and that protecting answer-length instructions prevents this.

REFERENCES CITED

Abdelali, A. et al. (2016) 'Farasa: a fast and furious segmenter for Arabic', in *Proceedings of the 2016 Conference of the North American Chapter of the Association for Computational Linguistics: Demonstrations*. San Diego, CA: Association for Computational Linguistics, pp. 11–16. Available at: https://doi.org/10.18653/v1/N16-3003.

Ahia, O. et al. (2023) 'Do all languages cost the same? Tokenization in the era of commercial language models', in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. Singapore: Association for Computational Linguistics, pp. 9904–9923. Available at: https://doi.org/10.18653/v1/2023.emnlp-main.614.

Ali, M. et al. (2024) 'Tokenizer choice for LLM training: negligible or crucial?', in *Findings of the Association for Computational Linguistics: NAACL 2024*. Mexico City: Association for Computational Linguistics, pp. 3907–3924. Available at: https://doi.org/10.18653/v1/2024.findings-naacl.247.

Alyafeai, Z. et al. (2024) 'CIDAR: culturally relevant instruction dataset for Arabic', in *Findings of the Association for Computational Linguistics: ACL 2024*. Bangkok: Association for Computational Linguistics, pp. 12878–12901. Available at: https://doi.org/10.18653/v1/2024.findings-acl.764.

Antoun, W., Baly, F. and Hajj, H. (2020) 'AraBERT: transformer-based model for Arabic language understanding', in *Proceedings of the 4th Workshop on Open-Source Arabic Corpora and Processing Tools, with a Shared Task on Offensive Language Detection*. Marseille: European Language Resources Association, pp. 9–15. Available at: https://aclanthology.org/2020.osact-1.2/ (Accessed: 5 October 2026).

Arnett, C. and Bergen, B. (2025) 'Why do language models perform worse for morphologically complex languages?', in *Proceedings of the 31st International Conference on Computational Linguistics*. Abu Dhabi: Association for Computational Linguistics, pp. 6607–6623. Available at: https://aclanthology.org/2025.coling-main.441/ (Accessed: 5 October 2026).

Asgari, E. et al. (2026) 'MorphBPE: morphology-aware tokenization for efficient LLM training', in *Findings of the Association for Computational Linguistics: ACL 2026*. Association for Computational Linguistics, pp. 41610–41621. Available at: https://doi.org/10.18653/v1/2026.findings-acl.2068.

Bari, M.S. et al. (2025) 'ALLaM: large language models for Arabic and English', in *Proceedings of the Thirteenth International Conference on Learning Representations (ICLR 2025)*. Singapore. Available at: https://proceedings.iclr.cc/paper_files/paper/2025/file/54c15a3033686e7999aecd2740c5a7c4-Paper-Conference.pdf (Accessed: 5 October 2026).

Chen, L., Zaharia, M. and Zou, J. (2024) 'FrugalGPT: how to use large language models while reducing cost and improving performance', *Transactions on Machine Learning Research*. Available at: https://openreview.net/forum?id=cSimKw5p6R (Accessed: 5 October 2026).

Chevalier, A. et al. (2023) 'Adapting language models to compress contexts', in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. Singapore: Association for Computational Linguistics, pp. 3829–3846. Available at: https://doi.org/10.18653/v1/2023.emnlp-main.232.

Chiang, C.-H. and Lee, H.-y. (2023) 'Can large language models be an alternative to human evaluations?', in *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Toronto: Association for Computational Linguistics, pp. 15607–15631. Available at: https://doi.org/10.18653/v1/2023.acl-long.870.

Chuang, Y.-N. et al. (2024) 'Learning to compress prompt in natural language formats', in *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*. Mexico City: Association for Computational Linguistics, pp. 7756–7767. Available at: https://doi.org/10.18653/v1/2024.naacl-long.429.

Clark, J.H. et al. (2020) 'TyDi QA: a benchmark for information-seeking question answering in typologically diverse languages', *Transactions of the Association for Computational Linguistics*, 8, pp. 454–470. Available at: https://doi.org/10.1162/tacl_a_00317.

Deng, C. et al. (2024) 'Investigating data contamination in modern benchmarks for large language models', in *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*. Mexico City: Association for Computational Linguistics, pp. 8706–8719. Available at: https://aclanthology.org/2024.naacl-long.482/ (Accessed: 5 October 2026).

Devlin, J. et al. (2019) 'BERT: pre-training of deep bidirectional transformers for language understanding', in *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*. Minneapolis, MN: Association for Computational Linguistics, pp. 4171–4186. Available at: https://doi.org/10.18653/v1/N19-1423.

Ding, D. et al. (2024) 'Hybrid LLM: cost-efficient and quality-aware query routing', in *Proceedings of the Twelfth International Conference on Learning Representations (ICLR 2024)*. Vienna. Available at: https://proceedings.iclr.cc/paper_files/paper/2024/hash/b47d93c99fa22ac0b377578af0a1f63a-Abstract-Conference.html (Accessed: 5 October 2026).

Dubois, Y. et al. (2024) 'Length-controlled AlpacaEval: a simple way to debias automatic evaluators', in *Proceedings of the First Conference on Language Modeling (COLM 2024)*. Philadelphia, PA. Available at: https://openreview.net/pdf/76a4afefba676b543f4c4ca61529f92e828e171f.pdf (Accessed: 5 October 2026).

El-Haj, M., Kruschwitz, U. and Fox, C. (2010) 'Using Mechanical Turk to create a corpus of Arabic summaries', in *Proceedings of the Workshop on Language Resources and Human Language Technologies for Semitic Languages, LREC 2010*. Valletta: European Language Resources Association, pp. 36–39.

Elmadany, A., Nagoudi, E.M.B. and Abdul-Mageed, M. (2023) 'ORCA: a challenging benchmark for Arabic language understanding', in *Findings of the Association for Computational Linguistics: ACL 2023*. Toronto: Association for Computational Linguistics, pp. 9559–9586. Available at: https://doi.org/10.18653/v1/2023.findings-acl.609.

Ge, T. et al. (2024) 'In-context autoencoder for context compression in a large language model', in *Proceedings of the Twelfth International Conference on Learning Representations (ICLR 2024)*. Vienna. Available at: https://proceedings.iclr.cc/paper_files/paper/2024/hash/0b276510ec2d3f6613a8b60c41ff0438-Abstract-Conference.html (Accessed: 5 October 2026).

Golchin, S. and Surdeanu, M. (2024) 'Time travel in LLMs: tracing data contamination in large language models', in *Proceedings of the Twelfth International Conference on Learning Representations (ICLR 2024)*. Vienna. Available at: https://proceedings.iclr.cc/paper_files/paper/2024/file/bc39a59c49b731c51398ad6b12f301d3-Paper-Conference.pdf (Accessed: 5 October 2026).

Habash, N.Y. (2010) *Introduction to Arabic natural language processing*. San Rafael, CA: Morgan & Claypool.

Han, T. et al. (2025) 'Token-budget-aware LLM reasoning', in *Findings of the Association for Computational Linguistics: ACL 2025*. Vienna: Association for Computational Linguistics, pp. 24842–24855. Available at: https://aclanthology.org/2025.findings-acl.1274/ (Accessed: 5 October 2026).

Hasan, T. et al. (2021) 'XL-Sum: large-scale multilingual abstractive summarization for 44 languages', in *Findings of the Association for Computational Linguistics: ACL-IJCNLP 2021*. Online: Association for Computational Linguistics, pp. 4693–4703. Available at: https://doi.org/10.18653/v1/2021.findings-acl.413.

He, H. (2025) 'Defeating nondeterminism in LLM inference', *Connectionism* (Thinking Machines Lab), 10 September. Available at: https://thinkingmachines.ai/blog/defeating-nondeterminism-in-llm-inference/ (Accessed: 17 October 2026).

Hevner, A.R. et al. (2004) 'Design science in information systems research', *MIS Quarterly*, 28(1), pp. 75–105.

Holm, S. (1979) 'A simple sequentially rejective multiple test procedure', *Scandinavian Journal of Statistics*, 6(2), pp. 65–70.

Huang, H. et al. (2024a) 'AceGPT, localizing large language models in Arabic', in *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*. Mexico City: Association for Computational Linguistics, pp. 8139–8163. Available at: https://doi.org/10.18653/v1/2024.naacl-long.450.

Huang, X. et al. (2024b) 'Fewer is more: boosting math reasoning with reinforced context pruning', in *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*. Miami, FL: Association for Computational Linguistics, pp. 13674–13695. Available at: https://doi.org/10.18653/v1/2024.emnlp-main.758.

Jha, S. et al. (2024) 'Characterizing prompt compression methods for long context inference', in *Workshop on Efficient Systems for Foundation Models (ES-FoMo II), 41st International Conference on Machine Learning*. Vienna. Available at: https://openreview.net/forum?id=vs6CCDuK7l (Accessed: 5 October 2026).

Jiang, H. et al. (2023) 'LLMLingua: compressing prompts for accelerated inference of large language models', in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. Singapore: Association for Computational Linguistics, pp. 13358–13376. Available at: https://doi.org/10.18653/v1/2023.emnlp-main.825.

Jiang, H. et al. (2024) 'LongLLMLingua: accelerating and enhancing LLMs in long context scenarios via prompt compression', in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Bangkok: Association for Computational Linguistics, pp. 1658–1677. Available at: https://doi.org/10.18653/v1/2024.acl-long.91.

Jie, R. et al. (2024) 'Prompt-based length controlled generation with multiple control types', in *Findings of the Association for Computational Linguistics: ACL 2024*. Bangkok: Association for Computational Linguistics, pp. 1067–1085. Available at: https://doi.org/10.18653/v1/2024.findings-acl.63.

Jung, H. and Kim, K.-J. (2024) 'Discrete prompt compression with reinforcement learning', *IEEE Access*, 12, pp. 72578–72587. Available at: https://doi.org/10.1109/ACCESS.2024.3403426.

Khondaker, M.T.I. et al. (2023) 'GPTAraEval: a comprehensive evaluation of ChatGPT on Arabic NLP', in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. Singapore: Association for Computational Linguistics, pp. 220–247. Available at: https://doi.org/10.18653/v1/2023.emnlp-main.16.

Koto, F. et al. (2024) 'ArabicMMLU: assessing massive multitask language understanding in Arabic', in *Findings of the Association for Computational Linguistics: ACL 2024*. Bangkok: Association for Computational Linguistics, pp. 5622–5640. Available at: https://doi.org/10.18653/v1/2024.findings-acl.334.

Łajewska, W. et al. (2025) 'Understanding and improving information preservation in prompt compression for LLMs', in *Findings of the Association for Computational Linguistics: EMNLP 2025*. Suzhou: Association for Computational Linguistics, pp. 17520–17541. Available at: https://aclanthology.org/2025.findings-emnlp.949/ (Accessed: 5 October 2026).

Li, Y. et al. (2023) 'Compressing context to enhance inference efficiency of large language models', in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. Singapore: Association for Computational Linguistics, pp. 6342–6353. Available at: https://doi.org/10.18653/v1/2023.emnlp-main.391.

Li, Z. et al. (2025) 'Prompt compression for large language models: a survey', in *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*. Albuquerque, NM: Association for Computational Linguistics, pp. 7182–7195. Available at: https://doi.org/10.18653/v1/2025.naacl-long.368.

Limisiewicz, T., Balhar, J. and Mareček, D. (2023) 'Tokenization impacts multilingual language modeling: assessing vocabulary allocation and overlap across languages', in *Findings of the Association for Computational Linguistics: ACL 2023*. Toronto: Association for Computational Linguistics, pp. 5661–5681. Available at: https://doi.org/10.18653/v1/2023.findings-acl.350.

Liskavets, B. et al. (2025) 'Prompt compression with context-aware sentence encoding for fast and improved LLM inference', *Proceedings of the AAAI Conference on Artificial Intelligence*, 39(23), pp. 24595–24604. Available at: https://doi.org/10.1609/aaai.v39i23.34639.

Liu, R. et al. (2024) 'Best practices and lessons learned on synthetic data', in *Proceedings of the First Conference on Language Modeling (COLM 2024)*. Philadelphia, PA. Available at: https://openreview.net/forum?id=OJaWBhh61C (Accessed: 5 October 2026).

Long, L. et al. (2024) 'On LLMs-driven synthetic data generation, curation, and evaluation: a survey', in *Findings of the Association for Computational Linguistics: ACL 2024*. Bangkok: Association for Computational Linguistics, pp. 11065–11082. Available at: https://doi.org/10.18653/v1/2024.findings-acl.658.

McNemar, Q. (1947) 'Note on the sampling error of the difference between correlated proportions or percentages', *Psychometrika*, 12(2), pp. 153–157. Available at: https://doi.org/10.1007/BF02295996.

Mozannar, H. et al. (2019) 'Neural Arabic question answering', in *Proceedings of the Fourth Arabic Natural Language Processing Workshop*. Florence: Association for Computational Linguistics, pp. 108–118. Available at: https://doi.org/10.18653/v1/W19-4612.

Mu, J., Li, X. and Goodman, N. (2023) 'Learning to compress prompts with gist tokens', in *Advances in Neural Information Processing Systems 36*. New Orleans, LA: Curran Associates, pp. 19327–19352.

Nagle, A. et al. (2024) 'Fundamental limits of prompt compression: a rate-distortion framework for black-box language models', in *Advances in Neural Information Processing Systems 37*. Vancouver: Curran Associates, pp. 94934–94970.

Nagoudi, E.M.B. et al. (2023) 'Dolphin: a challenging and diverse benchmark for Arabic NLG', in *Findings of the Association for Computational Linguistics: EMNLP 2023*. Singapore: Association for Computational Linguistics, pp. 1404–1422. Available at: https://doi.org/10.18653/v1/2023.findings-emnlp.98.

Ong, I. et al. (2025) 'RouteLLM: learning to route LLMs from preference data', in *Proceedings of the Thirteenth International Conference on Learning Representations (ICLR 2025)*. Singapore. Available at: https://iclr.cc/virtual/2025/poster/30737 (Accessed: 5 October 2026).

Ouyang, S. et al. (2025) 'An empirical study of the non-determinism of ChatGPT in code generation', *ACM Transactions on Software Engineering and Methodology*, 34, pp. 1–28. Available at: https://doi.org/10.1145/3697010.

Pan, Z. et al. (2024) 'LLMLingua-2: data distillation for efficient and faithful task-agnostic prompt compression', in *Findings of the Association for Computational Linguistics: ACL 2024*. Bangkok: Association for Computational Linguistics, pp. 963–981. Available at: https://doi.org/10.18653/v1/2024.findings-acl.57.

Park, H.H. et al. (2021) 'Morphology matters: a multilingual language modeling analysis', *Transactions of the Association for Computational Linguistics*, 9, pp. 261–276. Available at: https://doi.org/10.1162/tacl_a_00365.

Pasha, A. et al. (2014) 'MADAMIRA: a fast, comprehensive tool for morphological analysis and disambiguation of Arabic', in *Proceedings of the Ninth International Conference on Language Resources and Evaluation (LREC'14)*. Reykjavik: European Language Resources Association, pp. 1094–1101. Available at: https://aclanthology.org/L14-1479/ (Accessed: 17 October 2026).

Pedregosa, F. et al. (2011) 'Scikit-learn: machine learning in Python', *Journal of Machine Learning Research*, 12, pp. 2825–2830.

Petrov, A. et al. (2023) 'Language model tokenizers introduce unfairness between languages', in *Advances in Neural Information Processing Systems 36*. New Orleans, LA: Curran Associates, pp. 36963–36990.

Rice, J.R. (1976) 'The algorithm selection problem', *Advances in Computers*, 15, pp. 65–118.

Sainz, O. et al. (2023) 'NLP evaluation in trouble: on the need to measure LLM data contamination for each benchmark', in *Findings of the Association for Computational Linguistics: EMNLP 2023*. Singapore: Association for Computational Linguistics, pp. 10776–10787. Available at: https://doi.org/10.18653/v1/2023.findings-emnlp.722.

Šakota, M., Peyrard, M. and West, R. (2024) 'Fly-swat or cannon? Cost-effective language model choice via meta-modeling', in *Proceedings of the 17th ACM International Conference on Web Search and Data Mining (WSDM '24)*. Merida: ACM, pp. 606–615. Available at: https://doi.org/10.1145/3616855.3635825.

Samsi, S. et al. (2023) 'From words to watts: benchmarking the energy costs of large language model inference', in *2023 IEEE High Performance Extreme Computing Conference (HPEC)*. Boston, MA: IEEE, pp. 1–9. Available at: https://doi.org/10.1109/HPEC58863.2023.10363447.

Singh, S. et al. (2024) 'Aya dataset: an open-access collection for multilingual instruction tuning', in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Bangkok: Association for Computational Linguistics, pp. 11521–11567. Available at: https://doi.org/10.18653/v1/2024.acl-long.620.

Singhal, P. et al. (2024) 'A long way to go: investigating length correlations in RLHF', in *Proceedings of the First Conference on Language Modeling (COLM 2024)*. Philadelphia, PA. Available at: https://openreview.net/forum?id=G8LaO1P0xv (Accessed: 5 October 2026).

Song, Y. et al. (2025) 'The good, the bad, and the greedy: evaluation of LLMs should not ignore non-determinism', in *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*. Albuquerque, NM: Association for Computational Linguistics, pp. 4195–4206. Available at: https://aclanthology.org/2025.naacl-long.211/ (Accessed: 5 October 2026).

Wan, Z. et al. (2024) 'Efficient large language models: a survey', *Transactions on Machine Learning Research*. Available at: https://openreview.net/forum?id=bsCCJHbO8A (Accessed: 5 October 2026).

Wilcoxon, F. (1945) 'Individual comparisons by ranking methods', *Biometrics Bulletin*, 1(6), pp. 80–83. Available at: https://doi.org/10.2307/3001968.

Xu, F., Shi, W. and Choi, E. (2024) 'RECOMP: improving retrieval-augmented LMs with context compression and selective augmentation', in *Proceedings of the Twelfth International Conference on Learning Representations (ICLR 2024)*. Vienna. Available at: https://proceedings.iclr.cc/paper_files/paper/2024/file/bda88ed2892f5e61c9a9bf215c566913-Paper-Conference.pdf (Accessed: 5 October 2026).

Zhang, T. et al. (2020) 'BERTScore: evaluating text generation with BERT', in *Proceedings of the Eighth International Conference on Learning Representations (ICLR 2020)*. Addis Ababa (virtual). Available at: https://openreview.net/forum?id=SkeHuCVFDr (Accessed: 5 October 2026).

Zheng, L. et al. (2023) 'Judging LLM-as-a-judge with MT-Bench and Chatbot Arena', in *Advances in Neural Information Processing Systems 36 (Datasets and Benchmarks Track)*. New Orleans, LA: Curran Associates, pp. 46595–46623.

Zhu, J. et al. (2025) 'Second language (Arabic) acquisition of LLMs via progressive vocabulary expansion', in *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Vienna: Association for Computational Linguistics, pp. 2025–2042. Available at: https://aclanthology.org/2025.acl-long.100/ (Accessed: 5 October 2026).

APPENDICES

Appendix A. Approved Dissertation Proposal

UNIVERSITY OF LIVERPOOL

Aug 22

Computing

CSCK700 - Computer Science Capstone Project

Project Proposal

Student's Name	Badr Abouabdou
Student's Number	30026
Student's Email Address	B.A.E.Abouabdou@liverpool.ac.uk
The MSc Programme	MSc Artificial Intelligence
Project Title	Design and Evaluation of an Arabic-Aware Prompt Compression Selector for Large Language Models
Version Number of the Proposal	3
Final Dissertation Submission Date	02/11/2026
Proposal Submission Date	28/03/2026
Name of Dissertation Advisor (DA)	Dr. Laud Charles Ochei
Class Start Date	23/01/2026
Name of Dissertation Lead (DL):	Andrea Corradini
Ethics Approval Needed?	No.

Proposal Approval *[Do not fill out this section. To be completed by the Dissertation Lead & Dissertation Advisor]*	
Proposal approved by	Dr. Laud Charles Ochei
Date of the approval (DA)	April 5, 2026
Date of approval confirmed by the Dissertation Lead	April 5, 2026

UNIVERSITY OF LIVERPOOL

Aug 22

PROJECT

The Project Aims and Objectives:

The increasing adoption of large language models (LLMs) in applications such as question answering, summarisation, and conversational systems has led to significant growth in prompt length and complexity. Long prompts increase computational cost, latency, and API usage fees, particularly in token-based pricing models. Prompt compression techniques have emerged as a promising solution for reducing token usage while maintaining semantic fidelity.
However, most existing prompt compression techniques have been developed and evaluated primarily on English text. Languages such as Arabic present additional challenges due to their rich morphology, clitic attachment, and flexible syntax, which may influence tokenisation behaviour and compression effectiveness. As a result, developers lack systematic guidance on how compression techniques perform on Arabic prompts and which technique should be selected for a given prompt type.

Aim
To investigate the effectiveness of prompt compression techniques for Arabic prompts and to design, implement, and evaluate an Arabic-Aware Prompt Compression Selector (APCS) that recommends the most suitable compression strategy for different Arabic prompt types while balancing token reduction and semantic fidelity.

Objectives
1. To critically review the literature on prompt compression, efficient prompting for large language models, Arabic NLP, tokenisation, and multilingual prompt optimisation in order to identify research gaps relevant to Arabic prompt compression.
2. To design a benchmarking framework and recommendation architecture for Arabic prompt compression, including the structure of AraPromptBench, prompt feature definitions, evaluation criteria, and the decision logic for the APCS selector.
3. To implement the system by constructing AraPromptBench, integrating selected prompt compression techniques, and developing the Arabic-Aware Prompt Compression Selector (APCS) as a Python-based tool.
4. To evaluate the performance of the benchmark and APCS tool using token compression ratio, semantic fidelity, recommendation accuracy, and estimated cost savings across multiple Arabic prompt categories.
5. To provide recommendations for the practical use of Arabic prompt compression strategies in application contexts such as Arabic question answering, summarisation, educational systems, and conversational assistants.

The project will address the following research questions:
1. How do existing prompt compression techniques perform on Arabic prompts across different prompt types (e.g., instruction, summarisation, creative generation, and question answering)?
2. How do prompt characteristics such as length, structure, and linguistic complexity affect compression efficiency and output fidelity?
3. Can a feature-based recommendation system accurately identify the most suitable prompt compression strategy for a given Arabic prompt?

To answer these questions, the project will develop a benchmark dataset named AraPromptBench, designed to evaluate prompt compression techniques specifically on Arabic prompts. The benchmark will evaluate several state-of-the-art compression methods using metrics such as token compression ratio and semantic similarity.

Based on the results of the benchmark analysis, the project will produce an IT artefact called the Arabic-Aware Prompt Compression Selector (APCS). The APCS will be a Python-based tool that analyses the characteristics of a prompt and recommends the compression strategy that best balances token savings and semantic fidelity.

Existing solutions do not currently provide such guidance for Arabic prompts. Most available compression tools are designed as general-purpose frameworks without language-specific optimisation. Therefore, a systematic benchmark and recommendation framework tailored to Arabic prompts represents a novel contribution to the field.

The scope of the project is appropriate for a master's dissertation because it combines empirical experimentation, data analysis, and software artefact development within a clearly defined research problem. The findings of the study could contribute to the growing body of research on efficient prompting and multilingual NLP.

In the table below, please state your dissertation question(s); the research methods you will use to guide the development of your IT artefact; the kind of IT artefact you will produce; and how you will evaluate the IT artefact in the light of the dissertation question(s)

Step	Short Description
Dissertation Question(s)	1. How effective are existing prompt compression techniques on Arabic prompts? 2. How do prompt features influence compression performance? 3. Can a recommendation system accurately select the optimal compression strategy?
Research Methods	This project adopts a design science approach for the development and evaluation of the APCS artefact, supported by experimental evaluation, empirical analysis, and feature-based analysis. Experimental evaluation is conducted using a fixed LLM under consistent settings to ensure fair comparison. Empirical analysis is performed on AraPromptBench (approx. 800–1200 prompts), split into development and held-out test sets (e.g., 80/20). Selected compression methods (e.g., LLMLingua, LongLLMLingua, and a baseline) are compared. Feature-based analysis operationalises prompt characteristics such as length, token count, structural complexity, and morphological indicators. Semantic fidelity is measured using BERTScore, alongside Token Compression Ratio (TCR), cost, and task accuracy.

IT Artefact	The APCS is a Python-based recommendation system that takes an Arabic prompt as input and outputs the most suitable compression strategy. It consists of: (1) input layer (prompt text), (2) feature extraction module (linguistic and structural features), (3) decision engine (rule-/feature-based logic derived from analysis), and (4) output module (recommended method(s), optionally ranked with confidence score). The system supports command-line and notebook usage, with potential extensibility to API integration.
Evaluation	The artefact is evaluated based on its ability to recommend the compression method that achieves the best balance between token reduction and semantic fidelity. Metrics include Token Compression Ratio (TCR), BERTScore, recommendation accuracy, and estimated cost savings, with comparison against baseline selection strategies.

Project Outline

This project adopts an experimental and design-oriented approach to investigate prompt compression for Arabic and develop the APCS artefact.

A benchmark dataset, AraPromptBench, will be constructed using a combination of publicly available Arabic datasets, translated prompts adapted from English sources, and manually designed prompts. Prompts will be categorised into task types including instruction-following, summarisation, question answering, and creative generation, reflecting common LLM use cases and varying sensitivity to compression. The dataset will primarily use Modern Standard Arabic, with limited inclusion of dialectal Arabic to ensure linguistic diversity while maintaining consistency.

Selected compression techniques, including LLMLingua, LongLLMLingua, and baseline approaches, will be integrated into a unified experimental pipeline. Each prompt will be compressed and processed by a fixed LLM under consistent settings to ensure fair comparison. Evaluation will focus on both token reduction and preservation of semantic meaning, measured by comparing outputs from compressed and original prompts.

The dataset will be divided into a development set and a held-out test set. The development set will be used for benchmarking and deriving decision rules, while the test set will be reserved for evaluating generalisation.

The results will be analysed to identify relationships between prompt characteristics (e.g., length, structure, and morphological complexity) and compression performance. These insights will inform the design of the APCS, a feature-based system that recommends suitable compression strategies. The APCS will be evaluated on the held-out test set against baseline selection strategies.

Literature Survey / Resources' List:

The following literature establishes the academic context of the proposed research by examining prompt compression, efficient LLM usage, and multilingual NLP, with particular relevance to Arabic.

1 Prompt Compression and Optimisation

Recent work on prompt compression demonstrates that substantial reductions in token usage can be achieved while maintaining task performance. Methods such as LLMLingua and LongLLMLingua selectively remove low-importance tokens using language model-based scoring, achieving significant compression gains while preserving accuracy (Jiang et al., 2023; Jiang et al., 2024). In parallel, research on efficient LLM usage highlights that adaptive strategies, such as dynamic model and prompt selection, can reduce computational cost while maintaining performance (Chen, Zaharia and Zou, 2023).

However, these approaches are primarily developed and evaluated on English datasets and assume relatively stable tokenisation behaviour, limiting their applicability to morphologically rich languages.

2 Multilingual NLP and Tokenisation

Studies in multilingual NLP show that linguistic properties such as morphology and tokenisation significantly influence model behaviour across languages. Blevins et al. (2021) demonstrate that morphological complexity directly affects language modelling performance, while Limisiewicz, Balhar and Mareček (2023) show that tokenisation strategies and vocabulary allocation can lead to uneven performance across languages.

Further work by Asgari, El Kheir and Javaheri (2025) introduces morphology-aware tokenisation methods, showing that incorporating linguistic structure improves representation and model efficiency for morphologically complex languages. These findings suggest that techniques developed for English may not generalise effectively without adaptation.

3 Arabic Natural Language Processing

Arabic presents additional challenges due to its rich morphology, clitic attachment, and flexible syntax (Habash, 2010). Prior work demonstrates that language-specific preprocessing and segmentation are critical for effective NLP performance. For example, Abdelali et al. (2016) show that accurate segmentation significantly improves downstream task performance, while Antoun, Baly and Hajj (2020) demonstrate that adapting preprocessing and training strategies to Arabic leads to improved model performance.

These studies highlight that tokenisation behaviour and linguistic structure play a central role in how information is represented and processed in Arabic.

Collectively, the literature shows that prompt compression and optimisation techniques are effective in reducing token usage, while multilingual NLP research demonstrates that linguistic characteristics such as morphology and tokenisation significantly influence model performance across languages.

However, these research areas remain largely disconnected. Existing prompt compression methods are evaluated primarily on English data and do not account for the linguistic complexity of Arabic, which has been shown to require specialised preprocessing and modelling approaches.

Furthermore, research on prompt optimisation suggests that adaptive strategies outperform fixed approaches, yet no existing work applies this principle to the selection of prompt compression techniques based on input characteristics.

There is therefore a clear gap in the literature:

- lack of Arabic-specific evaluation of prompt compression techniques

- absence of systematic benchmarking across prompt types
- no feature-based recommendation framework for compression strategy selection

This gap motivates the development of AraPromptBench and the Arabic-Aware Prompt Compression Selector (APCS), which aim to provide both empirical evidence and practical guidance for efficient prompting in Arabic.

Scholarly Contributions of the Project

This project contributes to the fields of natural language processing and efficient LLM usage by addressing the lack of Arabic-specific evaluation and optimisation of prompt compression techniques.

The key contributions are:

- **Arabic-Specific Benchmark Design**
  Development of AraPromptBench, a structured benchmark for evaluating prompt compression techniques across diverse Arabic prompt types, providing a reproducible evaluation framework.
- **Evidence-Driven Empirical Analysis**
  Systematic comparison of compression techniques on Arabic prompts, generating new insights into how linguistic features such as morphology, tokenisation, and prompt structure affect compression performance and semantic fidelity.
- **Feature-Based Recommendation Framework (APCS)**
  Design and implementation of the Arabic-Aware Prompt Compression Selector (APCS), which links prompt characteristics to empirically derived performance outcomes to recommend suitable compression strategies.
- **Integrated Framework for Prompt Optimisation**
  A unified approach combining benchmarking, empirical analysis, and recommendation logic, moving beyond single-method assumptions toward adaptive, context-aware compression.

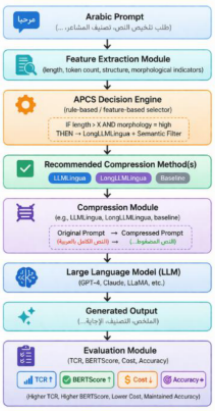


Figure 1. Architecture of the Arabic-Aware Prompt Compression Selector (APCS)

This work advances current research by extending prompt compression beyond English, demonstrating the importance of linguistic characteristics in compression effectiveness, and introducing a practical, data-driven approach for selecting compression strategies.

Description of the Deliverables:

The expected deliverables of the project include:

1. AraPromptBench Dataset
   A curated dataset of Arabic prompts categorised by prompt type.
2. Experimental Benchmark Framework
   A Python-based evaluation pipeline for testing prompt compression techniques.
3. APCS Tool
   The Arabic-Aware Prompt Compression Selector implemented as an open-source Python package.
4. Experimental Results and Analysis
   A detailed analysis of prompt compression performance across different prompt categories.
5. Dissertation Report
   A comprehensive written report documenting the methodology, experiments, findings, and conclusions of the research.
6. Source code repository (GitHub)
7. APCS documentation/manual
8. Experiment logs and configuration files
9. Example usage notebook

Evaluation Criteria:

The evaluation will assess both the effectiveness of compression techniques and the performance of the APCS recommendation system.

Definition of "best balance"
A compression method will be considered optimal if it maximises token reduction while maintaining semantic fidelity above a defined threshold (e.g., BERTScore ≥ 0.85).

Metrics

- Token Compression Ratio (TCR)
- Semantic Fidelity (BERTScore; supplemented with qualitative evaluation where needed)
- Recommendation Accuracy (percentage of cases where APCS selects the Pareto-optimal method)
- Estimated Cost Savings (based on token usage and API pricing models)

Baselines

- No compression
- Fixed compression method (e.g., always LLMLingua)
- Random selection

Additional analysis

- Per-category performance analysis

- Confusion matrix for APCS recommendations
- Statistical comparison of compression methods where applicable

Resource Plan:

The project will require the following resources:

Hardware
• Personal computer capable of running Python-based NLP tools

Software
• Python programming environment
• NLP libraries such as Transformers and NLTK
• Prompt compression libraries such as LLMLingua
• Access to large language model APIs (e.g., GPT-based or equivalent models)
• Contingency: use smaller models if cost is high
• Optional: open-source Arabic-capable models

Data
• Publicly available prompt datasets
• Custom-curated Arabic prompts

Financial Costs
• Potential API usage costs for LLM inference during experiments

All required software resources are publicly available and can be accessed remotely. The computational requirements of the project are modest and can be supported using a personal computer together with cloud-based LLM APIs, GPU will mostly not be required as we will opt to use API-based testing primarily.

Project Plan and Timing

Phase	Weeks	Description
Project Initiation	9–10	Finalise topic, confirm supervisor expectations, and refine research questions.
Literature Review	11–14	Review prompt compression, efficient LLM usage, Arabic NLP, and multilingual tokenisation; identify research gaps and refine methodology.
Design & Specification	15–17	Define experimental framework, dataset structure, prompt features, APCS architecture, and evaluation strategy.
Dataset Development (AraPromptBench)	18–21	Collect, translate, and categorise Arabic prompts; ensure diversity across task types and language variations.
Framework Implementation	22–25	Implement pipeline, integrate compression methods, and configure LLM with consistent settings.
Experimental Evaluation	26–29	Run benchmark experiments and collect metrics (TCR, semantic fidelity).

Phase	Weeks	Description
Analysis & APCS Development	30–35	Analyse results, derive decision rules, and implement APCS (feature extraction and recommendation logic).
Artefact Evaluation	36–37	Evaluate APCS on held-out test set and compare with baseline strategies.
Dissertation Writing	38–40	Write and refine dissertation (methods, results, discussion, conclusions).
Buffer / Contingency	40	Allow time for delays, additional experiments, and final revisions
Project Initiation	9–10	Finalise topic, confirm supervisor expectations, and refine research questions.

Table 1. Plan and Tming

Risk Assessment:

Several potential risks may arise during the project.

Risk	Likelihood	Impact	Mitigation
Resource and project constraints (API cost, time management)	Medium	High	Use cost-efficient models, limit initial experiments, and utilise buffer time.
Integration and tool limitations (compatibility, limited Arabic support)	Medium	High	Use well-documented tools, modular design, and replace unsupported methods if needed.
Dataset limitations and bias (limited diversity or over-reliance on manual prompts)	Medium	High	Combine multiple sources and ensure coverage across prompt types and linguistic variations.
Evaluation limitations (semantic metrics and subjectivity in creative tasks)	High	High	Use multiple metrics and include limited qualitative evaluation where necessary.
Model generalisation risk (APCS overfitting to benchmark data)	Medium	High	Use held-out test set and validate across diverse prompt categories.

Table 2. Project Risk Assessment

Quality Assurance:

Quality assurance measures will include:

- Regular meetings and defined checkpoints (e.g., every 2–3 weeks)
- Reproducible experiments with logging and configuration tracking
- Version control and documentation using GitHub
- Validation of results through testing and dataset quality checks

References:

Abdelali, A., Darwish, K., Durrani, N. and Mubarak, H., 2016. Farasa: A fast and furious segmenter for Arabic. In: *Proceedings of the 2016 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL-HLT)*, San Diego, USA, pp. 11–16. Available at: https://aclanthology.org/N16-3003/ (Accessed: 20 March 2026).

Asgari, E., El Kheir, Y. and Javaheri, M., 2025. MorphBPE: A morpho-aware tokenizer bridging linguistic complexity for efficient LLM training across morphologies. *arXiv preprint arXiv:2502.00894*. Available at: https://arxiv.org/abs/2502.00894 (Accessed: 16 February 2026).

Antoun, W., Baly, F. and Hajj, H. (2020) 'AraBERT: Transformer-based model for Arabic language understanding', *Proceedings of the 4th Workshop on Open-Source Arabic Corpora and Processing Tools, with a Shared Task on Offensive Language Detection*, Marseille, France, pp. 9–15. Association for Computational Linguistics. Available at: https://aclanthology.org/2020.osact-1.2/ (Accessed: 20 March 2026).

Blevins, T., Levy, O., Zettlemoyer, L. and Hermann, K., 2021. Morphology matters: A multilingual language modeling analysis. *Transactions of the Association for Computational Linguistics*, 9, pp. 494–510. Available at: https://direct.mit.edu/tacl/article/doi/10.1162/tacl_a_00365 (Accessed: 15 February 2026).

Chen, L., Zaharia, M. and Zou, J., 2023. FrugalGPT: How to use large language models while reducing cost and improving performance. *arXiv preprint arXiv:2305.05176*. Available at: https://arxiv.org/abs/2305.05176 (Accessed: 20 March 2026).

Habash, N., 2010. *Introduction to Arabic Natural Language Processing*. San Rafael: Morgan & Claypool Publishers.

Jiang, H., Wu, Q., Lin, C.-Y., Yang, Y. and Qiu, L. (2023) 'LLMLingua: Compressing prompts for accelerated inference of large language models', *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, Singapore, pp. 13358–13376. Association for Computational Linguistics. Available at: https://aclanthology.org/2023.emnlp-main.825/ (Accessed: 13 February 2026).

Jiang, H., Wu, Q., Luo, X., Li, D., Lin, C.-Y., Yang, Y. and Qiu, L. (2024) 'LongLLMLingua: Accelerating and enhancing LLMs in long context scenarios via prompt compression', *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL), Volume 1: Long Papers*, Bangkok, Thailand, pp. 1658–1677. Association for Computational Linguistics. Available at: https://aclanthology.org/2024.acl-long.91/ (Accessed: 13 February 2026).

Limisiewicz, T., Balhar, J. and Mareček, D. (2023) 'Tokenization impacts multilingual language modeling: Assessing vocabulary allocation and overlap across languages', *Findings of the Association for Computational Linguistics: ACL 2023*, Toronto, Canada, pp. 5661–5681. Association for Computational Linguistics. Available at: https://aclanthology.org/2023.findings-acl.350/ (Accessed: 16 February 2026).

Appendix B. SPECIFICATION & DESIGN REPORT

Specification and Design Report

Student's Name: Badr Abouabdou

Student's Number: 30026

Student's Email Address: B.A.E.Abouabdou@liverpool.ac.uk

Project Title: Design and Evaluation of an Arabic-Aware Prompt Compression Selector for Large Language Models

Name of Dissertation Advisor (DA): Laud Charles Ochei

Name of Dissertation Instructor (DI): Andrea Corradini

1 The Specification

1.1 Project Context, Aims, Objectives, Ethical Considerations and Outcomes

1.1.1 The Context of the Research

The deployment of large language models (LLMs) in question answering, summarisation, and conversational systems has produced rapid growth in the volume of prompts processed at inference time. Because most commercial LLM APIs are priced and rate-limited per token, prompt length translates directly into latency, cost, and carbon footprint. Prompt compression — the family of techniques that reduce a prompt's token count while preserving its task-relevant meaning — has emerged as a practical mitigation, with state-of-the-art methods such as LLMLingua and LLMLingua-2 demonstrating 2x to 5x compression on English prompts with limited loss of task performance.

The literature, however, has developed almost exclusively on English data and on tokenizers optimised for English. Arabic, by contrast, is a morphologically rich language whose surface forms encode person, number, gender, definiteness, mood, and clitic attachment within single words. When an Arabic prompt is processed by an English-trained tokenizer, individual words frequently fragment into many sub-word tokens. Two consequences follow: Arabic prompts incur disproportionately higher token counts (and therefore higher costs) than English prompts of equivalent informational content; and sub-word fragmentation may interact unpredictably with compression heuristics that assume each token carries roughly equal information. Whether existing English-trained compressors transfer cleanly to Arabic is therefore an open empirical question — and the question this project addresses.

1.1.2 The Problem Statement

Practitioners deploying LLMs in Arabic-language applications face two unresolved problems. First, they have no empirical evidence that compression methods developed for English deliver comparable token-savings and semantic-fidelity trade-offs on

Arabic. Second, even if compression is broadly effective, no published guidance exists on selecting between methods (e.g. LLMLingua versus LLMLingua-2) for a given Arabic prompt, and the prompt characteristics that drive this choice — length, structural complexity, morphological density, task type — have not been systematically related to compression performance for Arabic. The problem is therefore twofold: a measurement problem and a selection problem. This project addresses both, constructing an Arabic-specific benchmark to answer the first and using the benchmark's findings to design a recommendation system that answers the second.

1.1.3 The Dissertation Questions

The project is guided by three research questions, each operationalised through specific empirical or design activities:

RQ	Question	How it is addressed
RQ1	How do existing prompt compression techniques perform on Arabic prompts across different prompt types?	Empirical benchmarking of LLMLingua, LLMLingua-2, and a random-deletion baseline on AraPromptBench across four task categories.
RQ2	How do prompt characteristics (length, structure, morphology) affect compression efficiency and output fidelity?	Feature extraction over each prompt and correlation analysis between features and compression outcomes (TCR, BERTScore F1).
RQ3	Can a feature-based recommendation system accurately identify the most suitable prompt compression strategy for a given Arabic prompt?	Design and evaluation of APCS — a rule-based selector operating on extracted prompt features — on a held-out test set, compared against fixed-method and random baselines.

Table 1. RQs and how they're addressed

1.1.4 Ethical Implications

The project does not involve human participants, surveys, interviews, personal data, or clinical material. AraPromptBench is constructed from publicly available datasets distributed under permissive licences and from prompts written by the author. No personally identifiable information is collected, processed, or stored. LLM API calls send only the constructed prompts and contain no end-user data. The project therefore does not require formal ethical approval, consistent with the original proposal's ethics determination. Two indirect ethical considerations are nonetheless explicitly addressed: dataset licences are tracked at source level and respected in distribution; and findings are reported with explicit acknowledgement of the linguistic varieties covered (predominantly Modern Standard Arabic) to avoid overgeneralising to all Arabic speakers and contexts.

1.1.5 Anticipated Outcomes and the IT Artefact

The project produces three complementary outcomes: AraPromptBench (a curated benchmark of approximately one thousand Arabic prompts, balanced across four task categories and split 80/20 dev/test); an empirical analysis of LLMLingua, LLMLingua-

2, and a random-deletion baseline on Arabic prompts; and the APCS — a Python package that, given an Arabic prompt, recommends a compression method based on rules derived from the benchmark. The IT artefact (the APCS) is a software system, evaluated empirically on the held-out test set against four baselines: no compression, always-LLMLingua, always-LLMLingua-2, and random selection. The primary metric is recommendation accuracy — the percentage of test prompts on which APCS selects a Pareto-optimal method (achieving the best balance of token reduction and semantic fidelity available among the candidates). Secondary metrics include estimated cost savings, average output similarity, and per-category recommendation consistency.

1.1.6 Major Modifications Since the Proposal

The proposal as approved by the Dissertation Advisor has been refined in three respects, none of which alters the project's aims or research questions. First, the LLM has been locked to gpt-4o-mini at temperature 0.0 for reproducibility and cost control (the proposal had indicated openness between OpenAI-family and open Arabic-capable models). Second, the compression methods are now LLMLingua, LLMLingua-2, and a random-deletion baseline, an explicit pairing that permits a methodologically meaningful comparison between perplexity-based and classifier-based scoring within the dissertation's findings. Third, the APCS decision engine is now specified as a rule-based selector operating on extracted prompt features; this narrows scope, increases explainability, and aligns with the project's feasibility envelope. A trained-classifier extension over the same feature set is identified as a desirable but not essential deliverable, and would enable a direct comparison between hand-written and learned decision rules.

1.2 Literature Survey:

This section presents ten annotated bibliographic entries organised across four threads: prompt compression methods, efficient LLM usage, multilingual tokenisation and morphology, and Arabic NLP. Each annotation summarises the source's contribution and indicates how it informs the present project. References follow Harvard style; a consolidated reference list appears in the Appendix.

1.2.1 Prompt Compression Methods

Jiang, H., Wu, Q., Lin, C.-Y., Yang, Y. and Qiu, L. (2023) 'LLMLingua: Compressing prompts for accelerated inference of large language models', in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. Singapore: Association for Computational Linguistics, pp. 13358–13376.

LLMLingua introduces a coarse-to-fine prompt compression algorithm that uses a small language model to score the importance of tokens by their contribution to next-token perplexity, then iteratively removes low-scoring tokens until a target compression ratio is achieved. The authors demonstrate compression ratios up to 20x on English prompts across four benchmarks spanning different scenarios — GSM8K (math reasoning), BBH (general reasoning), ShareGPT (conversational), and Arxiv-March23 (academic text) — with minimal degradation of downstream task performance. The paper establishes the methodological foundation on which the present project builds: it defines the token-importance objective, the budget controller, and the iterative compression loop reused with Arabic prompts in this dissertation. Crucially, all of LLMLingua's experiments are on English data using English-trained scorer models

(GPT2-small or LLaMA-7B), and the paper does not address whether the same token-importance heuristics transfer to morphologically rich languages where surface tokens fragment unpredictably. This unaddressed transferability question is the empirical gap that motivates RQ1 of the present project.

Jiang, H., Wu, Q., Luo, X., Li, D., Lin, C.-Y., Yang, Y. and Qiu, L. (2024) 'LongLLMLingua: Accelerating and enhancing LLMs in long context scenarios via prompt compression', in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Vol. 1: Long Papers)*. Bangkok, Thailand: Association for Computational Linguistics, pp. 1658–1677.

LongLLMLingua extends the LLMLingua framework to long-context settings such as retrieval-augmented generation by introducing question-aware compression and document reordering. Tokens are scored not only by intrinsic perplexity but also by relevance to the user's query. The authors report substantial improvements on long-context benchmarks: a 21.4% performance boost on NaturalQuestions with around 4x fewer tokens on GPT-3.5-Turbo, a 94% cost reduction on LooGLE, and end-to-end latency acceleration of 1.4x–2.6x when compressing 10k-token prompts at 2x–6x ratios. For the present project, LongLLMLingua is relevant chiefly as a methodological precedent for context-sensitive compression: it demonstrates that compression decisions benefit from prompt structure awareness. While the present project does not target long-context retrieval specifically, LongLLMLingua's feature-aware approach informs the design of the APCS decision rules, which similarly use prompt features (length, structural complexity) to drive method selection. The paper's English-only evaluation again leaves Arabic transferability unresolved.

Pan, Z., Wu, Q., Jiang, H., Xia, M., Luo, X., Zhang, J., Lin, Q., Rühle, V., Yang, Y., Lin, C.-Y., Awadallah, H.A., Zhang, D. and Qiu, L. (2024) 'LLMLingua-2: Data distillation for efficient and faithful task-agnostic prompt compression', in *Findings of the Association for Computational Linguistics: ACL 2024*. Bangkok, Thailand: Association for Computational Linguistics, pp. 963–981.

LLMLingua-2 reformulates prompt compression as a token classification task — each token is labelled keep or discard — using a BERT-level Transformer encoder trained on data distilled from GPT-4. Compared to perplexity-based scoring (LLMLingua), the classifier approach is 3x–6x faster, more deterministic, and shown to be more faithful to the original prompt across diverse tasks. The authors justify the reformulation on two grounds: information entropy from a causal LM is unidirectional and may miss essential information, and it is not aligned with the compression objective. This paper is methodologically central to the present project: it provides the second compression method benchmarked here, and the contrast between perplexity-based (LLMLingua) and classifier-based (LLMLingua-2) scoring permits a substantive comparison within the dissertation's findings. As with prior work, the evaluation is on English prompts (MeetingBank, LongBench, ZeroScrolls, GSM8K, BigBench-Hard). The present project tests whether the classifier's English-trained labels transfer to Arabic prompts whose token distributions differ markedly from English.

1.2.2 Efficient LLM Usage

Chen, L., Zaharia, M. and Zou, J. (2024) 'FrugalGPT: How to use large language models while reducing cost and improving performance', *Transactions on Machine Learning Research*. Available at: https://openreview.net/forum?id=cSimKw5p6R.

FrugalGPT proposes adaptive strategies for using LLMs cost-effectively, organised under three categories: prompt adaptation (identifying shorter prompts that achieve the same result), LLM approximation (creating cheaper LLMs that approximate a powerful one on specific tasks), and LLM cascade (adaptively triaging queries to different LLMs

in a learned cascade). The authors demonstrate that their cascade implementation matches GPT-4 performance with up to 98% cost reduction, or improves accuracy by 4% at matched cost. The paper establishes a framing directly relevant to APCS: that adaptive, query-aware strategies outperform fixed strategies on the cost-performance frontier. The present project applies the same principle within a narrower domain — selecting among compression methods for Arabic prompts — and uses prompt features as the basis for adaptation. FrugalGPT thereby provides theoretical support for APCS's core design hypothesis: that no single compression method dominates across all Arabic prompts, and that selecting per prompt yields better outcomes than committing to one method globally. The paper also provides a methodological template for the cost-savings analysis reported in the present project's evaluation.

1.2.3 Multilingual Tokenisation and Morphology

Limisiewicz, T., Balhar, J. and Mareček, D. (2023) 'Tokenization impacts multilingual language modeling: Assessing vocabulary allocation and overlap across languages', in *Findings of the Association for Computational Linguistics: ACL 2023*. Toronto, Canada: Association for Computational Linguistics, pp. 5661–5681.

This paper proposes two new criteria for evaluating multilingual tokenizers: vocabulary allocation (measured by average token rank in in-language distributions) and vocabulary overlap (measured by Jensen-Shannon divergence between in-language distributions). The authors show that overlap is detrimental to certain downstream tasks (POS tagging, dependency labelling) but beneficial for others (NER, cross-lingual retrieval, NLI), and that uneven vocabulary allocation across languages correlates with downstream performance gaps. For the present project, the paper supplies the linguistic basis for why language-specific evaluation of tokeniser-dependent methods is necessary: tokenisers do not allocate vocabulary uniformly across languages, and the resulting per-language tokenisation behaviour matters for downstream performance. This grounds the project's hypothesis that compression methods evaluated on English may not transfer cleanly to Arabic. The paper also informs the APCS feature extractor, where the fragmentation_ratio feature operationalises a similar idea of tokeniser-driven cost per orthographic word, providing a measurable input to the rule-based decision engine.

Petrov, A., La Malfa, E., Torr, P. and Bibi, A. (2023) 'Language model tokenizers introduce unfairness between languages', in *Advances in Neural Information Processing Systems 36*. New Orleans, USA: Curran Associates, pp. 36963–36990.

Petrov and colleagues quantify a fairness problem in commercial LLM tokenisers: the same semantic content costs substantially more tokens in some languages than others, with the same text producing token lengths that differ by up to 15x across language pairs. The paper analyses both commercial subword tokenisers and character/byte-level alternatives, finding that even byte-level encoding produces over 4x differences between some language pairs. Three concrete fairness implications are documented: cost (some users pay 2.5x or more for the same service), latency, and effective context length. The paper directly motivates the practical relevance of prompt compression for non-English languages: speakers of under-allocated languages pay disproportionately per unit of meaning. For the present project, the paper supplies the rationale for the project's focus on Arabic (one of the languages documented as disadvantaged) and provides the methodological blueprint for measuring tokenisation efficiency reused in the APCS feature extractor. It also supports the project's comparative design: rather than assuming Arabic behaves like English under compression, language-specific evaluation is established as necessary before extending compression claims across languages.

1.2.4 Arabic Natural Language Processing

Habash, N.Y. (2010) *Introduction to Arabic Natural Language Processing*. San Rafael, CA: Morgan & Claypool Publishers.

Habash's monograph provides the canonical introduction to Arabic NLP, covering Arabic script, orthography, morphology, syntax, and the major preprocessing decisions that distinguish Arabic from Indo-European languages. It anchors the present project's linguistic claims (clitic attachment, root-and-pattern morphology, diacritisation) and provides the conceptual vocabulary used throughout the dissertation. The monograph's discussion of tokenisation and segmentation is directly drawn upon when defining the APCS feature set, and its treatment of varieties of Arabic informs the project's decision to focus on Modern Standard Arabic with limited dialectal representation. While the monograph predates the LLM era, its foundations remain valid; the present project frequently cites Habash for definitions, conventions, and design rationales rather than for empirical findings. The work also offers practical guidance on Arabic-specific normalisation (alef forms, taa marbuta, diacritic removal) that the present project adopts as preprocessing prior to feature extraction and tokenisation.

Antoun, W., Baly, F. and Hajj, H. (2020) 'AraBERT: Transformer-based model for Arabic language understanding', in *Proceedings of the 4th Workshop on Open-Source Arabic Corpora and Processing Tools, with a Shared Task on Offensive Language Detection*. Marseille, France: European Language Resources Association, pp. 9–15.

AraBERT is the most widely adopted Arabic-specific BERT model and is reported to substantially outperform multilingual BERT on Arabic understanding tasks. The paper documents the preprocessing decisions (Farasa segmentation, diacritic handling, normalisation) that underpin AraBERT's gains. For the present project, AraBERT is doubly relevant. First, it serves as the secondary scoring model for BERTScore in the evaluation protocol, providing an Arabic-specific check on the multilingual BERT primary scorer. Second, the preprocessing pipeline described in the paper informs the cleaning and normalisation steps applied to AraPromptBench. The paper's demonstration that Arabic-specific models materially outperform multilingual ones on Arabic strengthens the rationale for treating Arabic prompt compression as a separate research question rather than a footnote within multilingual NLP. The two-scorer design in the present project's evaluation is motivated directly by this paper: claims that depend solely on a multilingual scorer would be less defensible without the AraBERT cross-check.

Abdelali, A., Darwish, K., Durrani, N. and Mubarak, H. (2016) 'Farasa: A fast and furious segmenter for Arabic', in *Proceedings of the 2016 Conference of the North American Chapter of the Association for Computational Linguistics: Demonstrations*. San Diego, California: Association for Computational Linguistics, pp. 11–16.

Farasa is a high-throughput Arabic segmenter that splits orthographic words into their constituent clitics, prefixes, stems, and suffixes. For the present project, Farasa is an operational tool: the APCS feature extractor uses it (or a comparable lightweight segmenter) to compute morphological-density features such as average sub-word count per orthographic word. Beyond its operational role, Farasa is methodologically interesting because its segmentation is independent of any neural tokeniser, providing a linguistically grounded reference against which the cl100k_base tokeniser's behaviour on Arabic can be compared. The paper therefore supplies both a concrete preprocessing component and a methodological reference point for the empirical comparison of tokenisers reported in the dissertation. Its emphasis on speed (the eponymous "fast and furious") aligns with the present project's requirement that feature

extraction remain lightweight enough to run at inference time without imposing meaningful latency on the compression pipeline.

1.2.5 Research Methodology

Hevner, A.R., March, S.T., Park, J. and Ram, S. (2004) 'Design science in information systems research', *MIS Quarterly*, 28(1), pp. 75–105.

Hevner et al.'s framework for design science in information systems is the methodological reference under which the present project operates. The framework emphasises that design science research must produce a purposeful artefact, demonstrate its utility through rigorous evaluation, and contribute knowledge both about the artefact and about the problem it addresses. The present project follows this prescription explicitly: APCS is the artefact, the empirical benchmark provides the evaluation, and the dissertation reports both the artefact's performance and the empirical findings about Arabic prompt compression. The seven guidelines proposed by Hevner et al. (problem relevance, design as artefact, design evaluation, research contributions, research rigour, design as a search process, communication) are used in Section 4 of the present report to structure the evaluation strategy and to justify methodological choices. The framework's explicit separation of artefact contribution from empirical contribution helps the present project articulate both its design contribution (APCS) and its empirical contribution (AraPromptBench findings) clearly.

1.3 Conduct of the Project:

1.3.1 Research Methods

The project follows the design science research methodology of Hevner et al. (2004), which is appropriate because it both develops a purposeful IT artefact (APCS) and contributes empirical knowledge about a previously unstudied phenomenon (compression on Arabic prompts). Within the design-science frame, three complementary methods are employed.

- Empirical benchmarking. A controlled, repeatable pipeline applies each compression method to each prompt, queries the LLM under fixed settings (gpt-4o-mini, temperature 0.0, max_tokens 512), and records token counts, latencies, costs, and outputs. The same prompts are processed by every method, allowing paired comparisons.
- Feature-based analysis. Each prompt is associated with a vector of computable features (length, structural complexity, morphological density, task category, tokeniser fragmentation ratio). Relationships between features and compression outcomes are analysed using correlation and per-category breakdowns; the resulting evidence informs the rules in the APCS decision engine.
- Artefact construction and evaluation. APCS is implemented as a Python package and evaluated on the held-out test set against four baselines (no compression, always-LLMLingua, always-LLMLingua-2, random selection). Statistical comparisons use Wilcoxon signed-rank tests for paired metrics and McNemar's test for recommendation accuracy.

1.3.2 Data and Sources

AraPromptBench will be constructed from a mix of public Arabic datasets and manually curated prompts. The composition is shown in Table 2 below. All public sources are openly licensed and used in accordance with their terms; the manually curated portion is original work released under CC BY 4.0. Each prompt is annotated with: task category, original source and licence, length in characters and tokens, computed features, and a dev/test flag. Quality control includes deduplication, length-balancing within categories, and manual inspection of a 5% sample. Dialectal Arabic is intentionally a minority of the dataset; findings are therefore reported as primarily applicable to Modern Standard Arabic.

Source	Category	License	Variety	Target n	Use
CIDAR (arbml/CIDAR)	Instruction	CC BY-NC 4.0	MSA	250	Sampled, deduplicated, length-balanced.
XL-Sum (Arabic)	Summarisation	CC BY-NC-SA 4.0	MSA	150	News articles paired with summarisation prompts.
EASC	Summarisation	Research-use	MSA	100	Academic Arabic summarisation corpus.
TyDi-QA (Arabic)	QA	Apache 2.0	MSA	150	Reading comprehension prompts with passages.
ARCD	QA	CC BY-SA 4.0	MSA	100	SQuAD-style Arabic QA prompts.
Aya (Arabic split)	Mixed	Apache 2.0	MSA + dialect	100	Cross-task instructions for diversity.
Manually curated	Creative	Original (CC BY 4.0)	MSA	150	Author-written stories, poems, descriptions, essays.
Total	—	—	—	**1,000**	80/20 dev/test split.

Table 2. AraPromptBench composition

1.3.3 Skills and Software

The project requires consolidation of three skill areas: practical use of the LLMLingua family (acquired through documentation and the vertical-slice integration); Arabic NLP preprocessing (segmentation via Farasa or PyArabic, normalisation conventions); and statistical analysis for paired non-parametric comparisons (Wilcoxon, McNemar, bootstrap confidence intervals). All are within reach given the timetable and my background in machine learning and Python. Implementation is in Python 3.10+ using the following libraries (LLMLingua, transformers, bert-score, pyarabic, openai, pandas, numpy, pytest). Source control uses Git with a private GitHub repository; experiments are tracked through configuration files and CSV result logs for reproducibility. The vertical-slice scaffold (already implemented) demonstrates the pipeline end-to-end on a 20-prompt subset and provides the integration foundation for the Framework Implementation phase.

2 The Design

2.1 Linking the Artefact to the Dissertation Questions

Each component of the system is justified by the research question it serves:

- AraPromptBench operationalises RQ1 by providing the test material on which existing compression methods are evaluated.
- The feature extractor and benchmarking pipeline operationalise RQ2 by enabling correlation analysis between prompt features and compression outcomes.
- The APCS rule-based selector operationalises RQ3 by translating the empirical findings into an explicit, evaluable recommendation system.

2.2 System Architecture

Figure 1 shows the architecture of the system. The pipeline accepts an Arabic prompt and emits a recommended compression method (and the compressed prompt that follows from it). Internally it composes five modules: feature extraction, the APCS decision engine, the compression layer, the LLM, and the evaluation module that supplies the metrics on which the system is benchmarked.

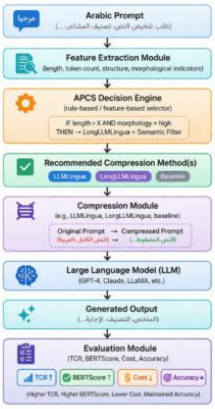


Figure 1. Architecture of the Arabic-Aware Prompt Compression Selector.

2.1 Module Specifications

2.1.1 Feature Extraction Module

Input: a raw Arabic prompt string. Output: a feature vector summarising properties believed to interact with compression performance. Six features are computed: character_length (characters after Unicode normalisation); token_count (tokens under cl100k_base, the gpt-4o-mini tokeniser); fragmentation_ratio (token_count divided by orthographic word count, a tokeniser-agnostic measure of sub-word splitting); structural_complexity (count of structural markers — line breaks, colons, lists, quoted blocks — indicating embedded passages); morphological_density (average Farasa segments per orthographic word, proxying inflectional complexity); and task_category (one of instruction, summarisation, qa, creative, assigned at dataset construction).

2.1.2 APCS Decision Engine

The decision engine is a rule-based selector implemented as an ordered list of guarded clauses; the first matching clause selects the method, with a default fallback for prompts matching no rule. Rules are derived empirically from the development-set benchmark: each rule encodes a regularity observed between feature combinations and the Pareto-optimal compression method. Rules at the SDR stage are illustrative and will be calibrated against benchmark data in APCS Design & Implementation Phase.

```
Algorithm 1. APCS rule-based selector
Input:  prompt p
Output: method m ∈ {LLMLingua, LLMLingua-2, NoCompression}, target ratio
t ∈ (0,1]
1. f ← FeatureExtraction(p)
2. if f.token_count < 80                 return (NoCompression, 1.0)
3. if f.category == "creative" and f.token_count < 250  return
(LLMLingua-2, 0.7)
4. if f.structural_complexity ≥ HIGH and f.token_count ≥ 300  return
(LLMLingua, 0.5)
5. if f.morphological_density ≥ HIGH    return (LLMLingua-2, 0.6)
6. default: return (LLMLingua-2, 0.5)
```

Thresholds (HIGH for structural complexity and morphological density; the constants 80, 250, 300; the target ratios) are placeholders, set on the development set by maximising the per-prompt Pareto-optimality rate.

2.1.3 Compression, LLM, Evaluation, and Interface Modules

Each compression method implements a uniform BaseCompressor interface whose compress(prompt, target_ratio) method returns a CompressionResult with both texts, both token counts, the achieved ratio, latency, and method-specific metadata. Four concrete compressors are implemented: NoOpCompressor (passthrough), LLMLinguaCompressor (perplexity-based), LLMLingua2Compressor (classifier-based), and RandomDeletionCompressor (random baseline). The LLM is accessed through OpenAIClient, which conforms to a BaseLLMClient interface (provider-agnostic; an AnthropicClient or local-model client substitutes with no pipeline changes) and supplies prompt-tokens, completion-tokens, latency, and estimated cost per call. Temperature is fixed at 0.0 and max_tokens at 512. The Evaluation Module computes TCR, prompt-level BERTScore F1, output-level BERTScore F1, and estimated cost using two scorer models (multilingual BERT primary, AraBERT-v02 secondary) for cross-validation, writing results to a timestamped CSV per experiment. APCS exposes two user-facing interfaces: a command-line interface that accepts a prompt and prints the recommended (method, ratio), and a Python library API exposing APCSSelector.recommend(prompt) → Recommendation for programmatic use.

2.2 Data Structures and Calibration

Three principal data structures govern the system. PromptRecord (id, category, text, source, licence, dev/test flag, feature vector) is the unit of AraPromptBench. CompressionResult (defined above) is emitted by every compressor. ExperimentRow flattens a (prompt, method, llm_response_original, llm_response_compressed,

metrics) tuple into a single row in the results CSV. The schema is fixed at the SDR stage and governs all downstream analysis.

Rule calibration (Algorithm 2). On the development set, each prompt is processed by every compression method at each candidate target ratio (0.3, 0.5, 0.7); the achieved TCR and output BERTScore F1 are recorded. For each prompt, the Pareto frontier over (TCR, output F1) is computed and the Pareto-optimal (method, ratio) pair is taken as the prompt's ground-truth label. For each rule slot in Algorithm 1, threshold values are searched to maximise label-matching within that rule's branch. Once thresholds are fixed, rule order is frozen and not adjusted on test data.

2.3 Evaluation Protocol

The evaluation protocol below operationalises every metric and statistical test used in the dissertation. The protocol is fixed before any held-out evaluation is run, to prevent post-hoc selection.

Aspect	Specification
LLM under test	gpt-4o-mini, temperature 0.0, max_tokens 512.
Token compression ratio	Computed using the LLM-native tokenizer (tiktoken cl100k_base).
Semantic fidelity	BERTScore F1 with multilingual BERT (primary) and AraBERT-v02 (sensitivity check).
Fidelity threshold	Acceptable compression: BERTScore F1 ≥ 0.85 between original and compressed prompts.
Output similarity	BERTScore F1 between LLM responses to original vs compressed prompts.
Recommendation accuracy	Percentage of held-out prompts where APCS selects a Pareto-optimal method.
Cost saving	Estimated USD savings vs no-compression, using current OpenAI pricing.
Baselines	(a) no compression; (b) always-LLMLingua; (c) always-LLMLingua-2; (d) random selection.
Statistical tests	Wilcoxon signed-rank for paired method comparisons; McNemar's test for recommendation accuracy vs baselines; bootstrap 95% CIs.
Reporting unit	Per-category breakdown plus aggregate; dev set used for rule derivation, held-out test set for final reporting.

Table 3. Evaluation protocol specifications

Definition of "Pareto-optimal method" for a prompt: among the compressors evaluated, the one whose (TCR, output BERTScore F1) pair is not dominated by any other (no other compressor produces both higher TCR and higher F1). When more than one method is Pareto-optimal for a prompt, APCS is credited if it selects any of them.

2.4 Quality Assurance and Test Strategy

Quality assurance follows three layers. Unit testing covers each module (feature extractor, each compressor wrapper, the decision engine, the metrics functions) via pytest; tests check known invariants such as zero TCR for NoOpCompressor, determinism for RandomDeletionCompressor with seed 42, and graceful handling of empty input. Integration testing uses the vertical-slice pipeline as an end-to-end regression test: a fixed-seed run on the twenty seed prompts must produce a results CSV identical to the committed baseline up to expected numerical tolerance. Validation reserves the held-out test set for final claims only — never used to derive rules. A

reproducibility manifest (Python version, library versions, seeds, model snapshots, dataset hash) is committed with each results CSV.

3 Statement of Deliverables

The project's essential deliverables are AraPromptBench (≈1,000 prompts, 80/20 split), the experimental benchmarking framework, the empirical results (CSV outputs and per-category analyses), the APCS Python package with CLI and library API, the held-out evaluation report, the dissertation document itself, and the source repository with reproducibility manifest. Desirable deliverables include cross-LLM replication on a second model (e.g. claude-haiku-4-5), a trained-classifier extension to APCS contrasted with the rule-based selector, and public release of AraPromptBench with an example usage notebook.

4 Project Plan

The plan below reflects the schedule from the proposal, adjusted for the actual project-initiation date and the SDR submission window. Phase 1 (repository, environment, vertical-slice pipeline) has been completed ahead of the SDR. The plan reserves at least one month before the final dissertation deadline (2 November 2026) for full draft writing and revision, in compliance with the rubric's feasibility criterion.

#	Phase	Window	Deliverable / Milestone
1	Project Initiation & Setup	Week 9–10 (Mar 23 – Apr 5)	Repository scaffold, environment, vertical slice. Completed ✅
2	Literature Review (extended)	Week 11–14 (Apr 6 – May 3)	Annotated bibliography (10+ sources). Completed ✅
3	Specification & Design Report	Week 14–16 (Apr 27 – May 17)	SDR submitted to DA. Completed ✅
4	Dataset Construction (AraPromptBench)	Week 17–20 (May 18 – Jun 14)	~1,000 prompts, 80/20 split, validated.
5	Framework Implementation	Week 21–24 (Jun 15 – Jul 12)	Compression wrappers, evaluation pipeline.
6	Experimental Benchmarking	Week 25–28 (Jul 13 – Aug 9)	Results CSVs, statistical analysis.
7	APCS Design & Implementation	Week 29–32 (Aug 10 – Sep 6)	Rule-based selector + tests.
8	Artefact Evaluation	Week 33–35 (Sep 7 – Sep 27)	Held-out evaluation + recommendation accuracy.
9	Dissertation Writing (draft)	Week 35–38 (Sep 21 – Oct 18)	Full draft to DA ≥1 month before deadline.
10	Revision, Buffer & Final Submission	Week 39–40 (Oct 19 – Nov 2)	Address feedback; final formatting; submit dissertation.

Table 4. Project plan

5 References

Abdelali, A., Darwish, K., Durrani, N. and Mubarak, H. (2016) 'Farasa: A fast and furious segmenter for Arabic', in *Proceedings of the 2016 Conference of the North American Chapter of the Association for Computational Linguistics: Demonstrations*. San Diego, California: Association for Computational Linguistics, pp. 11–16.

Antoun, W., Baly, F. and Hajj, H. (2020) 'AraBERT: Transformer-based model for Arabic language understanding', in *Proceedings of the 4th Workshop on Open-Source Arabic Corpora and Processing Tools, with a Shared Task on Offensive Language Detection*. Marseille, France: European Language Resources Association, pp. 9–15.

Chen, L., Zaharia, M. and Zou, J. (2024) 'FrugalGPT: How to use large language models while reducing cost and improving performance', *Transactions on Machine Learning Research*. Available at: https://openreview.net/forum?id=cSimKw5p6R.

Habash, N.Y. (2010) *Introduction to Arabic Natural Language Processing*. San Rafael, CA: Morgan & Claypool Publishers.

Hevner, A.R., March, S.T., Park, J. and Ram, S. (2004) 'Design science in information systems research', *MIS Quarterly*, 28(1), pp. 75–105.

Jiang, H., Wu, Q., Lin, C.-Y., Yang, Y. and Qiu, L. (2023) 'LLMLingua: Compressing prompts for accelerated inference of large language models', in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. Singapore: Association for Computational Linguistics, pp. 13358–13376.

Jiang, H., Wu, Q., Luo, X., Li, D., Lin, C.-Y., Yang, Y. and Qiu, L. (2024) 'LongLLMLingua: Accelerating and enhancing LLMs in long context scenarios via prompt compression', in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Vol. 1: Long Papers)*. Bangkok, Thailand: Association for Computational Linguistics, pp. 1658–1677.

Limisiewicz, T., Balhar, J. and Mareček, D. (2023) 'Tokenization impacts multilingual language modeling: Assessing vocabulary allocation and overlap across languages', in *Findings of the Association for Computational Linguistics: ACL 2023*. Toronto, Canada: Association for Computational Linguistics, pp. 5661–5681.

Pan, Z., Wu, Q., Jiang, H., Xia, M., Luo, X., Zhang, J., Lin, Q., Rühle, V., Yang, Y., Lin, C.-Y., Awadallah, H.A., Zhang, D. and Qiu, L. (2024) 'LLMLingua-2: Data distillation for efficient and faithful task-agnostic prompt compression', in *Findings of the Association for Computational Linguistics: ACL 2024*. Bangkok, Thailand: Association for Computational Linguistics, pp. 963–981.

Petrov, A., La Malfa, E., Torr, P. and Bibi, A. (2023) 'Language model tokenizers introduce unfairness between languages', in *Advances in Neural Information Processing Systems 36*. New Orleans, USA: Curran Associates, pp. 36963–36990.

Appendix C. Ethical Approval Form

No ethical approval application was required. The approved proposal (Appendix A) records "Ethics Approval Needed? No", and the SDR (Appendix B, "Ethical Implications") confirms the determination. The project involved no human participants, surveys, interviews, personal data or user-generated prompts. All prompts were taken from publicly released research corpora under their stated licences, or were generated synthetically, and LLM API calls contained only these prompts. Nothing in the final design changed this determination.

Appendix D. DETAILED DESIGN

D.1 Feature definitions

Feature	Definition	Used by
character_length	Characters after Unicode NFC normalisation	Analysis
token_count	Tokens under tiktoken cl100k_base	APCS 1.0.0, APCS-v2, APCS-cost
word_count	Whitespace-separated words	Analysis
fragmentation_ratio	token_count / word_count	APCS-v2
structural_complexity	Count of line breaks, colons, quoted blocks, brackets, bullets and enumerations	APCS-v2
morphological_density	Farasa segments per Arabic word (tokens containing an Arabic letter only)	Analysis (not predictive)
has_length_instruction	Prompt contains an Arabic length marker (Section D.5)	APCS-cost
task_category	instruction / summarisation / qa / creative (one-hot)	APCS-v2, APCS-cost

Table 21. APCS feature definitions

D.2 APCS 1.0.0 rules (rules_default.json)

```
version: 1.0.0-final; calibrated on AraPromptBench v2 dev (800 prompts)
LLM under test: gpt-5.6-luna; fidelity threshold tau = 0.65 (AraBERT)
if token_count < 80   -> no compression
if token_count >= 250 -> LLMLingua-2 @ 0.3
otherwise             -> LLMLingua-2 @ 0.5
```

D.3 APCS-v2 decision tree (depth 4, minimum leaf 20; dev 10-fold CV accuracy 43.5%)

```
cat_creative <= 0.5 (not creative)
|  token_count <= 104.5
|  |  fragmentation_ratio <= 4.85
|  |  |  structural_complexity <= 6.5  -> LLMLingua-2 @ 0.5
|  |  |  structural_complexity >  6.5  -> LLMLingua-2 @ 0.7
|  |  fragmentation_ratio >  4.85      -> no compression
|  token_count > 104.5
|  |  not QA:  token_count <= 243      -> LLMLingua-2 @ 0.5
|  |           token_count >  243      -> LLMLingua-2 @ 0.3
|  |  QA:      token_count <= 433      -> LLMLingua-2 @ 0.5
|  |           token_count >  433      -> LLMLingua-2 @ 0.7
cat_creative > 0.5 (creative)
|  token_count <= 85.5                 -> no compression
|  token_count > 85.5
|  |  fragmentation_ratio <= 4.12      -> LLMLingua-2 @ 0.7
|  |  4.12 < fragmentation_ratio <= 4.43 -> LLMLingua-2 @ 0.5
|  |  fragmentation_ratio >  4.43      -> LLMLingua-2 @ 0.7
```

D.4 APCS-cost decision tree (depth 3, minimum leaf 40; dev 10-fold CV cost change −1.9%)

```
token_count <= 127.5                          -> no compression
token_count > 127.5
|  no length instruction:  not QA             -> no compression
|                          QA                 -> LLMLingua-2 @ 0.5
```

```
|  length instruction:     not creative       -> LLMLingua-2-protected @ 0.3
|                          creative           -> LLMLingua-2 @ 0.7
```

(The learned tree contains two further splits, on token count and on morphological density, whose branches all lead to no compression; they are collapsed here.)

D.5 Length-instruction markers

The protected compressor and the has_length_instruction feature use one regular expression over the following Arabic markers: بإيجاز ("briefly"), موجز ("concise"), جملتين ("two sentences"), ثلاث جمل ("three sentences"), جملة واحدة ("one sentence"), فقرة واحدة ("one paragraph"), لا يتجاوز ("not exceeding") and كلمة ("word", as in "fifty words").

D.6 Pre-registration (Experiment 11, committed before the exam run)

Data: 400 never-seen exam prompts with zero exact, 12-word or record overlap with the pilot, v2 or probe sets. Protocol identical to Experiment 10, with $\tau = 0.65$ fixed from Experiment 10. Frozen selectors: APCS 1.0.0 (replication), APCS-v2 (primary) and APCS-cost (extension). Hypotheses, with Holm correction across H1–H3 at $\alpha = 0.05$:

- **H1 (primary):** APCS-v2 best-balance accuracy > always LLMLingua-2 @ 0.5, the best fixed strategy on dev. Exact McNemar; supported if the Holm-adjusted $p < 0.05$ and APCS-v2 is ahead.
- **H2:** APCS-v2 accuracy > APCS 1.0.0. Exact McNemar; same criterion.
- **H3 (extension):** APCS-cost lowers the measured total bill relative to never compressing. One-sided Wilcoxon signed-rank test on per-prompt cost differences; supported if the Holm-adjusted $p < 0.05$ and the total cost change < 0.

Reported regardless of outcome: APCS 1.0.0 replication accuracy; every policy's best-balance and cheapest-faithful accuracy, mean TCR, mean output F1, share below τ, total cost change and QA correctness; APCS-v2 against every other policy (secondary, uncorrected); and accuracy by category. No selector, threshold, label definition or test could be altered

after the exam answers were collected. The full document is `experiments/11_fresh_exam/PREREGISTRATION.md` (commit 318d593).

D.7 Worked example: how compression lengthens the answer

Exam prompt exam-qa-043 (76 cl100k tokens) asks, based on a passage, "Where is the headquarters of Forbes magazine?" and begins "answer briefly" (أجب بإيجاز):

بناءً على الفقرة التالية، أجب بإيجاز: تصنيف: شركات مقرها في جيرسي سيتي. السؤال: اين يقع مقر مجلة فوربس؟

Plain LLMLingua-2 @ 0.5 deleted "briefly" and garbled the question:

الفقرة أجب:شركات جيرسي سيتي مقر مجلة فوربس؟

The protected variant kept the instruction sentence verbatim and compressed the rest:

بناءً على الفقرة التالية، أجب بإيجاز: جيرسي سيتي مقر مجلة فوربس؟

Variant	Billed input tokens	Answer tokens	Call cost (US$ × 10^{-6})	Output F1
Uncompressed	44	23	36.4	1.000
LLMLingua-2 @ 0.5	24	85	106.8	0.638
LLMLingua-2-protected @ 0.5	30	17	26.4	0.768

Table 22. Worked example of plain vs protected compression (exam-qa-043)

Plain compression saved 20 input tokens but the model, no longer told to be brief and given a garbled question, wrote a hedged 85-token answer, so the call cost nearly three times as much. The protected variant saved 14 input tokens, kept the answer short and was the cheapest of the three.

Appendix E. CODE USED TO DEVELOP THE IT ARTEFACT

E.1 Repository and reproduction

All source code, data and results are in the project repository (GitHub: skybadr/CSCK700-Computer-Science-Capstone-Project; access is provided to the examiners, as the module guidance requires). Its structure is:

```
README.md, requirements.txt    start here: overview, quick start and pinned dependencies
AraPromptBench_dataset.json     pilot set (500 prompts, dev-only)
AraPromptBench_v2.json          final benchmark (1,000 prompts, 800 dev / 200 test)
AraPromptBench_exam.json        fresh exam set (400 prompts)
apcs/                           the IT artefact (Python package, tests, README)
experiments/
  01_scorer_selection/ ... 12_compression_overhead/   one folder per experiment:
      README.md (question, method), *.py (scripts),
      results/ (raw outputs, configuration, FINDINGS.md)
  EXPERIMENT_LOG.md             chronological index of all experiments
  AUDIT/                        independent verification of every reported number
```

Reproducing the final results: create a Python 3.11 virtual environment and run pip install -r requirements.txt (the top-level README gives step-by-step instructions), set the OPENAI_API_KEY environment variable, then run `experiments/10_final_benchmark/run_local.py`, `run_api.py` and `run_chain.py`, followed by the Experiment 11 scripts in the order given in its README. Stored responses make every analysis re-runnable offline. `experiments/AUDIT/verify_results.py` re-checks every headline number from the raw data.

E.2 Experiment log

#	Experiment	Outcome
01	Scorer selection	AraBERT primary, mBERT sensitivity check
02	Compression vertical slice	GPT-2 scorer unusable on Arabic; Qwen2.5-0.5B adopted
03	Local benchmark	Prompt-level F1 ranks random above LLMLingua-2
04	Output-level evaluation	Ranking flips at output level
05	Feature analysis (erratum 05e)	Length dominant in pilot; morphology weak
06	APCS calibration (pilot)	41.2% in-sample; Pareto-hit degenerate
07	Protocol checks	Noise ceiling; 0.85 threshold retired
08	AraPromptBench v2	1,000 prompts frozen, 800/200 split
09	Synthetic-data sensitivity	Safe for calibration, not for fidelity measurement
10	Final benchmark	RQ1/RQ2 answered; cost penalty; APCS 1.0.0 31.0% on test
11	Fresh exam	APCS-v2 38.5%; H1–H3 supported
12	Compression overhead (supplementary)	Selector ≈0.05 ms; LLMLingua-2 ≈30 ms (GPU), ≈0.4 s (CPU)

Table 23. Experiment log summary

E.3 Source code of the APCS package

Listing: apcs/apcs/features.py

```
"""Feature extraction for Arabic prompts (APCS Feature Extraction Module).

Mirrors the definitions used throughout the AraPromptBench experiments
(Experiment 05): the same code path produced the calibration data, so
inference-time features are consistent with the rules by construction.
"""

from __future__ import annotations

import re
import unicodedata
from dataclasses import dataclass, asdict

import tiktoken

_ENC = tiktoken.get_encoding("cl100k_base")

_STRUCT_PATTERNS = [
    r"\n",               # line breaks
    r"[:：]",             # colons (list/definition markers)
    r"«[^»]*»",          # quoted blocks
    r"[\(\)\[\]]",       # parentheses/brackets
    r"^\s*[-•*]\s",      # bullets
    r"^\s*\d+[\.\)]\s",  # enumerations
]


# explicit answer-length instructions ("briefly", "in three sentences",
# "no more than N words"); compressors that delete them make answers longer
# (Experiment 10, F5)
LENGTH_MARKERS = re.compile(
    r"بإيجاز|موجز|جملتين|ثلاث جمل|جملة واحدة|فقرة واحدة|لا يتجاوز|كلمة")


@dataclass(frozen=True)
class PromptFeatures:
    character_length: int
    token_count: int
    word_count: int
    fragmentation_ratio: float
    structural_complexity: int
    morphological_density: float | None  # requires optional Farasa/Java
    has_length_instruction: bool = False

    def as_dict(self) -> dict:
        return asdict(self)


def has_length_instruction(text: str) -> bool:
    return bool(LENGTH_MARKERS.search(text))


def structural_complexity(text: str) -> int:
    return sum(len(re.findall(p, text, flags=re.MULTILINE))
               for p in _STRUCT_PATTERNS)


def extract_features(text: str, *, morphology: bool = False) -> PromptFeatures:
    """Compute the APCS feature vector for one prompt.

    morphology=True computes Farasa morphological density (requires the
    'morphology' extra and a Java runtime; adds noticeable latency). The
    shipped decision rules do not use it (Experiment 05: weak predictor),
    so it defaults to off.
    """
    if not isinstance(text, str) or not text.strip():
        raise ValueError("prompt must be a non-empty string")
```

```
    text = unicodedata.normalize("NFC", text)
    words = text.split()
    n_tok = len(_ENC.encode(text))
    density = _morphological_density(text) if morphology else None
    return PromptFeatures(
        character_length=len(text),
        token_count=n_tok,
        word_count=len(words),
        fragmentation_ratio=round(n_tok / len(words), 4),
        structural_complexity=structural_complexity(text),
        morphological_density=density,
        has_length_instruction=has_length_instruction(text),
    )


_ARABIC_LETTER = re.compile(r"[ء-غف-يآ]")


def density_from_segmented(segmented: str) -> float | None:
    """Farasa segments per Arabic word, from one segmented line.

    Farasa emits punctuation and numerals as separate tokens; only tokens
    containing an Arabic letter count as words, otherwise punctuation
    inflates or deflates the measure depending on the denominator."""
    words = [t for t in segmented.split() if _ARABIC_LETTER.search(t)]
    if not words:
        return None
    return round(sum(w.count("+") + 1 for w in words) / len(words), 4)


def _morphological_density(text: str) -> float | None:
    # standalone mode: farasapy's interactive mode corrupts Arabic through
    # the Windows process pipe (AraPromptBench Experiment 05 finding)
    from farasa.segmenter import FarasaSegmenter
    seg = FarasaSegmenter(interactive=False).segment(" ".join(text.split()))
    return density_from_segmented(seg)
```

Listing: apcs/apcs/selector.py

```
"""APCS decision engine.

Three selectors, all calibrated on the AraPromptBench v2 dev split (800
prompts) and frozen as data files:

- "v1"   (APCS 1.0.0, rules_default.json): ordered token-count rules
         (Experiment 10). Needs only the prompt.
- "v2"   (APCS-v2, selectors/apcs_v2.json): depth-4 decision tree over the
         feature vector and the task category, maximising best-balance
         accuracy (Experiment 11, H1/H2 supported on the fresh exam).
- "cost" (APCS-cost, selectors/apcs_cost.json): depth-3 tree choosing the
         cheapest strategy that keeps output fidelity above tau, including
         the length-protected LLMLingua-2 variant (Experiment 11, H3).

The tree selectors need the prompt's task category, one of
instruction / summarisation / qa / creative.
"""

from __future__ import annotations

import json
from dataclasses import dataclass
from importlib import resources
from pathlib import Path

from .features import PromptFeatures, extract_features

CATEGORIES = ("instruction", "summarisation", "qa", "creative")
_TREE_FILES = {"v2": "apcs_v2.json", "cost": "apcs_cost.json"}
_METHOD_NAMES = {"noop": "none", "llmlingua2": "llmlingua2",
                 "llmlingua2_protected": "llmlingua2_protected",
                 "llmlingua_qwen": "llmlingua"}
```

```
@dataclass(frozen=True)
class Recommendation:
    method: str          # "none" | "llmlingua2" | "llmlingua2_protected" | "llmlingua"
    rate: float          # target keep-rate (1.0 for "none")
    rule: str            # human-readable guard(s) that fired
    features: PromptFeatures
    rules_version: str

    def as_dict(self) -> dict:
        d = {"method": self.method, "rate": self.rate, "rule": self.rule,
             "rules_version": self.rules_version}
        d["features"] = self.features.as_dict()
        return d



def _load(rules_path, package_file):
    if rules_path is not None:
        return json.loads(Path(rules_path).read_text(encoding="utf-8"))
    src = resources.files("apcs")
    for part in package_file.split("/"):
        src = src.joinpath(part)
    return json.loads(src.read_text(encoding="utf-8"))



def tree_inputs(f: PromptFeatures, category: str) -> dict:
    """Feature dictionary in the form the frozen trees were trained on."""
    if category not in CATEGORIES:
        raise ValueError(f"category must be one of {CATEGORIES}")
    x = {"token_count": f.token_count,
         "structural_complexity": f.structural_complexity,
         "fragmentation_ratio": f.fragmentation_ratio,
         "morphological_density": f.morphological_density,
         "has_length_instruction": int(f.has_length_instruction)}
    x.update({f"cat_{c}": int(c == category) for c in CATEGORIES})
    return x



def walk_tree(node: dict, x: dict) -> tuple[str, list[str]]:
    """Return (leaf label, path of guards). A missing feature value (only
    morphological density, which is optional) follows the <= branch; in the
    shipped trees both of its branches lead to the same strategy."""
    path = []
    while "leaf" not in node:
        name, thr = node["feature"], node["threshold"]
        v = x[name]
        if v is None or v <= thr:
            path.append(f"{name} <= {thr:g}")
            node = node["le"]
        else:
            path.append(f"{name} > {thr:g}")
            node = node["gt"]
    return node["leaf"], path



class APCSSelector:
    """Arabic prompt-compression selector.

    >>> APCSSelector().recommend("...").method                    # APCS 1.0.0
    >>> APCSSelector("v2").recommend("...", category="qa").method  # APCS-v2
    """

    def __init__(self, selector: str = "v1", rules_path: str | Path | None = None):
        if selector not in ("v1", "v2", "cost"):
            raise ValueError("selector must be 'v1', 'v2' or 'cost'")
        self.selector = selector
        if selector == "v1":
            cfg = _load(rules_path, "rules_default.json")
            self.version = cfg["version"]
            self.tau = cfg["fidelity_threshold_tau"]
            t = cfg["thresholds"]
            self.t1, self.t2 = t["T1"], t["T2"]
            self.r_mid, self.r_long = t["r_mid"], t["r_long"]
        else:
            cfg = _load(rules_path, "selectors/" + _TREE_FILES[selector])
            self.version = cfg["name"]
            self.tau = cfg["tau"]
```

```
            self.tree = cfg["tree"]

    def recommend(self, prompt: str, category: str | None = None) -> Recommendation:
        f = extract_features(prompt)
        if self.selector == "v1":
            method, rate, rule = self._rules_v1(f)
        else:
            if category is None:
                raise ValueError(f"selector '{self.selector}' needs the task "
                                 f"category, one of {CATEGORIES}")
            leaf, path = walk_tree(self.tree, tree_inputs(f, category))
            name, r = leaf.split("@")
            method, rate, rule = _METHOD_NAMES[name], float(r), " and ".join(path)
        return Recommendation(method=method, rate=rate, rule=rule,
                              features=f, rules_version=self.version)

    def _rules_v1(self, f):
        if f.token_count < self.t1:
            return "none", 1.0, f"token_count {f.token_count} < {self.t1}"
        if f.token_count >= self.t2:
            return ("llmlingua2", self.r_long,
                    f"token_count {f.token_count} >= {self.t2}")
        return ("llmlingua2", self.r_mid,
                f"{self.t1} <= token_count {f.token_count} < {self.t2}")

    def compress(self, prompt: str, category: str | None = None) -> str:
        """Recommend and apply (requires the 'compression' extra)."""
        rec = self.recommend(prompt, category)
        if rec.method == "none":
            return prompt
        from llmlingua import PromptCompressor  # lazy heavy import
        if rec.method == "llmlingua":
            pc = PromptCompressor(model_name="Qwen/Qwen2.5-0.5B")
            return pc.compress_prompt(prompt, rate=rec.rate)["compressed_prompt"]
        pc = PromptCompressor(
            model_name="microsoft/llmlingua-2-xlm-roberta-large-meetingbank",
            use_llmlingua2=True)
        if rec.method == "llmlingua2_protected":
            from .protect import compress_protected
            return compress_protected(pc, prompt, rec.rate)
        return pc.compress_prompt(prompt, rate=rec.rate)["compressed_prompt"]
```

Listing: apcs/apcs/protect.py

```
"""LLMLingua-2 with length instructions protected (Experiment 11).

Splits the prompt into sentence-like segments, keeps every segment that
contains a length instruction verbatim, compresses the remaining text with
LLMLingua-2 at the target rate and reassembles it in the original order.
Prompts without a length instruction are compressed exactly as plain
LLMLingua-2 would compress them.
"""

from __future__ import annotations

import re

from .features import LENGTH_MARKERS

# split after sentence punctuation, colons and newlines, keeping delimiters
_SPLIT = re.compile(r"(?<=[.?!:\n])")



def compress_protected(pc, text: str, rate: float) -> str:
    """pc: an llmlingua PromptCompressor configured for LLMLingua-2."""
    if not LENGTH_MARKERS.search(text):
        return pc.compress_prompt(text, rate=rate)["compressed_prompt"]
    out, buffer = [], []

    def flush():
        if buffer:
            chunk = "".join(buffer)
            out.append(pc.compress_prompt(chunk, rate=rate)["compressed_prompt"]
                       if chunk.strip() else chunk)
            buffer.clear()
```

```
for seg in (s for s in _SPLIT.split(text) if s.strip()):
    if LENGTH_MARKERS.search(seg):
        flush()
        out.append(seg.strip())
    else:
        buffer.append(seg)
flush()
return " ".join(s for s in out if s)
```